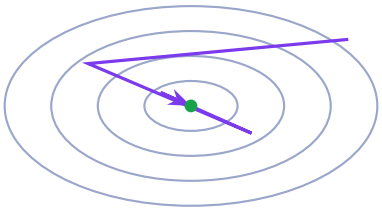
EN3330
Engineering Optimization

Study Notes - Part 1



University of Moratuwa
Semester 7

Chapters 1–6

Formulation · Geometry and optimality · Search methods
First- and second-order algorithms · Python · Stochastic methods

A self-contained guide to mathematical formulation, numerical algorithms, and practical implementation.

Contents

How to Use These Notes **vii**

1 Introduction to Engineering Optimization **1**

1.1 Administrative Information . . . 1

1.1.1 Learning outcomes . . . 1

1.1.2 Module outline . . . 2

1.1.3 Evaluation criterion . . . 2

1.1.4 Textbooks and software tools . . . 2

1.2 What is Optimization? . . . 3

1.2.1 "Best" can vary . . . 3

1.2.2 Why mathematical optimization? . . . 4

1.3 Types of Optimization Problems . . . 4

1.4 The Standard Form of an Optimization Problem . . . 5

1.4.1 Constrained optimization problem . . . 5

1.4.2 Where do decision variables and constraints come from? . . . 6

1.5 Unconstrained Optimization . . . 6

1.6 Constrained Optimization and the Feasible Region . . . 8

1.6.1 Drawing a feasible region . . . 8

1.7 Engineering Examples of Optimization . . . 9

1.7.1 Minimizing total transmitting power . . . 9

1.7.2 Total variation inpainting . . . 10

1.7.3 Support vector machine classifier . . . 11

1.7.4 Further application areas . . . 13

1.8 Basic Concepts: Minima and Optimality Conditions . . . 13

1.8.1 Global vs local minimum . . . 13

1.8.2 Necessary conditions for a local minimum . . . 15

1.9 Convex vs Non-Convex Optimization . . . 16

1.10 Key Takeaways . . . 17

1.11 References . . . 17

2 Optimization - Fundamental Concepts **18**

2.1 Notation . . . 18

2.2 Recap: The Optimization Problem . . . 19

2.3 Formulation Exercise: The Shopping Problem . . . 19
2.4 Recap Examples: Unconstrained and Constrained . . . 20
2.4.1 The “are these the same problem?” question . . . 21
2.5 Graph of a Function . . . 21
2.5.1 Single-variable case . . . 21
2.5.2 Two-variable case . . . 22
2.6 Level Sets and Contour Plots . . . 22
2.7 Implications in the Gradient . . . 24
2.7.1 Gradient of a single-variable function . . . 24
2.7.2 Gradient of a multi-variable function . . . 25
2.8 The Hessian . . . 26
2.8.1 Worked examples . . . 27
2.9 Directional Derivatives . . . 27
2.10 The Hessian and Optimality (Unconstrained Problems) . . . 28
2.10.1 Why FONC alone is not enough . . . 28
2.10.2 Why SONC is also not enough . . . 30
2.10.3 The Second-Order Sufficient Condition (SOSC) . . . 30
2.11 Classifying Critical Points - Practice Set . . . 31
2.12 Numerical Differentiation . . . 32
2.12.1 Finite difference methods . . . 33
2.12.2 The Complex Step Method . . . 33
2.13 Resources . . . 35
2.14 Key Takeaways . . . 36

3 Solution Methods for Unconstrained Optimization Problems 37
3.1 Recap and Two Formulation Exercises . . . 37
3.1.1 Exercise A - the cardboard box . . . 37
3.1.2 Exercise B - the largest triangle in an ellipse . . . 38
3.2 How Optimization Problems Get Solved . . . 39
3.2.1 Vocabulary for one-D methods . . . 39
3.3 Bracketing . . . 40
3.3.1 The bracketing condition . . . 40
3.3.2 Finding the initial bracket . . . 41
3.3.3 Worked example . . . 42
3.3.4 When bracketing fails . . . 43
3.4 Fibonacci Search . . . 44
3.4.1 The two-query rule . . . 44

3.4.2 The Fibonacci sequence . . . 44
3.4.3 Why Fibonacci numbers are the right lengths . . . 45
3.4.4 The last-iteration problem . . . 46
3.4.5 Choosing the number of queries N . . . 47
3.5 Golden Section Search . . . 47
3.6 The Bisection Method . . . 49
3.6.1 Using bisection for optimization . . . 50
3.7 Descent Direction - The Multivariate Framework . . . 51
3.8 Line Search . . . 51
3.8.1 Worked example . . . 52
3.8.2 Exercises . . . 53
3.9 Approximate Line Search and the Wolfe Conditions . . . 53
3.9.1 The first Wolfe condition (sufficient decrease / Armijo) . . . 54
3.9.2 Backtracking line search . . . 54
3.9.3 The second Wolfe condition (curvature) . . . 55
3.9.4 Worked example - backtracking through the first Wolfe condition . . . 56
3.9.5 Exercises . . . 57
3.10 Newton's Method . . . 58
3.10.1 Geometric interpretation: quadratic approximation . . . 59
3.10.2 When Newton's method goes wrong . . . 60
3.10.3 Convergence . . . 61
3.10.4 Worked examples . . . 61
3.11 The Secant Method and Quasi-Newton (DFP) . . . 63
3.11.1 Quasi-Newton methods . . . 64
3.12 Trust-Region Methods . . . 64
3.12.1 Adjusting the trust-region radius . . . 66
3.12.2 Worked example - where Newton stalls and trust region does not . . . 66
3.13 Termination Conditions of Iterative Algorithms . . . 67
3.14 Method Comparison . . . 68
3.15 Key Takeaways . . . 69

4 Unconstrained Optimization - First and Second Order Methods 70
4.1 Choosing Directions and Step Sizes . . . 70
4.2 Gradient Descent . . . 71
4.2.1 Computing the steepest-descent direction . . . 71
4.2.2 A full run . . . 72
4.2.3 Exercise: termination conditions . . . 73

4.3 The Step Size (Learning Rate) α . . . 73
4.3.1 The effect of conditioning . . . 74
4.3.2 Narrow valleys . . . 75
4.4 Conjugate Gradient . . . 76
4.4.1 Deriving the exact step size . . . 76
4.4.2 Deriving β - the conjugacy condition . . . 76
4.4.3 Practical choices for β . . . 77
4.5 Momentum . . . 78
4.5.1 Nesterov momentum . . . 78
4.6 Adaptive Step Sizes . . . 79
4.6.1 Adagrad - Adaptive sub-gradient method . . . 79
4.6.2 RMSProp . . . 80
4.6.3 Adadelta . . . 81
4.6.4 Adam - Adaptive moment estimation . . . 81
4.6.5 Hypergradient descent . . . 82
4.7 First-Order Methods - Collected Update Rules . . . 82
4.8 The Head-to-Head Comparison . . . 83
4.9 Second-Order Methods . . . 86
4.9.1 Newton's method . . . 86
4.9.2 Quasi-Newton methods . . . 86
4.9.3 Rank-one correction - the derivation . . . 87
4.9.4 DFP - Davidon–Fletcher–Powell . . . 88
4.9.5 BFGS and L-BFGS . . . 89
4.10 Second-Order Methods on the Test Problem . . . 89
4.10.1 Descent direction - the three families side by side . . . 90
4.11 Summary - One Formula for Everything . . . 90
4.12 Key Takeaways . . . 92

5 Coding Examples - Unconstrained Optimization in Python 93
5.1 Environment and Tooling . . . 93
5.2 Bracketing - `scipy.optimize.bracket` . . . 94
5.3 Bisection - `scipy.optimize.bisect` . . . 95
5.4 Line Search - `scipy.optimize.line_search` . . . 97
5.5 Newton's Method - `scipy.optimize.newton` . . . 98
5.6 Multivariate Newton - `minimize(method='Newton-CG')` . . . 100
5.6.1 Hessian-vector products - `hessp` . . . 101
5.7 Quasi-Newton - `minimize(method='BFGS')` . . . 102

5.8 Trust Region - `minimize(method='trust-ncg')` . . . 103
5.9 First-Order Methods in PyTorch . . . 105
5.9.1 A convex example . . . 106
5.9.2 A multimodal example . . . 107
5.10 Quick Reference - Theory to Code . . . 107
5.11 Key Takeaways . . . 109

6 Stochastic Methods for Unconstrained Optimization 110
6.1 Recap - Gradient Methods and the Orthogonality Property . . . 110
6.2 Why Stochastic Methods? Exploration vs Exploitation . . . 112
6.3 Noisy Descent . . . 113
6.3.1 Worked demonstration: the saddle $f(\boldsymbol{x}) = x_1^2 - x_2^2$ . . . 114
6.4 Stochastic Gradient Descent (SGD) . . . 115
6.4.1 GD vs SGD on a quadratic . . . 116
6.4.2 The saddle again: $f(\boldsymbol{x}) = (x_1^2 - 1)^2 + x_2^2$ . . . 117
6.5 Interlude - Information Theory . . . 118
6.5.1 What is information? . . . 118
6.5.2 Self-information . . . 118
6.5.3 Entropy . . . 119
6.5.4 Conditional entropy, joint entropy, mutual information . . . 120
6.5.5 Kullback–Leibler divergence and cross-entropy . . . 121
6.6 The Cross-Entropy Method . . . 122
6.6.1 The update rule . . . 122
6.6.2 Elite samples and maximum likelihood . . . 123
6.6.3 A concrete parametrisation . . . 124
6.6.4 The multivariate normal proposal . . . 124
6.6.5 Pseudocode . . . 125
6.7 Cross-Entropy in Action: $f(x) = x^2$ . . . 126
6.7.1 Good exploration: $\mu_0 = 20$, $\sigma_0^2 = 70$, $N = 100$, $N_e = 10$ . . . 126
6.7.2 Poor exploration: the same problem with $\sigma_0^2 = 20$ . . . 127
6.8 Cross-Entropy on Multimodal Problems . . . 128
6.8.1 The Branin function . . . 128
6.8.2 The Rastrigin function . . . 129
6.8.3 Cross-entropy vs SGD on Rastrigin . . . 130
6.9 Comparison of All Three Stochastic Methods . . . 131
6.10 Key Takeaways . . . 132

A Revision Reference 133

A.1 Formulation and Optimality 133
A.2 Directions and Step Sizes 133
A.3 Deterministic Update Rules 135
A.4 Adaptive and Stochastic Methods 136

How to Use These Notes

Read the overview at the start of each chapter, follow the worked examples, and use the self-check questions to test recall. Blue boxes explain concepts, navy boxes collect formulas, purple boxes show examples, and amber or red boxes flag assumptions and pitfalls.

The chapters develop optimization from formulation and geometric reasoning through deterministic and stochastic algorithms. Each method includes its update rule, assumptions, worked examples, and practical limitations. The revision appendix collects the main formulas.

Figures are included next to the concepts they illustrate. Numerical solver transcripts are illustrative: exact counts and last digits can vary with software versions and tolerances. Stochastic results also depend on the random seed.

Chapter 1

Introduction to Engineering Optimization

Overview

Optimization is the process of choosing the *best* option from a set of available alternatives, where "best" is defined by an **objective** and restricted by **constraints**. This chapter establishes the vocabulary and mathematical framework of optimization: decision variables, objective functions, constraint functions, feasible regions, optimal solutions, and the conditions that characterise a minimum.
Core idea: every engineering design problem can be written as "minimize this quantity, subject to these restrictions" - and once written in that standard form, a large library of general-purpose algorithms becomes available to solve it.

1.1 Administrative Information

Module details

- **Course:** EN3330 - Introduction to Engineering Optimization
- **Lecturer:** Dr. Sampath K. Perera, Department of Electronic & Telecommunication Engineering, University of Moratuwa
- **Contact:** sampathk@uom.lk · Room EB 111 · Office hours Tuesdays 13.15–15.00
- **Credits:** 3 (one credit = 50 notional hours of study - lecture hours + labs and projects + self-learning)
- **Lecture time:** Wednesday 8.15–10.15 a.m.
- **LearnOrg (Moodle)** is the primary source for announcements, lecture notes, and assignments.

1.1.1 Learning outcomes

#	Outcome
LO1	Recognize the concept of optimality with respect to objective, decision variables, and constraints.
LO2	Mathematically formulate certain design problems in engineering using the decision variables, objective, and constraints framework.
LO3	Rationalize the disparity between an **optimal** design and a **feasible** design.
LO4	Propose solution methods to an optimization problem by capitalizing on off-the-shelf algorithms.
LO5	Implement the solution methods numerically and compare them.

1.1.2 Module outline

Topic (hours)	Content	LOs
Optimization Process with Examples (4 h)	Shannon capacity as an optimization problem, minimum-energy signal design in digital communication, interference mitigation via power control, determining the shortest path in a communication network, processor–task assignment problem in multiprocessor environments, synthesis of the control command for state targeting, transistor sizing for digital CMOS circuits, digital filter designs by reference matching, machine learning (optimal separating hyperplane, binary classification, matrix completion problem, non-negative matrix factorization), image denoising	LO1, LO2
Fundamental Characteristics of an Optimization Problem (2 h)	Decision variables, objective function, constraint functions, basic optimization problem, unconstrained optimization, constrained optimization, solution of a problem, a feasible point of an optimization problem	LO1, LO2, LO3
Euclidean Geometric Interpretation of an Optimization Problem (2 h)	Graph of a function, level sets of a function, implications in gradient, directional derivatives, and Hessian on optimality	LO1, LO3
Solution Algorithms for Unconstrained Optimization Problems (10 h)	One-D search methods (bisection, Newton's method, bracketing, line search in multidimensional optimization), descent/ascent methods (steepest descent, gradient descent, Newton's method, cyclic/block coordinate search), conjugate direction methods (conjugate direction/gradient algorithms), quasi-Newton methods (DFP algorithm, BFGS algorithm)	LO4, LO5
Introduction to Stochastic Methods for Unconstrained Optimization Problems (3 h)	Noisy descent, stochastic gradient, cross-entropy method	LO4, LO5
Solution Algorithms for Constrained Optimization Problems (7 h)	Equality constrained (e.g. Newton's method, infeasible start Newton method, method of Lagrange multipliers)	LO4, LO5

1.1.3 Evaluation criterion

Assessment breakdown

Continuous Assessment - 70 %

	Assessment Component	Weight (%)
A1	Assignment 1	10
A2	Assignment 2	15
A3	Mid-term Exam (closed book, in-person)	30
A4	Assignment 3	15

Final Written Examination (closed book, in-person) - **30 %**

1.1.4 Textbooks and software tools

Textbooks

- Kochenderfer, Mykel J., and Tim A. Wheeler. *Algorithms for Optimization*. MIT Press, 2019.
- S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge Univ. Press, 2006.
- A. Antoniou, W. S. Lu. *Practical Optimization: Algorithms and Engineering Applications*. Springer, 2017.

Software and toolboxes

- **CVX** - MATLAB software for disciplined convex programming
- **CVXOPT** - convex optimization based on the Python programming language
- **MATLAB Optimization Toolbox**
- **SciPy**

Example libraries: https://web.cvxr.com/cvx/examples/index.html and https://www.cvxpy.org/examples/

1.2 What is Optimization?

Origin of the word

"Optimization" comes from the Latin word **optimus**, meaning **"best"**. When you optimize something, you are essentially making it the best.

Two working definitions used in the course:

- *"Doing the most with the least."*
- *"The process of finding the most effective or favorable value or condition."*

1.2.1 "Best" can vary

Choosing a route from A to B

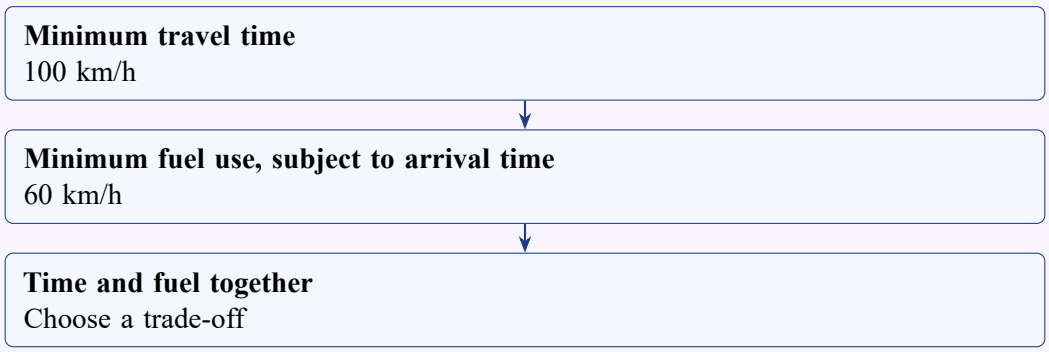
A map application offers several routes and modes of travel. Which option is "best" depends entirely on **what you are optimizing for**:

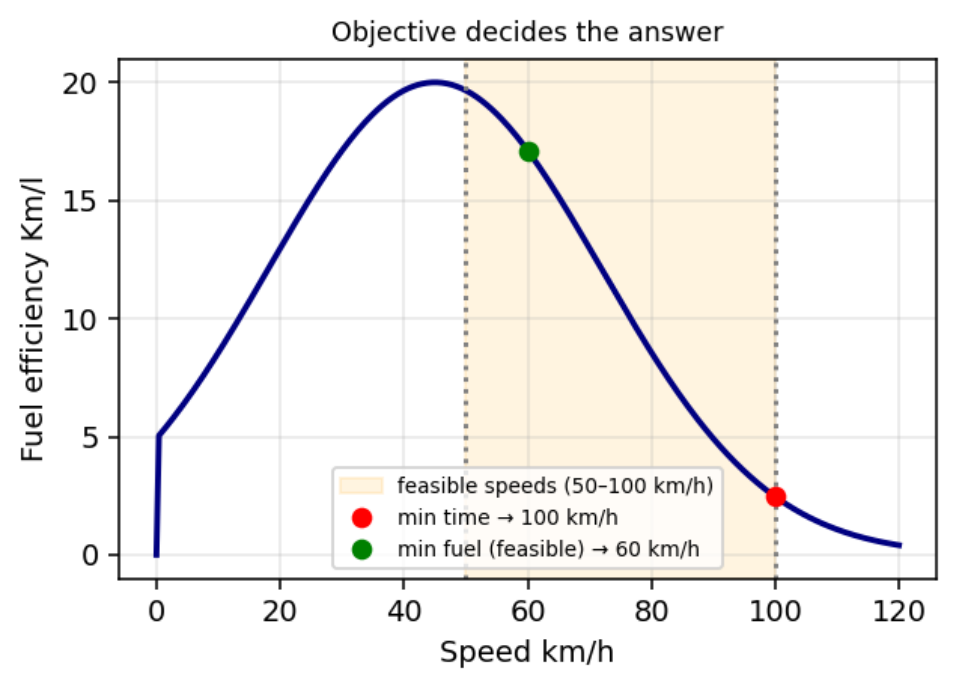
Choice	Criterion being optimized
Motorcycle	Time
Car	Comfort
Walking	Well-being

There is no universal "best" - "best" is meaningless until an objective is stated.

The optimal-speed problem

You must travel from A to B in **60 minutes or less**. The distance is **60 km**. The highway has a **maximum speed limit of 100 km/h** and a **minimum of 50 km/h**. Fuel efficiency (km/l) rises with speed, peaks around 40–60 km/h, then falls.





The answer **depends on your objective(s) and constraints**. With two competing objectives there is no single obvious answer - this is the essence of engineering trade-off.

1.2.2 Why mathematical optimization?

Motivation

1. To **improve decision making**.
2. **Efficient resource utilization**.
3. It serves as the **foundation for many advanced topics**, including artificial intelligence, robotics, telecommunication, bioinformatics and many more.
4. **Enhance quality of life**.

1.3 Types of Optimization Problems

Three axes of classification

1. **Constrained vs unconstrained** - constraints may exist in certain problems, while others may not have any (unconstrained).
2. **Univariate vs multivariate** - problems may involve either one variable (univariate) or multiple variables.
3. **Discrete vs continuous** - variables can be discrete, taking only integer values, or continuous.

Constraints
Constrained / unconstrained

↓

Number of variables
Univariate / multivariate

↓

Variable domain
Discrete / continuous

Effect of restricting the feasible domain

For the objective $f(x) = (x-2)^2$, the unconstrained minimizer is $x = 2$, with value zero. Restricting the feasible domain to $0 \leq x \leq 1$ moves the minimizer to $x = 1$, with value one. The objective is unchanged; the admissible choices determine which minimum is attainable.

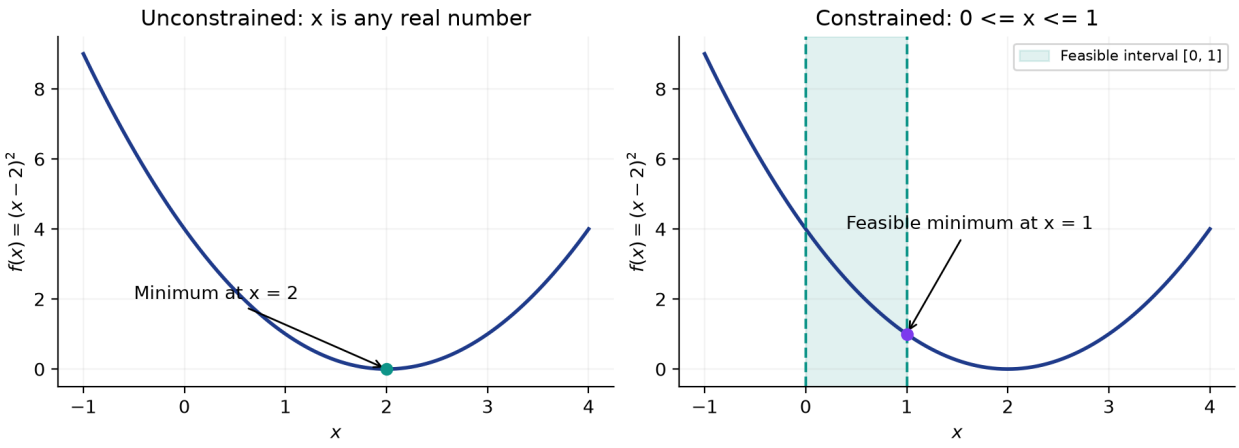


Figure 1.1: The same quadratic objective on an unrestricted domain and on the feasible interval $[0, 1]$.

1.4 The Standard Form of an Optimization Problem

1.4.1 Constrained optimization problem

Standard form

$$\begin{aligned} \underset{\boldsymbol{x}}{\text{minimize}} \quad & f_0(\boldsymbol{x}) \\ \text{subject to} \quad & f_i(\boldsymbol{x}) \leq b_i, \quad i = 1, \ldots, m \\ & g_i(\boldsymbol{x}) = 0, \quad i = 1, \ldots, n \end{aligned}$$

Each ingredient:

Ingredient	Symbol / type	Meaning
Objective function	$f_0 : \mathbb{R}^n \to \mathbb{R}$	The function you try to minimize or maximize
Optimization (decision) variables	$\boldsymbol{x} = [x_1 \ \cdots \ x_n]^T \in \mathbb{R}^n$	The quantities that need to be chosen
Inequality constraint functions	$f_i : \mathbb{R}^n \to \mathbb{R},\ i = 1, \ldots, m$	Restrictions of the form $f_i(\boldsymbol{x}) \leq b_i$
Equality constraint functions	$g_i : \mathbb{R}^n \to \mathbb{R},\ i = 1, \ldots, n$	Restrictions of the form $g_i(\boldsymbol{x}) = 0$
Optimal solution	$\boldsymbol{x}^*$	Has the smallest value of f_0 among all vectors satisfying the given constraints

Optimal solution as an argmin

$$\begin{aligned} \boldsymbol{x}^* = \arg\min_{\boldsymbol{x}} \quad & f_0(\boldsymbol{x}) \\ \text{subject to} \quad & f_i(\boldsymbol{x}) \leq b_i, \quad i = 1, \ldots, m \\ & g_i(\boldsymbol{x}) = 0, \quad i = 1, \ldots, n \end{aligned}$$

Formally, $\boldsymbol{x}^*$ is optimal if $f_i(\boldsymbol{x}^*) \leq b_i$ for all i, $g_i(\boldsymbol{x}^*) = 0$ for all i, and no other vector satisfying these gives a smaller f_0.

Optimality and feasibility

- **Feasible points** are a set of values for the decision variables that satisfy **all** the constraints of the optimization problem.
- The **feasible region** is the set of **all** points satisfying the constraints. It represents all possible values of the decision variables for which the problem is defined and constraints are not violated.
- An **optimal point** is a feasible point that additionally achieves the **smallest objective value**.

Every optimal design is feasible; most feasible designs are not optimal.

1.4.2 Where do decision variables and constraints come from?

- **Decision variables** $\boldsymbol{x} = [x_1 \ \cdots \ x_n]^T \in \mathbb{R}^n$ are the **parameters of the model** - they represent the *actions* you can take.
- **Constraints restrict actions or set conditions on the outcome.**

Five sources of constraints

1. **Physical limitations** (e.g. a speed limit sign, a maximum meter reading)
2. **Resource availability**
3. **Regulatory and legal requirements**
4. **Environmental factors**
5. **Financial factors**

1.5 Unconstrained Optimization

Unconstrained problem

$$\underset{\boldsymbol{x}}{\text{minimize}} \quad f_0(\boldsymbol{x})$$

with objective function $f_0(\boldsymbol{x}) : \mathbb{R}^n \to \mathbb{R}$ and optimization variable $\boldsymbol{x} \in \mathbb{R}^n$, $\boldsymbol{x} = (x_1, x_2, \ldots, x_n)$.
Goal: minimize $f_0(\boldsymbol{x})$ over the entire space - nothing forbids any point.

Example 1 - univariate unconstrained

$$f_0(x) : \mathbb{R} \to \mathbb{R}, \qquad f_0(x) = x_1^2 - 2x_1 + 2$$

This is a **univariate function** (involves only a single variable). Completing the square: $f_0 = (x_1 - 1)^2 + 1$, so

$$x_1^* = 1, \qquad f_0(x_1^*) = 1$$

The optimal solution is denoted x_1^*. The plot is a parabola with its single minimum marked at $(1, 1)$.

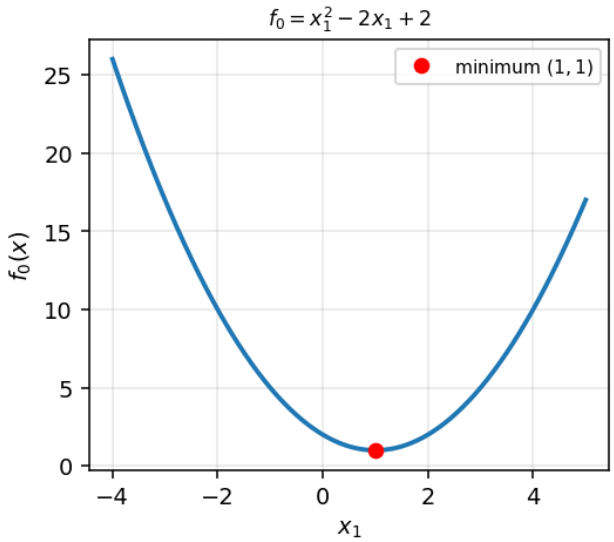
Example 2 - multivariate unconstrained

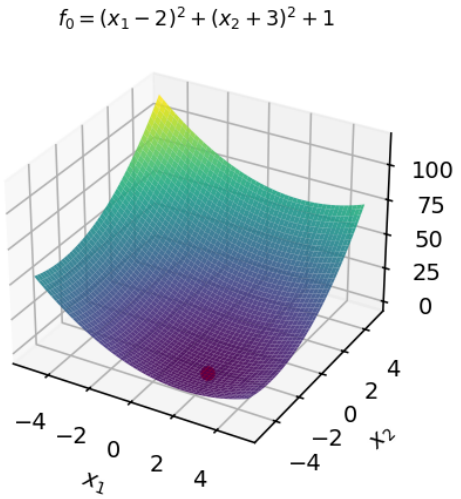
$$\underset{\boldsymbol{x}\in\mathbb{R}^2}{\text{minimize}} \quad f_0(\boldsymbol{x}) = (x_1 - 2)^2 + (x_2 + 3)^2 + 1$$

Each squared term is minimized independently by setting it to zero:

$$x_1^* = 2, \quad x_2^* = -3 \implies \boldsymbol{x}^* = \begin{bmatrix} 2 & -3 \end{bmatrix}^T, \qquad f_0(\boldsymbol{x}^*) = 1$$

The surface plot is a bowl (paraboloid) whose lowest point sits at $(2, -3)$ with height 1.





Left: the univariate parabola of Example 1 with its single minimum at $(1, 1)$. *Right: the bivariate bowl of Example 2, lowest at* $(2, -3)$.

1.6 Constrained Optimization and the Feasible Region

Constrained problem (restated)

$$\begin{aligned} \underset{\boldsymbol{x}}{\text{minimize}} \quad & f_0(\boldsymbol{x}) \\ \text{subject to} \quad & f_i(\boldsymbol{x}) \leq b_i, \quad i = 1, \ldots, m \\ & g_i(\boldsymbol{x}) = 0, \quad i = 1, \ldots, n \end{aligned}$$

where $f_i(\boldsymbol{x})$ are inequality constraint functions and $g_i(\boldsymbol{x})$ are equality constraint functions.

Strict vs non-strict inequalities

For the constraint $4 > x_1 > 2$ the feasible set is the **open** interval - with **strict inequalities ($>$ or $<$) the feasible set does not include the constraint boundary**. This matters: a problem whose minimum "wants" to sit on the boundary may then have **no solution at all** (see Example 5).

1.6.1 Drawing a feasible region

Example 3 - sketching a feasible region

Draw the feasible region for $x_1 \geq 0$, $x_2 \geq 0$, and $x_1 + x_2 \leq 2$.
Build it up one constraint at a time:

Nonnegative coordinates
$x_1 \geq 0, \; x_2 \geq 0$

↓

Intersect with half-plane
$x_1 + x_2 \leq 2$

↓

Feasible triangle
$(0,0), \; (2,0), \; (0,2)$

The intersection of the half-planes is the shaded triangle with vertices $(0,0)$, $(2,0)$ and $(0,2)$.

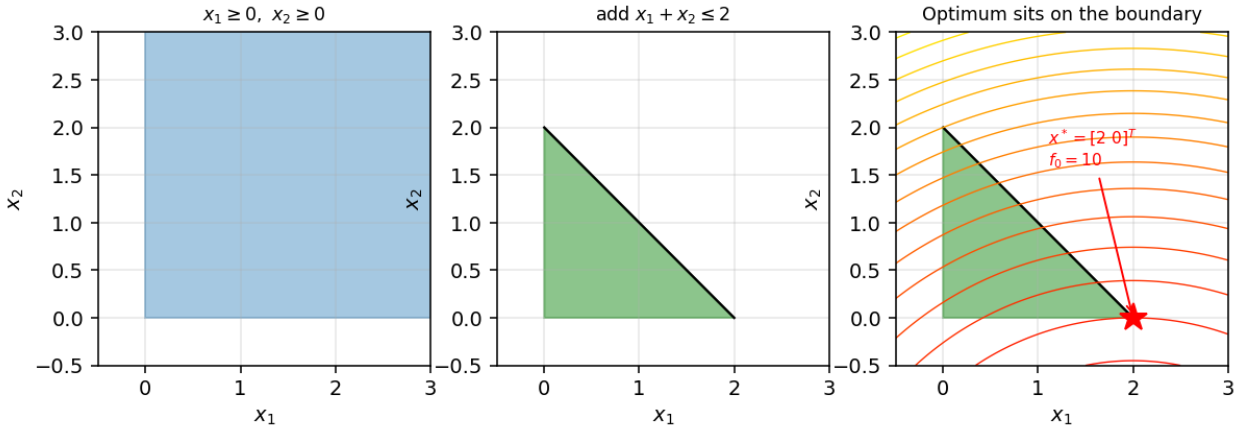
Example 4 - constrained version of the bowl

$$\begin{aligned} \underset{\boldsymbol{x} \in \mathbb{R}^2}{\text{minimize}} \quad & f_0(\boldsymbol{x}) = (x_1 - 2)^2 + (x_2 + 3)^2 + 1 \\ \text{subject to} \quad & f_1(\boldsymbol{x}) = (x_1 + x_2) \leq 2 \\ & f_2(\boldsymbol{x}) = -x_1 \leq 0 \\ & f_3(\boldsymbol{x}) = -x_2 \leq 0 \end{aligned}$$

i.e. $x_1 \geq 0$, $x_2 \geq 0$, $x_1 + x_2 \leq 2$ - the same triangle as above.
The unconstrained minimum sat at $(2, -3)$, which violates $x_2 \geq 0$. Pushed back into the feasible set:

$$x_1^* = 2, \quad x_2^* = 0 \implies \boldsymbol{x}^* = \begin{bmatrix} 2 & 0 \end{bmatrix}^T, \qquad f_0(\boldsymbol{x}^*) = 10$$

Note how the constraints moved the solution from the interior onto the boundary and raised the optimal value from 1 to 10 - this is the *cost of feasibility*.



Building the feasible region one constraint at a time, then overlaying the objective's contours. The unconstrained minimum at $(2, -3)$ *lies outside the triangle, so the best feasible point is pushed onto the boundary at* $[2\ 0]^T$.

Example 5 - when strict inequalities destroy the solution

$$\begin{aligned} \underset{\boldsymbol{x}\in\mathbb{R}^2}{\text{minimize}} \quad & f_0(\boldsymbol{x}) = (x_1 - 2)^2 + (x_2 + 3)^2 + 1 \\ \text{subject to} \quad & f_2(\boldsymbol{x}) = -x_1 < 0 \\ & f_3(\boldsymbol{x}) = -x_2 < 0 \end{aligned}$$

Do we have a solution? Try:

- $x_1 = 2,\ x_2 = 0.01$
- $x_1 = 2,\ x_2 = 0.001$
- $x_1 = 2,\ x_2 = 0.000001$

Each is feasible and each is better than the last, but $x_2 = 0$ is **not** allowed. The infimum is never attained - **no optimal solution exists**. Strict inequalities give open feasible sets, and minima can fail to exist on them.

1.7 Engineering Examples of Optimization

1.7.1 Minimizing total transmitting power

Power control in a wireless network

Setup: n receivers and n transmitters.

Symbol	Meaning
G_{ij}	Path gain from transmitter j to receiver i
P_i	Transmitting power of the i-th transmitter
σ_i	Noise power at receiver i
$S_{\min}$	Required SINR threshold

Objective: minimize the total transmitter power while ensuring that the **Signal to Interference plus Noise Ratio (SINR)** for each transmitter–receiver pair is at least a given threshold.

SINR

$$\text{SINR} = \frac{\text{Desired signal power}}{\text{Interference power} + \text{Noise power}}$$

For the i-th receiver and i-th transmitter:

$$\text{SINR}_i = \frac{G_{ii}P_i}{\sigma_i + \sum_{k\neq i} P_k G_{ik}}$$

Full problem

$$\begin{aligned} \underset{\boldsymbol{p}\in\mathbb{R}^n}{\text{minimize}} \quad & P_1 + P_2 + \cdots + P_n \\ \text{subject to} \quad & P_i^{\min} \leq P_i \leq P_i^{\max} \\ & \frac{G_{ii}P_i}{\sigma_i + \sum_{k\neq i} P_k G_{ik}} \geq S_{\min} \end{aligned}$$

- **Objective function:** $P_1 + P_2 + \cdots + P_n$ (to be minimized)
- **Constraint 1:** SINR for each pair $\geq$ threshold, $\text{SINR}_{ij} \geq S_{\min}\ \forall i, j$
- **Constraint 2:** transmission power lies between a minimum and maximum value, $P_i^{\min} \leq P_i \leq P_i^{\max}\ \forall i$

Numerical power-allocation example

With the 5×5 gain matrix

$$G = \begin{bmatrix} 1.00 & 0.10 & 0.20 & 0.10 & 0.05 \\ 0.10 & 1.00 & 0.10 & 0.10 & 0.05 \\ 0.20 & 0.10 & 1.00 & 0.20 & 0.20 \\ 0.10 & 0.10 & 0.20 & 1.00 & 0.10 \\ 0.05 & 0.05 & 0.20 & 0.10 & 1.00 \end{bmatrix}$$

and $S_{\min} = 0.2$, $P_i^{\min} = 0.1$, $P_i^{\max} = 1$, the solver returns

$$\boldsymbol{P} = \begin{bmatrix} 0.18653846 & 0.16730769 & 0.23461538 & 0.19615385 & 0.17692308 \end{bmatrix}^T$$

Solved numerically with a Python convex-optimization package (CVXPY).

1.7.2 Total variation inpainting

Recovering a corrupted image

An image is stored as a matrix of pixel values U_{ij}, $i = 1, \ldots, m$, $j = 1, \ldots, n$. Some pixels are **corrupted or missing**. **How do we recover the image?**

- Known pixel values U_{ij}^{ori} for $(i, j) \in K$, where $K \subset \{1, \ldots, m\} \times \{1, \ldots, n\}$ is the set of **known** pixel locations.
- Let the recovered image be $\boldsymbol{U}$, with $U_{ij} = U_{ij}^{ori}$ for $(i, j) \in K$.

The idea: among all images that agree with the known pixels, pick the one that is **smoothest** - i.e. of minimum total variation.

Total variation in-painting

Minimize the **total variation** of $\boldsymbol{U}$:

$$\mathrm{tv}(\boldsymbol{U}) = \sum_{i=1}^{m}\sum_{j=1}^{n} \left\| \begin{bmatrix} U_{i+1,j} - U_{ij} \\ U_{i,j+1} - U_{ij} \end{bmatrix} \right\|_2$$

subject to the constraint

$$U_{ij} = U_{ij}^{ori} \quad \text{for } (i,j) \in K$$

Forward differences outside the image boundary are defined as zero. Here the decision variables are **all the unknown pixel values**, the objective penalises abrupt changes between neighbouring pixels, and the equality constraints pin down the pixels we already know.

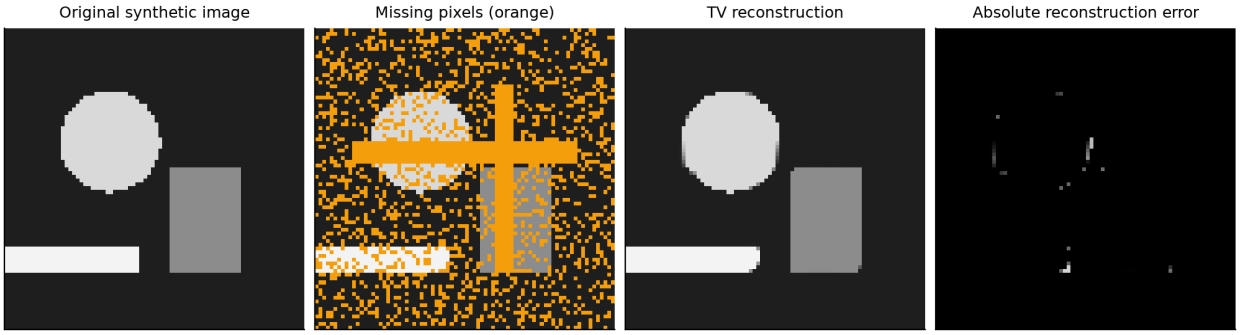


Figure 1.2: Total-variation inpainting on an 80×80 synthetic image. Orange pixels are unknown. A primal-dual iteration estimates the missing pixels while preserving every known pixel exactly.

Reconstruction and residual error

The reconstruction recovers broad constant-intensity regions from the known pixels. The absolute-error image localizes errors near edges and larger missing regions. Total variation encourages short, sharp boundaries, but cannot uniquely recover detail that is absent from the observations. The example uses a fixed random seed of 42 and 1,800 primal-dual iterations.

1.7.3 Support vector machine classifier

Optimal separating hyperplane

Given labelled points $(\boldsymbol{x}_i, y_i)$ with $y_i \in \{+1, -1\}$, find the hyperplane $w_1 x_1 + w_2 x_2 + b = 0$ that separates the two classes with the largest margin.

- $\boldsymbol{x}_i$ **positive** ($y_i = 1$): $\boldsymbol{x}_i \cdot \boldsymbol{w} + b \geq 1$
- $\boldsymbol{x}_i$ **negative** ($y_i = -1$): $\boldsymbol{x}_i \cdot \boldsymbol{w} + b \leq -1$
- **Combined:** $y_i(\boldsymbol{x}_i \cdot \boldsymbol{w} + b) - 1 \geq 0 \quad \forall i$

Distance from a point to the hyperplane:

$$\frac{|\boldsymbol{x}_i \cdot \boldsymbol{w} + b|}{\|\boldsymbol{w}\|}$$

For **support vectors** (the points closest to the boundary): $\boldsymbol{x}_i \cdot \boldsymbol{w} + b = \pm 1$.

From margin maximization to a standard QP

Maximize the margin:

$$\max_{\boldsymbol{w},b} \frac{|\boldsymbol{x}_i \cdot \boldsymbol{w} + b|}{\|\boldsymbol{w}\|} \quad \text{subject to } y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) \geq 1$$

Since $\boldsymbol{x}_i \cdot \boldsymbol{w} + b = \pm 1$ for support vectors, this is **equivalent to the primal hard-margin SVM**:

$$\min_{\boldsymbol{w},b} \tfrac{1}{2}\|\boldsymbol{w}\|^2 \quad \text{subject to } y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) \geq 1$$

Soft-margin SVM (with hinge loss), allowing some violations:

$$\min_{\boldsymbol{w},b} \tfrac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{i=1}^{n} \max\left(0,\ 1 - y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b)\right)$$

Regularized hinge loss with L_1 norm:

$$\min_{\boldsymbol{w},b} \frac{1}{n}\sum_{i=1}^{n} (1 - y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b))_+ + \lambda\|\boldsymbol{w}\|_1$$

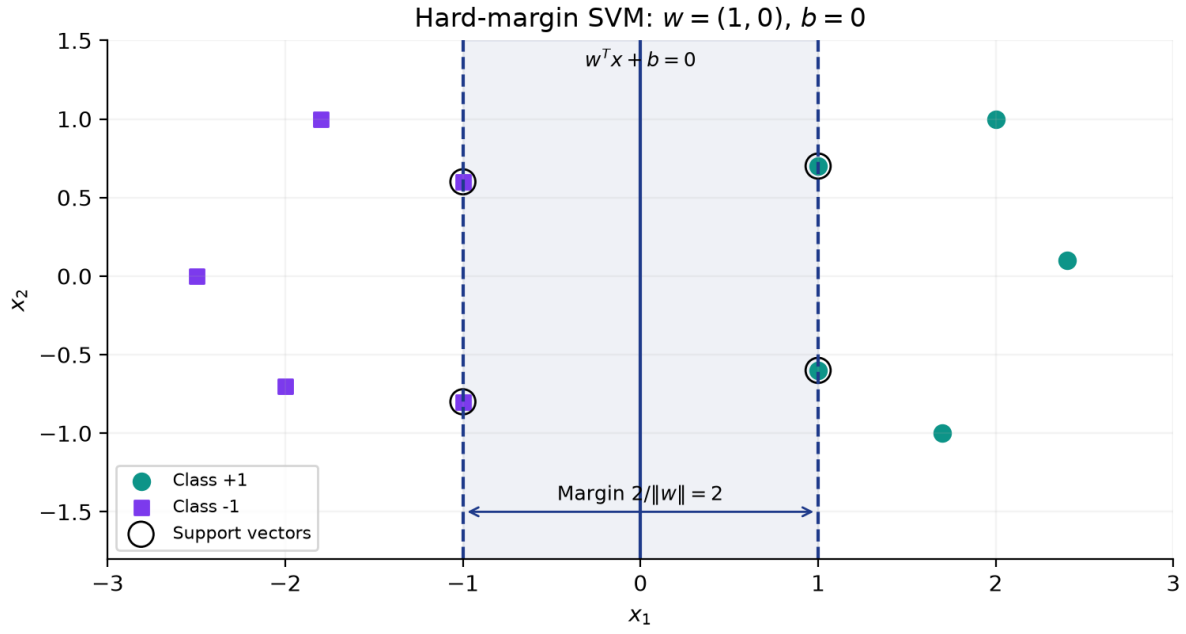


Figure 1.3: A hard-margin separator with $w = (1, 0)$ and $b = 0$. The dashed lines are $w^T x + b = \pm 1$; their separation is $2/\|w\|$. Circled points are support vectors.

Pattern to notice across all three examples

Each one follows the same recipe: **identify the decision variables → write down what "good" means as a scalar objective → write the physical/logical restrictions as inequalities or equalities → hand it to a solver.** This turns an engineering design objective into a mathematical optimization model.

1.7.4 Further application areas

Further optimization problems in engineering

- Interference mitigation via power control
- Determining the shortest path in a communication network
- Processor–task assignment problem in multiprocessor environments
- For state targeting, transistor sizing for digital CMOS circuits
- Digital filter designs by reference matching
- Matrix completion problem
- Non-negative matrix factorization
- Image denoising

1.8 Basic Concepts: Minima and Optimality Conditions

1.8.1 Global vs local minimum

Global minimum

A point $\boldsymbol{x}^*$ is a **global minimum** of a function $f(\boldsymbol{x})$ if

$$f(\boldsymbol{x}^*) \leq f(\boldsymbol{x}) \quad \text{for all } \boldsymbol{x} \text{ in the domain of } f.$$

$f(\boldsymbol{x}^*)$ is the smallest value that $f(\boldsymbol{x})$ can achieve over its **entire** domain.

Local minimum

A point $\boldsymbol{x}^*$ is a **local minimum** of $f(\boldsymbol{x})$ if

$$f(\boldsymbol{x}^*) \leq f(\boldsymbol{x}) \quad \text{for all } \boldsymbol{x} \text{ in some neighborhood of } \boldsymbol{x}^*.$$

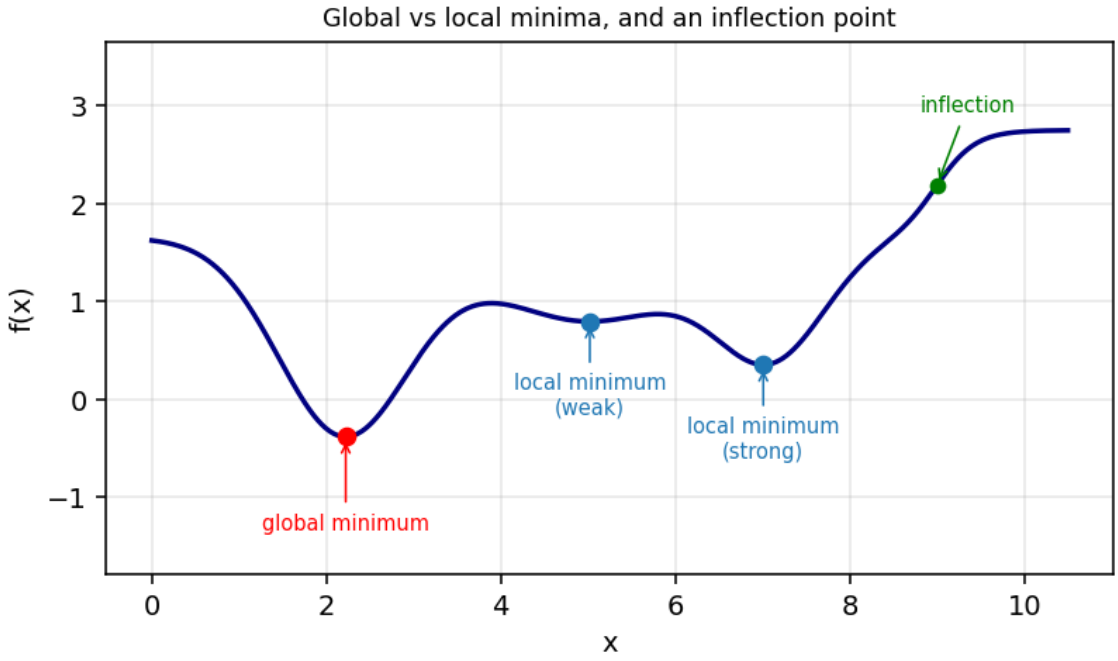
A **neighborhood** of $\boldsymbol{x}^*$ is a small region around $\boldsymbol{x}^*$ - e.g. an interval $(\boldsymbol{x}^* - \epsilon,\ \boldsymbol{x}^* + \epsilon)$ for some $\epsilon > 0$.

Types of stationary and minimizing points

An objective landscape can contain a **global minimum**, a **weak local minimum**, a **strong local minimum**, and an **inflection**.

Term	Definition
Critical points	Values within the domain where the function's derivative is equal to zero
Global minimum	The lowest **value** the function attains over its entire domain
Global minimizer	The **value of** x that minimizes $f(x)$

- A function may have **at most one global minimum** (value)…
- …but it may have **multiple global minimizers** (many x's achieving it).
- It is **challenging to prove** that a given candidate point is a global minimum. Generally, we check that it is a **local** minimum.

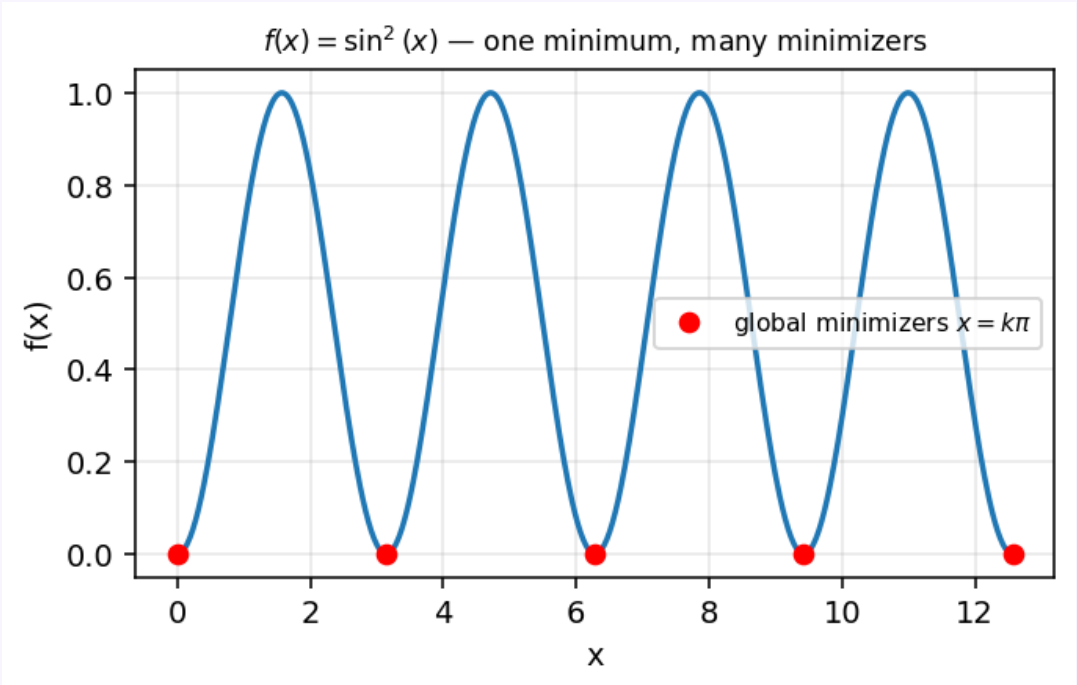


One curve, four kinds of critical point: the deepest valley is the global minimum, the shallow wide dip is a weak local minimum, the sharper dip is a strong local minimum, and the flat spot on the rising section is an inflection.

Many minimizers, one minimum

$$f(x) = \sin^2(x)$$

Global minimizers: $x = k\pi$ for integer k - infinitely many. **Global minimum:** 0 - a single value. This is exactly the "at most one global minimum, possibly many global minimizers" distinction.



1.8.2 Necessary conditions for a local minimum

Multivariate necessary conditions

For a multivariate function, a critical point $\boldsymbol{x}$ can be a local minimum if:

- **First-order necessary condition (FONC):** $\nabla f(\boldsymbol{x}) = 0$
- **Second-order necessary condition (SONC):** the **Hessian matrix is positive semidefinite**, i.e. $\boldsymbol{x}^T \boldsymbol{A}\boldsymbol{x} \geq 0$

Univariate necessary conditions

For a univariate function, a point x^* can be a local minimum if:

- **FONC:** $f'(x^*) = 0$
- **SONC:** $f''(x^*) \geq 0$

Necessary is not sufficient

These are called **necessary conditions** because:

- Every local minimum **must** satisfy both FONC and SONC.
- They are **required but not sufficient** on their own to guarantee a local minimum.

Not every point with a zero derivative and a zero second derivative is a local minimum.

Three univariate stationary-point examples

Picture	Conditions	Verdict
A straight increasing line through x^*	**SONC but not FONC** ($f'' = 0 \geq 0$ but $f' \neq 0$)	**Not** a local minimum
A bowl-shaped curve with x^* at the bottom	**FONC and SONC**	Local minimum
An inflection point (cubic-like curve flattening at x^*)	**FONC and SONC**	Satisfies both, yet is **not** a minimum

The 2-D surfaces make the same point:

Surface	Gradient	Hessian	Verdict
Saddle	Zero	Indefinite	**Not** a local minimum
Bowl	Zero	Positive definite	Local minimum

Stationarity
$\nabla f(x^*) = 0$ is necessary at an interior smooth minimum.

↓

Curvature
$H(x^*) \succeq 0$ is necessary; by itself it is inconclusive.

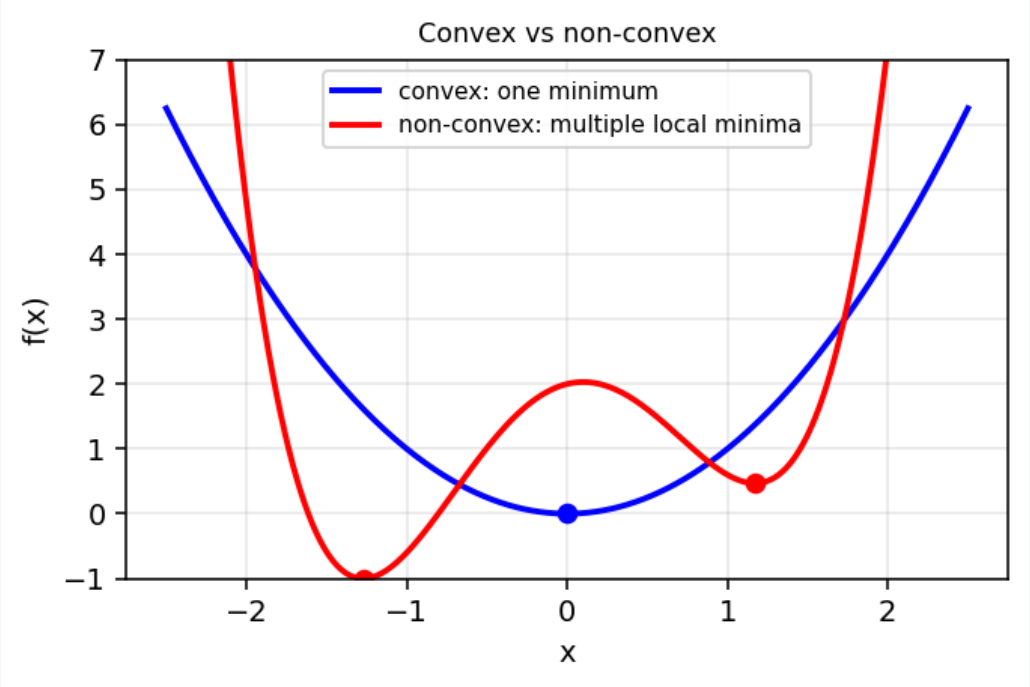
↓

Sufficiency
$H(x^*) \succ 0$ gives a strict local minimum; convex f makes every stationary point global.

1.9 Convex vs Non-Convex Optimization

The distinction

- **Convex functions:** one minimum - the optimization algorithm **won't get stuck** in a local minimum that isn't a global minimum.
- A **non-convex** function is *wavy* - it has some 'valleys' (local minima) that aren't as deep as the overall deepest 'valley' (the global minimum).



Why convex optimization

- Every **local minimum is a global minimum**.
- If the objective function is **strictly convex**, then the problem has **at most one optimal point**.
- **Convergence time is polynomial.**

Why non-convex optimization is hard

- **Multiple local minima** within each region.
- Multiple feasible and very flat regions, with widely varying curvature.
- May require an **initial guess, which is critical**.
- Little information is provided about how far from the global optimum the local solution is.
- Local optimization methods are often **sensitive to algorithm parameter values**.

Geometry and optimality

More on convex optimization: **Semester 8, EN4730 Convex Engineering Design**.

1.10 Key Takeaways

Summary

Optimization means **minimizing an objective $f_0(x)$ over decision variables x subject to constraints**. Constraints define the **feasible region**; the **optimal point** is the best feasible point. FONC and SONC are *necessary but not sufficient* for a local minimum - and only **convexity** upgrades a local minimum to a global one.

Self-check

1. Why is the question "what is the best speed?" unanswerable without stating an objective?
2. Write the general constrained optimization problem in standard form and name every symbol.
3. What is the difference between a **feasible** point and an **optimal** point?
4. Sketch the feasible region for $x_1 \geq 0$, $x_2 \geq 0$, $x_1 + x_2 \leq 2$, then explain why the minimizer of $(x_1 - 2)^2 + (x_2 + 3)^2 + 1$ moves from $(2, -3)$ to $(2, 0)$.
5. Why does the problem with **strict** constraints $-x_1 < 0$, $-x_2 < 0$ have no optimal solution?
6. State FONC and SONC for both univariate and multivariate functions. Give one example of a point satisfying both that is **not** a minimum.
7. A function may have at most one global _____ but possibly many global _____. Fill in and justify with $f(x) = \sin^2 x$.
8. List three reasons convex optimization is preferred over non-convex optimization.
9. Formulate the wireless power-control problem: what are the decision variables, objective, and constraints?
10. In total variation in-painting, what plays the role of the objective and what plays the role of the equality constraints?

1.11 References

Further reading

[1] Alan G. Gomez, William C. Oakes, and Les L. Leone, *Engineering Your Future: A Project-Based Introduction to Engineering*, Great Lakes Press, 2004. [2] S. D. Lockhart and C. M. Johnson, *Engineering Design Communication: Conveying Design Through Graphics*, Addison-Wesley, 1998. [3] Stephen Boyd, Seung-Jean Kim, Lieven Vandenberghe, and Arash Hassibi, "A tutorial on geometric programming," *Optimization and Engineering*, vol. 8, pp. 67–127, 2007.

CHAPTER 2

Optimization - Fundamental Concepts

Overview

Finding and classifying optima uses **calculus and geometry**: how a function looks (graphs, level sets, contour plots), how it changes (derivatives, gradients, directional derivatives), how that change itself changes (the Hessian), and how these objects certify a point as a minimum (FONC, SONC, **SOSC**). **Numerical differentiation** supplies derivative approximations when analytic differentiation is unavailable.
Core idea: the negative gradient gives the local steepest-descent direction in the Euclidean norm and the Hessian tells you whether you are in a bowl or on a saddle. Gradient-based and Newton-type methods use these objects to select directions and assess local curvature.

2.1 Notation

Notation conventions

- **Vectors** are denoted by boldface **lower-case** letters, e.g. $\boldsymbol{a}$.
- Vectors written **with commas represent column vectors**:

$$\boldsymbol{v} = [a,\ b,\ c], \quad \boldsymbol{v} = [a\ b\ c]^T, \quad \text{or} \quad \boldsymbol{v} = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$$

- The **Euclidean norm** of a vector $\boldsymbol{a}$ is $\|\boldsymbol{a}\| = \sqrt{\boldsymbol{a}^T\boldsymbol{a}}$.
- Two vectors $\boldsymbol{a}$ and $\boldsymbol{b}$ are **orthogonal** if $\boldsymbol{a}^T\boldsymbol{b} = \boldsymbol{b}^T\boldsymbol{a} = 0$.
- $[a,\ b,\ c]^T \equiv [a\ b\ c]^T$.
- **Matrices** are denoted by boldface **upper-case** letters, e.g. $\boldsymbol{A}$.

Comma trap

$[a,\ b,\ c]$ with commas is already a **column** vector in this course's notation. Do not read it as a row vector - this changes the shape of every product you write.

2.2 Recap: The Optimization Problem

Constrained optimization problem

$$\begin{aligned} \underset{\boldsymbol{x}}{\text{minimize}} \quad & f_0(\boldsymbol{x}) \\ \text{subject to} \quad & f_i(\boldsymbol{x}) \leq b_i, \quad i = 1, \ldots, m \\ & g_i(\boldsymbol{x}) = 0, \quad i = 1, \ldots, n \end{aligned}$$

- **Objective function:** $f_0 : \mathbb{R}^n \to \mathbb{R}$
- **Decision variables:** $\boldsymbol{x} = [x_1 \ \cdots \ x_n]^T \in \mathbb{R}^n$
- **Inequality constraint functions:** $f_i : \mathbb{R}^n \to \mathbb{R}, i = 1, \ldots, m$
- **Equality constraint functions:** $g_i : \mathbb{R}^n \to \mathbb{R}, i = 1, \ldots, n$
- **Optimal solution** $\boldsymbol{x}^*$: has the smallest value of f_0 among all vectors that satisfy the given constraints, i.e. $f_i(\boldsymbol{x}^*) \leq b_i$ and $g_i(\boldsymbol{x}^*) = 0$ for all i.

Optimal solution as an argmin

$$\begin{aligned} \boldsymbol{x}^* = \underset{\boldsymbol{x}}{\arg\min} \quad & f_0(\boldsymbol{x}) \\ \text{subject to} \quad & f_i(\boldsymbol{x}) \leq b_i, \quad i = 1, \ldots, m \\ & g_i(\boldsymbol{x}) = 0, \quad i = 1, \ldots, n \end{aligned}$$

Feasible points refer to a set of values for the decision variables that satisfies **all** the constraints of the optimization problem.

2.3 Formulation Exercise: The Shopping Problem

Problem statement

You need to purchase vegetables (carrot and beans), fish, and rice. The unit prices for one kilogram of each item are denoted U_c, U_b, U_f, and U_r respectively. You will receive a **5 % discount on the total cost of vegetables** and a **7 % discount on the total cost of fish**. Let x_1, x_2, x_3, x_4 represent the kilograms of carrot, beans, fish, and rice purchased. The maximum amount you can spend is **Rs. 10,000**. Formulate a constrained optimization problem to **maximize the total discount obtained**. Note that you must purchase all four items.

Identify the decision variables, formulate the objective, and state the constraints.

Step 1 - Decision variables

$$\boldsymbol{x} = [x_1 \ x_2 \ x_3 \ x_4]^T$$

where x_1 = kg of carrot, x_2 = kg of beans, x_3 = kg of fish, x_4 = kg of rice.

Step 2 - Objective (total discount)

Vegetable cost is $U_c x_1 + U_b x_2$ and attracts 5 %; fish cost is $U_f x_3$ and attracts 7 %; rice gets no discount.

$$D(\boldsymbol{x}) = 0.05\,(U_c x_1 + U_b x_2) + 0.07\,U_f x_3$$

We **maximize** $D(\boldsymbol{x})$, equivalently minimize $-D(\boldsymbol{x})$.

Step 3 - Constraints

Budget - what you actually pay is the discounted total:

$$0.95\,(U_c x_1 + U_b x_2) + 0.93\,U_f x_3 + U_r x_4 \leq 10000$$

All four items must be purchased (so no quantity may be zero):

$$x_1 > 0, \quad x_2 > 0, \quad x_3 > 0, \quad x_4 > 0$$

Full formulation

$$\begin{aligned} \underset{\boldsymbol{x}\in\mathbb{R}^4}{\text{maximize}} \quad & 0.05(U_c x_1 + U_b x_2) + 0.07 U_f x_3 \\ \text{subject to} \quad & 0.95(U_c x_1 + U_b x_2) + 0.93 U_f x_3 + U_r x_4 \leq 10000 \\ & x_1 > 0,\ x_2 > 0,\ x_3 > 0,\ x_4 > 0 \end{aligned}$$

Domain restrictions and attainment

1. **"Must purchase all four items"** forces *strict* inequalities $x_i > 0$. Strict inequalities produce an **open** feasible set on which the optimum may not be attained - practically, one imposes a minimum purchase $x_i \geq x_i^{\min}$ instead.
2. Rice appears **only in the constraint**, never in the objective. Since it earns no discount, every rupee spent on rice is a rupee unavailable for discounted items - the constraint is what makes the problem interesting.

2.4 Recap Examples: Unconstrained and Constrained

Unconstrained

$$\underset{\boldsymbol{x}\in\mathbb{R}^2}{\text{minimize}} \quad f_0(\boldsymbol{x}) = (x_1 - 2)^2 + (x_2 + 3)^2 + 1$$

$$x_1^* = 2,\ x_2^* = -3 \implies \boldsymbol{x}^* = [2\ \ -3]^T, \qquad f_0(\boldsymbol{x}^*) = 1$$

The surface is a bowl (paraboloid) with its lowest point at $(2, -3)$.

Constrained - same objective

$$\begin{aligned} \text{minimize} \quad & f_0(\boldsymbol{x}) = (x_1 - 2)^2 + (x_2 + 3)^2 + 1 \\ \text{subject to} \quad & f_1(\boldsymbol{x}) = (x_1 + x_2) \leq 2 \\ & f_2(\boldsymbol{x}) = -x_1 \leq 0 \\ & f_3(\boldsymbol{x}) = -x_2 \leq 0 \end{aligned}$$

Feasible set: the triangle $x_1 \geq 0,\ x_2 \geq 0,\ x_1 + x_2 \leq 2$.

$$x_1^* = 2,\ x_2^* = 0 \implies \boldsymbol{x}^* = [2\ 0]^T, \qquad f_0(\boldsymbol{x}^*) = 10$$

2.4.1 The "are these the same problem?" question

Problem 1 vs Problem 2

Problem 1:

$$\underset{\boldsymbol{x}\in\mathbb{R}^2}{\text{minimize}} \quad f_0(\boldsymbol{x}) = (x_1-2)^2 + (x_2+3)^2 + (y_1-3)^2 + y_2 + 1$$

Problem 2:

$$\underset{\boldsymbol{x},\ \boldsymbol{y}\in\mathbb{R}^2}{\text{minimize}} \quad f_0(\boldsymbol{x}) = (x_1-2)^2 + (x_2+3)^2 + (y_1-3)^2 + y_2 + 1$$

Are Problem 1 and 2 the same?

Decision variables determine the problem

The expression is identical; **what is being optimized over is not**.

	Problem 1	Problem 2
Decision variables	x_1, x_2 only	x_1, x_2, y_1, y_2
Role of y_1, y_2	**Fixed parameters** (given constants)	Decision variables
Minimizer	$\boldsymbol{x}^* = [2\ \ -3]^T$	$x^* = [2\ \ -3]^T$, $y_1^* = 3$, and $y_2 \to -\infty$
Optimal value	$1 + (y_1-3)^2 + y_2$ - depends on the given y	**Unbounded below** - no solution

Because y_2 enters **linearly**, letting it be a decision variable drives the objective to $-\infty$. Always state explicitly which symbols are decision variables - the same algebraic expression can be a well-posed problem or a meaningless one.

2.5 Graph of a Function

2.5.1 Single-variable case

Definition

The graph of a function $f(x)$ is the set of all ordered pairs $(x, f(x))$:

$$G(f) = \{(x, f(x)) : x \in X\}$$

- $f : X \mapsto Y$
- X is the **domain** of f ($\text{dom}(f)$)
- Y is the **codomain**
- x and $f(x)$ are real numbers $\to$ the $(x, f(x))$ pairs are **Cartesian coordinates of points in a plane**
- In this case they form a **curve**
- A **plot** is the graphical representation of the graph

Further reading

D. S. Bridges (1991). *Foundations of Real and Abstract Analysis*. Springer, p. 285.

2.5.2 Two-variable case

From curves to surfaces

With two variables we have ordered **triples** $(x, y, f(x, y))$, or (x, y, z) with $z = f(x, y)$. For a continuous real-valued function these form a **surface**.
The graph of $\sqrt{x^2 + y^2 + 20}$ is a smooth bowl. The coordinate planes $z = 0$, $y = 0$, and $x = 0$ provide reference surfaces for locating points in three-dimensional space.

One variable
$\{(x, f(x))\}$: a curve in two dimensions.

↓

Two variables
$\{(x, y, f(x, y))\}$: a surface in three dimensions.

↓

Higher dimensions
Use level sets and contour plots to reason geometrically.

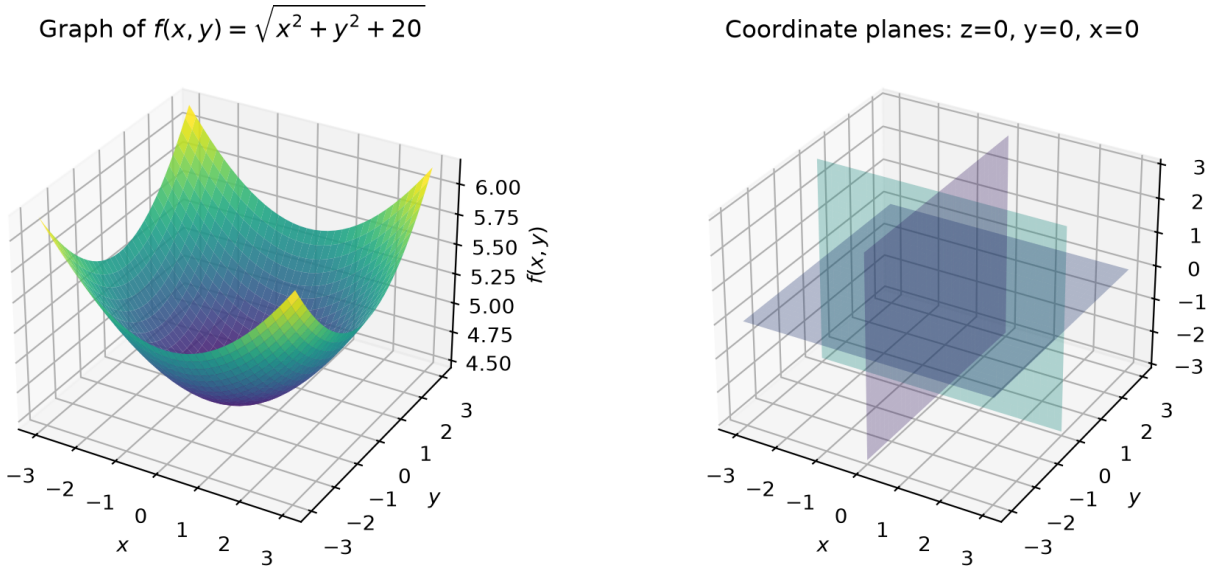


Figure 2.1: A two-variable function graph and the three coordinate planes used as geometric references.

2.6 Level Sets and Contour Plots

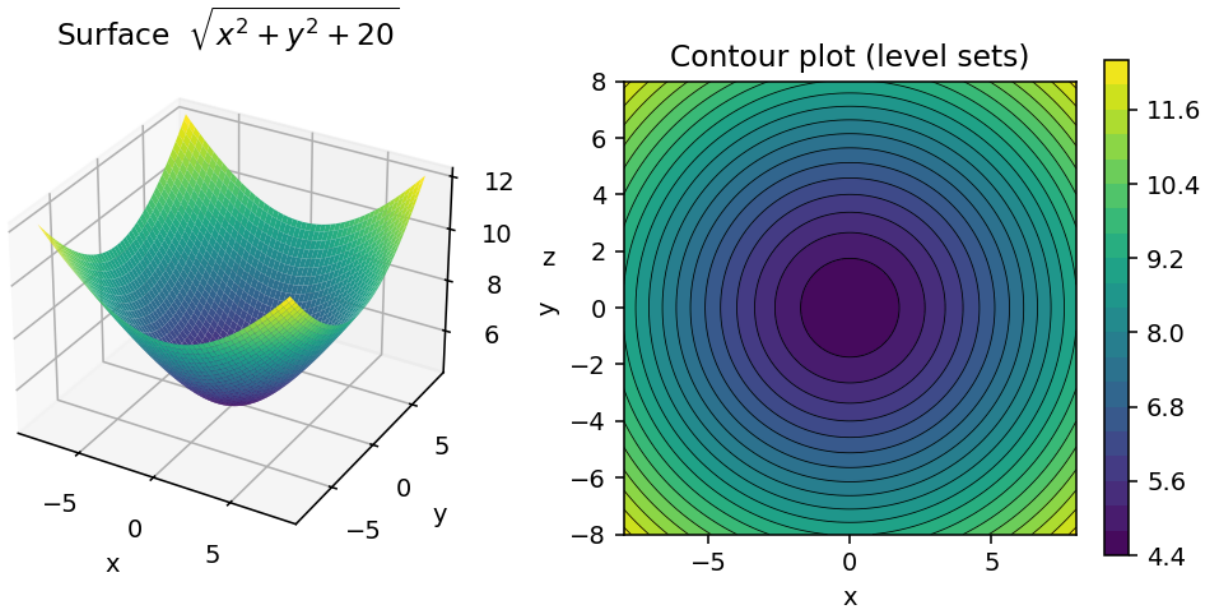
Visualizing a two-variable function

Take the surface $\sqrt{x^2 + y^2 + 20}$. Suppose you are looking at it **from directly above, down onto the XY plane**.

- **How are fixed function values represented?**
- **Why is it important?**

Level-set representation

Slicing the surface with horizontal planes at constant z and projecting the slices down gives a **contour plot**.
A contour plot **visually represents a three-dimensional surface by mapping regions with constant z values - referred to as contours - onto a two-dimensional plot**. The plot has axes indexed by x and y.
A **level set** of f at level c is $\{(x, y) : f(x, y) = c\}$ - one contour line.



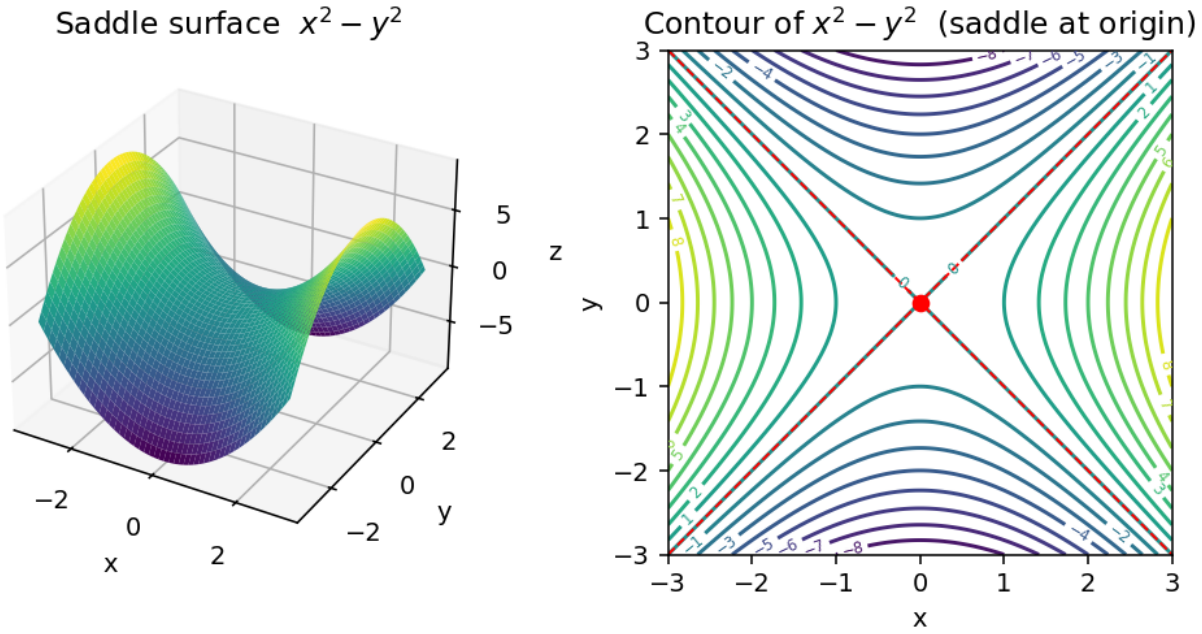
Left: the surface $\sqrt{x^2 + y^2 + 20}$. Right: the same function viewed from above as a contour plot - the level sets are concentric circles closing in on the single minimum at the origin.

Why level sets matter for optimization

- They reduce an n-dimensional surface to a **2-D picture** you can reason about.
- The **spacing** of contours encodes steepness: tightly packed contours = steep, widely spaced = flat.
- The **gradient at any point is perpendicular to the contour through that point** - so contours show you the descent directions directly.
- Minima appear as **nested closed loops shrinking to a point**; saddles appear as **crossing contours**.

Two contrasting shapes

Function	Surface	Contour plot
$\sqrt{x^2 + y^2 + 20}$	A bowl / cone, minimum at the origin	**Concentric circles**, values increasing outward - a single minimum
$x^2 - y^2$	A **saddle**, rising along x and falling along y	**Hyperbolas** with a crossing pair of straight lines ($y = \pm x$) through the origin - a saddle point



The saddle $x^2 - y^2$. Its contours are hyperbolas, and the two dashed red lines $y = \pm x$ are the zero-level set crossing at the origin - the visual signature of a saddle point.

The saddle is the key cautionary example: at the origin the surface is flat (gradient zero) yet it is **neither a minimum nor a maximum**.

2.7 Implications in the Gradient

Local change and optimization

- Optimization aims to find the design point that **minimizes or maximizes an objective function**.
- Understanding how a function's value **changes with respect to its input** is improvement.
- **Derivatives** (in one dimension) and **gradients** (in multiple dimensions) measure the **rate of change** in the function's value.

2.7.1 Gradient of a single-variable function

Meaning of $f'(x)$

The derivative $f'(x)$ represents the **rate at which the value of a single-variable function f changes at the point x** - geometrically, the slope of the **tangent line to the graph at x**.

- If $f'(x)$ is **positive** in the interval $[a, b] \rightarrow$ the function is **increasing** in that interval.
- If $f'(x)$ is **negative** in the interval $[a, b] \rightarrow$ the function is **decreasing** in that interval.

Linear approximation

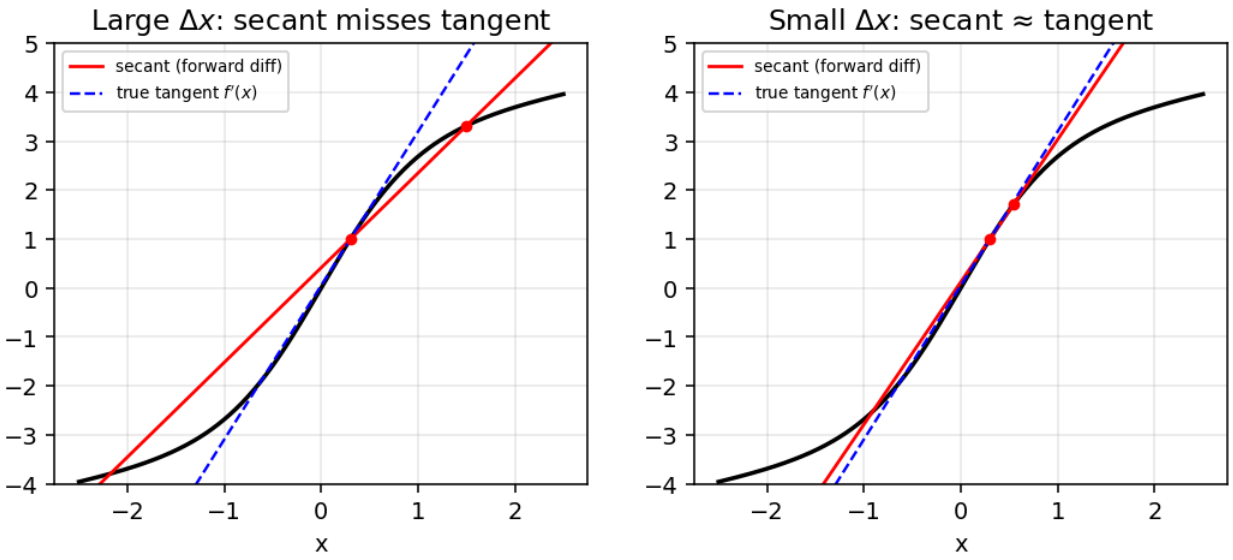
Near x:

$$f(x + \Delta x) \approx f(x) + f'(x)\Delta x$$

which rearranges to

$$f'(x) \approx \frac{f(x+\Delta x) - f(x)}{\Delta x}$$

Δx **needs to be sufficiently small** - a finite secant slope approximates the tangent slope only in the small-step limit.



The forward difference is the slope of the red secant line. With a large Δx it is a poor estimate of the true tangent (blue dashed); shrinking Δx makes the two coincide.

The three finite-difference forms

Forward difference

$$f'(x) = \lim_{\Delta x \to 0} \frac{f(x+\Delta x) - f(x)}{\Delta x}$$

Central difference

$$f'(x) = \lim_{\Delta x \to 0} \frac{f(x+\Delta x/2) - f(x-\Delta x/2)}{\Delta x}$$

Backward difference

$$f'(x) = \lim_{\Delta x \to 0} \frac{f(x) - f(x-\Delta x)}{\Delta x}$$

2.7.2 Gradient of a multi-variable function

From slope to steepest ascent

- The gradient **generalizes the concept of a derivative for multivariate functions**.
- It **captures the local slope** of a function, providing insights into changes in **any direction** from a given point.
- The **derivative** $\rightarrow$ the slope of the tangent **line**.
- The **gradient** $\rightarrow$ points in the direction of the **steepest ascent** of the tangent **hyperplane**.

Each component of the gradient represents a local tangent line; these tangent lines collectively define the local **tangent hyperplane**; the **gradient vector indicates the direction of steepest increase**.

What is a hyperplane?

A hyperplane in an n-dimensional space is $\boldsymbol{w}^T\boldsymbol{x} = w_1x_1 + \cdots + w_nx_n = b$, with vector $\boldsymbol{w} = [w_1 \cdots w_n]^T$ and scalar b. The vector $\boldsymbol{w}$ is the **normal vector** to the hyperplane.
A hyperplane is an $(n-1)$-dimensional subspace embedded within an n-dimensional space:

Space	Hyperplane is
3-dimensional	a **2-D plane**
2-dimensional	a **1-D line**

The gradient operator

$$\nabla \equiv \begin{bmatrix} \frac{\partial}{\partial x_1} \\ \frac{\partial}{\partial x_2} \\ \vdots \\ \frac{\partial}{\partial x_n} \end{bmatrix} \Longrightarrow \nabla f(x) = \begin{bmatrix} \frac{\partial f(x)}{\partial x_1} \\ \frac{\partial f(x)}{\partial x_2} \\ \vdots \\ \frac{\partial f(x)}{\partial x_n} \end{bmatrix}$$

2.8 The Hessian

What the second derivative tells you

- The **Hessian contains all second derivatives**.
- The second derivative provides information about **how the slope (rate of change) is changing**.
- It gives insight into the **curvature** of the curve.
- If the second derivative is **positive** in a range, the slope of the function is **increasing** - the curve is **bending upward / concave up**.

Definition

$$\nabla\nabla^T f(x) = \begin{bmatrix} \frac{\partial}{\partial x_1} \\ \vdots \\ \frac{\partial}{\partial x_n} \end{bmatrix} \begin{bmatrix} \frac{\partial}{\partial x_1} & \cdots & \frac{\partial}{\partial x_n} \end{bmatrix} f(x)$$

$$\boldsymbol{H} = \begin{bmatrix} \frac{\partial^2}{\partial x_1^2} & \frac{\partial^2}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2}{\partial x_1 \partial x_n} \\ \frac{\partial^2}{\partial x_2 \partial x_1} & \frac{\partial^2}{\partial x_2^2} & \cdots & \frac{\partial^2}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2}{\partial x_n \partial x_1} & \frac{\partial^2}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2}{\partial x_n^2} \end{bmatrix} f(x)$$

When is the Hessian symmetric?

The Hessian matrix can be expressed in terms of the **Jacobian matrix**. The Hessian is symmetric, $\frac{\partial^2 f}{\partial x_i \partial x_j} = \frac{\partial^2 f}{\partial x_j \partial x_i}$, **only if** all second derivatives of the function are **continuous in a neighborhood of the point** where you're evaluating the Hessian.

2.8.1 Worked examples

Example A: $f(x_1, x_2) = x_1^3 + x_2^3$

$$\nabla f(\boldsymbol{x}) = \left[\frac{\partial f}{\partial x_1}, \quad \frac{\partial f}{\partial x_2}\right] = \left[3x_1^2, \quad 3x_2^2\right]$$

$$\boldsymbol{H} = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} \end{bmatrix} = \begin{bmatrix} 6x_1 & 0 \\ 0 & 6x_2 \end{bmatrix}$$

Example B: $f(x_1, x_2) = x_1^3 x_2 + x_1 x_2^3$

$$\nabla f(\boldsymbol{x}) = \left[3x_1^2 x_2 + x_2^3, \quad x_1^3 + 3x_1 x_2^2\right]$$

$$\boldsymbol{H} = \begin{bmatrix} 6x_1 x_2 & 3x_1^2 + 3x_2^2 \\ 3x_1^2 + 3x_2^2 & 6x_1 x_2 \end{bmatrix}$$

Evaluated at $\boldsymbol{x} = [1,\ 2]$:

$$\nabla f(\boldsymbol{x}) = \left[3(1)^2(2) + (2)^3, \quad (1)^3 + 3(1)(2)^2\right] = \left[14, \quad 13\right]$$

$$\boldsymbol{H} = \begin{bmatrix} 6(1)(2) & 3(1)^2 + 3(2)^2 \\ 3(1)^2 + 3(2)^2 & 6(1)(2) \end{bmatrix} = \begin{bmatrix} 12 & 15 \\ 15 & 12 \end{bmatrix}$$

```python
import sympy as sp

x1, x2 = sp.symbols('x1 x2')
f = x1**3*x2 + x1*x2**3

grad = sp.Matrix([f]).jacobian([x1, x2])
H    = sp.hessian(f, (x1, x2))

print(grad)                          # [3*x1**2*x2 + x2**3, x1**3 + 3*x1*x2**2]
print(grad.subs({x1: 1, x2: 2}))     # [14, 13]
print(H.subs({x1: 1, x2: 2}))        # [[12, 15], [15, 12]]
```

2.9 Directional Derivatives

Definition

The **instantaneous rate of change of $f(\boldsymbol{x})$ as $\boldsymbol{x}$ is moved with velocity $\boldsymbol{s}$**:

$$\nabla_s f(\boldsymbol{x}) = \nabla f(\boldsymbol{x})^T \boldsymbol{s}$$

(Sometimes $\boldsymbol{s}$ is defined as a **unit** vector.)

Finite-difference forms

Forward difference

$$\nabla_s f(\boldsymbol{x}) = \lim_{\Delta x \to 0} \frac{f(\boldsymbol{x} + \boldsymbol{s}\Delta x) - f(\boldsymbol{x})}{\Delta x}$$

Central difference

$$\nabla_s f(\boldsymbol{x}) = \lim_{\Delta x \to 0} \frac{f(\boldsymbol{x} + \boldsymbol{s}\Delta x/2) - f(\boldsymbol{x} - \boldsymbol{s}\Delta x/2)}{\Delta x}$$

Backward difference

$$\nabla_s f(\boldsymbol{x}) = \lim_{\Delta x \to 0} \frac{f(\boldsymbol{x}) - f(\boldsymbol{x} - \boldsymbol{s}\Delta x)}{\Delta x}$$

The two facts every descent algorithm rests on

- **Maximal gradient direction:** the directional derivative is **maximized when moving in the direction of the gradient** of the function.
- **Minimal gradient direction:** the directional derivative is **minimized when moving in the opposite direction of the gradient**.

This is exactly why **gradient descent** steps along $-\nabla f(\boldsymbol{x})$: it is the direction of fastest decrease.

2.10 The Hessian and Optimality (Unconstrained Problems)

Stationary points and local extrema

Critical points are values within a function's domain where the function's derivative is equal to zero. A function can have a **global minimum**, a **weak local minimum**, a **strong local minimum**, and an **inflection** point.

Necessary conditions (multivariate)

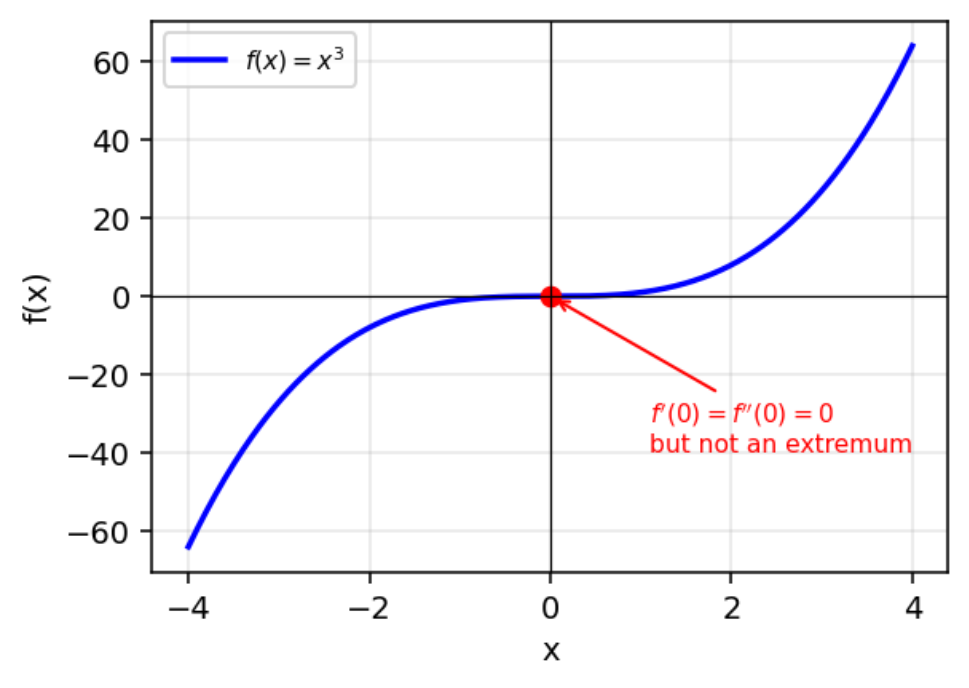
- **First-order necessary condition (FONC):** $\nabla f(\boldsymbol{x}) = 0$
- **Second-order necessary condition (SONC):** the Hessian matrix is **positive semidefinite** ($\boldsymbol{x}^T \boldsymbol{A}\boldsymbol{x} \geq 0$)

2.10.1 Why FONC alone is not enough

Counter-example 1 (univariate): $f(x) = x^3$

$$f'(x) = 3x^2$$

At $x = 0$, $f'(0) = 0$ - FONC is satisfied - **but** $x = 0$ **is not a minimum or maximum**. It is an inflection point: the cubic keeps increasing through it.

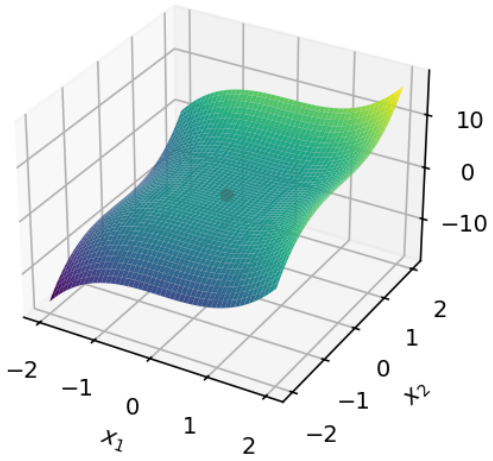


Counter-example 2 (multivariate): $f(\boldsymbol{x}) = x_1^3 + x_2^3$

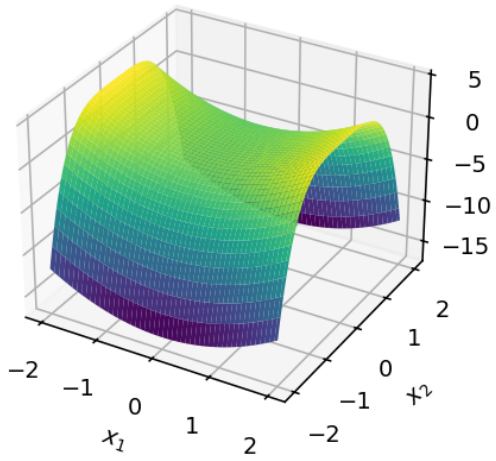
$$\nabla f(\boldsymbol{x}) = \left[3x_1^2, \quad 3x_2^2\right]$$

At $x_1 = 0,\ x_2 = 0$ we have $\nabla f(\boldsymbol{x}) = 0$ - **but** $(0, 0)$ **is not a minimum or maximum**. The surface plot shows it rising in one octant and falling in the opposite one.
FONC is used to **identify possible candidates, but is not sufficient**.

$f = x_1^3 + x_2^3$ — FONC holds, no extremum



$f = x_1^2 - x_2^4$ — FONC+SONC hold, no minimum



Left: $x_1^3 + x_2^3$ *- the gradient vanishes at the origin, yet the surface rises on one side and falls on the other. Right:* $x_1^2 - x_2^4$ *- both FONC and SONC hold at the origin, but moving along* x_2 *sends the function to* $-\infty$.

2.10.2 Why SONC is also not enough

Counter-example 3 (univariate): $f(x) = x^3$ again

$$f'(x) = 3x^2, \qquad f''(x) = 6x$$

At $x = 0$: $f'(0) = f''(0) = 0$, so both FONC and SONC hold - **yet $x = 0$ is not a minimum or maximum**.

Counter-example 4 (multivariate): $f(\boldsymbol{x}) = x_1^2 - x_2^4$

$$\nabla f(\boldsymbol{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 \\ -4x_2^3 \end{bmatrix}, \qquad \boldsymbol{H} = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & -12x_2^2 \end{bmatrix}$$

- At $x_1 = 0$ and $x_2 = 0$: $\nabla f(\boldsymbol{x}) = 0$ ✓ (FONC)
- At $x_1 = 0$ and $x_2 = 0$: eigenvalues of $\boldsymbol{H}$ are 2 and 0, so $\boldsymbol{H}$ is **PSD** ✓ (SONC)
- **But $(0, 0)$ is not a minimum** - moving along x_2 makes $-x_2^4$ decrease without bound.

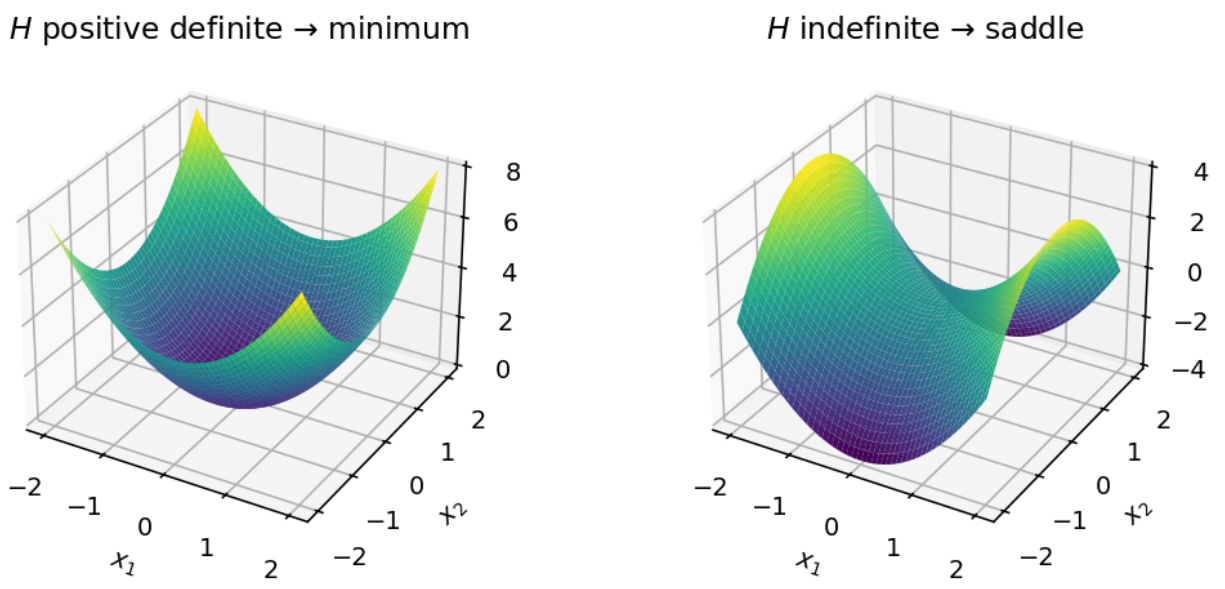
SONC (Hessian PSD) cannot guarantee that the point is a local minimum/maximum. For $f(x_1, x_2) = x_1^2 - x_2^4$, values increase along the x_1-axis and decrease along the x_2-axis, so the origin is a saddle despite its positive-semidefinite Hessian.

2.10.3 The Second-Order Sufficient Condition (SOSC)

SOSC

Strengthening the second-order necessary condition and combining it with the first-order necessary condition ($\nabla f(\boldsymbol{x}) = 0$):

- For a **local minimum**, the Hessian must be **positive definite** (all eigenvalues > 0).
- For a **local maximum**, the Hessian must be **negative definite**.



*The whole point of SOSC in one picture: a **positive definite** Hessian gives the bowl on the left (a genuine minimum); an **indefinite** Hessian gives the saddle on the right, where the gradient also vanishes.*

Critical-point classification

1. Apply **FONC** and **SONC** to identify **potential candidates** for local minima or maxima.
2. Use **SOSC** to **confirm** if a critical point is indeed a local minimum or maximum.
3. If a problem has **only one stationary point** (satisfying FONC, SONC, SOSC), this point is **likely to be the (global) optimum**.

First-order test
If $\nabla f(x^*) \neq 0$, reject an interior smooth minimum.

↓

Second-order test
Positive definite: strict minimum. Negative definite: strict maximum. Indefinite: saddle.

↓

Singular Hessian
Use higher-order terms or direct analysis. Uniqueness of a stationary point alone does not establish global optimality.

2.11 Classifying Critical Points - Practice Set

Identify the critical points and determine whether they are minima, maxima, or neither

1. $f(\boldsymbol{x}) = x_1^2 + x_2^2$
2. $f(\boldsymbol{x}) = x_1^2 - x_2^2$
3. $f(\boldsymbol{x}) = (1 - x_1)^2 + 5(x_2 - x_1^2)^2$
4. $f(\boldsymbol{x}) = 3x_1^2 + 8x_1x_2 + 5x_2^2$

The 2×2 determinant test

The determinant of a 2×2 matrix is the **product of its eigenvalues**.

- If the **determinant is negative**, the eigenvalues have **opposite signs** $\rightarrow$ **saddle**.
- If two eigenvalues have the **same sign**, determine that sign by inspecting the **upper-left (or lower-right) element**. If it is positive, the matrix has two **positive** eigenvalues.

This shortcut is only for 2×2 matrices - for other dimensions it will vary.

The three worked matrices

Matrix	Determinant	Conclusion	Eigenvalues
$\boldsymbol{A} = \begin{bmatrix} 5 & 7 \\ 7 & 10 \end{bmatrix}$	$50 - 49 = 1 > 0$, top-left $5 > 0$	Two **positive** eigenvalues $\rightarrow$ **positive definite**	$[0.066,\ 14.93]$
$\boldsymbol{A} = \begin{bmatrix} -5 & 6 \\ 6 & -10 \end{bmatrix}$	$50 - 36 = 14 > 0$, top-left $-5 < 0$	Two **negative** eigenvalues $\rightarrow$ **negative definite**	$[-1,\ -14]$
$\boldsymbol{A} = \begin{bmatrix} 5 & 8 \\ 8 & 10 \end{bmatrix}$	$50 - 64 = -14 < 0$	**Positive and negative** eigenvalues $\rightarrow$ **indefinite (saddle)**	$[-0.88,\ 15.88]$

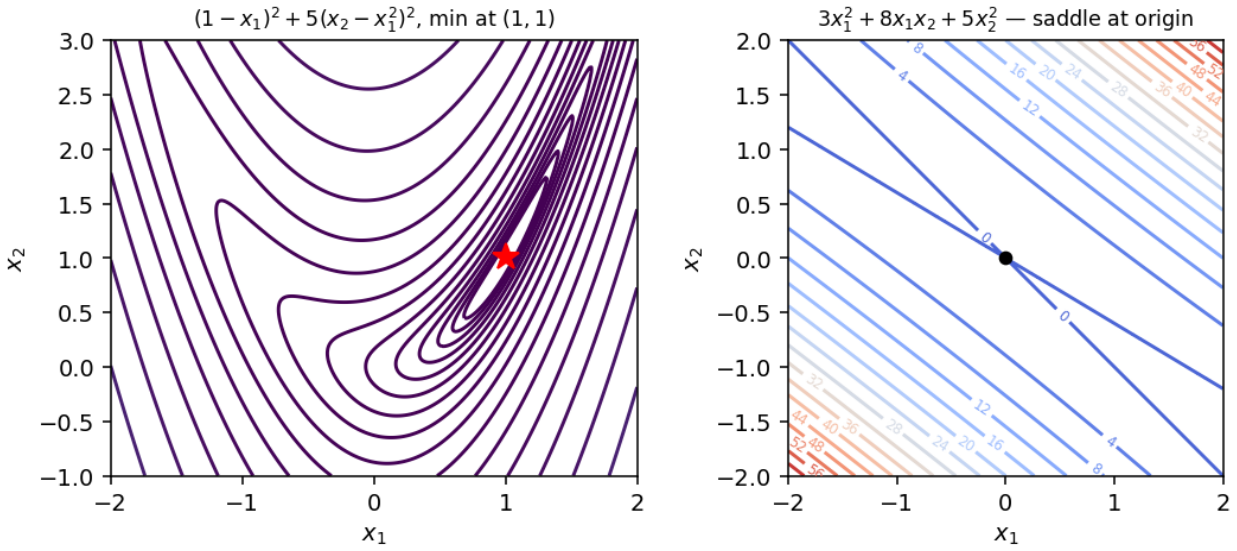
Solutions to the four practice functions

1. $f = x_1^2 + x_2^2$ - $\nabla f = [2x_1,\ 2x_2]^T = 0 \Rightarrow (0,0)$. $\boldsymbol{H} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$, $\det = 4 > 0$, top-left $2 > 0 \rightarrow$ **positive definite** $\rightarrow$ **strict local (and global) minimum** at the origin, $f = 0$.

2. $f = x_1^2 - x_2^2$ - $\nabla f = [2x_1,\ -2x_2]^T = 0 \Rightarrow (0,0)$. $\boldsymbol{H} = \begin{bmatrix} 2 & 0 \\ 0 & -2 \end{bmatrix}$, $\det = -4 < 0 \rightarrow$ **indefinite** $\rightarrow$ **saddle point**, neither a maximum nor a minimum. (This is the contour plot with crossing hyperbolas for the saddle function.)

3. $f = (1 - x_1)^2 + 5(x_2 - x_1^2)^2$ (a Rosenbrock-type function) - setting $\nabla f = 0$ gives $x_2 = x_1^2$ from the second equation and then $x_1 = 1$, so the only critical point is $(1, 1)$ with $f = 0$. There $\boldsymbol{H} = \begin{bmatrix} 42 & -20 \\ -20 & 10 \end{bmatrix}$, $\det = 420 - 400 = 20 > 0$ with top-left $42 > 0 \rightarrow$ **positive definite** $\rightarrow$ **local minimum**. As $f \geq 0$ everywhere and equals 0 here, it is the **global** minimum.

4. $f = 3x_1^2 + 8x_1x_2 + 5x_2^2$ - $\nabla f = [6x_1 + 8x_2,\ 8x_1 + 10x_2]^T = 0 \Rightarrow (0,0)$. $\boldsymbol{H} = \begin{bmatrix} 6 & 8 \\ 8 & 10 \end{bmatrix}$, $\det = 60 - 64 = -4 < 0 \rightarrow$ **indefinite** $\rightarrow$ **saddle point**.



Left: the curved valley of function 3, with its single minimum starred at $(1, 1)$ *- narrow valleys like this are what make gradient methods slow. Right: function 4, whose contours open outward along two directions from the origin, confirming the saddle predicted by* $\det \boldsymbol{H} < 0$.

2.12 Numerical Differentiation

When to use numerical differentiation

Numerical differentiation is a technique used to **estimate the derivative of a function at a particular point using numerical methods**.
When? When an **analytical expression for the derivative is not available or is difficult to compute** - for instance when f is the output of a simulation or a black-box program.

2.12.1 Finite difference methods

The three approximations

Forward difference

$$f'(x) \approx \frac{f(x+\Delta x) - f(x)}{\Delta x}$$

Central difference

$$f'(x) \approx \frac{f(x+\Delta x/2) - f(x-\Delta x/2)}{\Delta x}$$

Backward difference

$$f'(x) \approx \frac{f(x) - f(x-\Delta x)}{\Delta x}$$

The step-size dilemma

Smaller $\Delta x \rightarrow$ better estimation **in theory**. But a too-small Δx causes **numerical subtractive cancellation**: in finite-precision arithmetic $f(x) - f(x-\Delta x) \approx 0$, and dividing near-zero by near-zero destroys the answer.
This is why the error curve is **V-shaped**: truncation error falls as Δx shrinks, then round-off error explodes.

2.12.2 The Complex Step Method

The idea

- **Bypasses the effect of subtractive cancellation.**
- Evaluates the gradient by using a **single function** evaluation.
- The **step size is taken in the imaginary direction** (using complex numbers), i.e. $x + j\Delta x$.

For a complex function f, with $z = x + jy$ and $f(z) = f(x + jy) = u(x, y) + jv(x, y)$, u and v are the real and imaginary parts of $f(z)$.

Taylor expansion

A function $f(y)$ around a point y_0:

$$f(y) = f(y_0) + f'(y_0)(y - y_0) + \frac{f''(y_0)}{2!}(y - y_0)^2 + \frac{f'''(y_0)}{3!}(y - y_0)^3 + \cdots$$

For an imaginary step:

$$\begin{aligned} f(x + j\Delta x) &= f(x) + f'(x)(j\Delta x) + \frac{f''(x)}{2!}(j\Delta x)^2 + \frac{f'''(x)}{3!}(j\Delta x)^3 + \cdots \\ &= f(x) + f'(x)(j\Delta x) - \frac{f''(x)}{2!}(\Delta x)^2 - \frac{f'''(x)}{3!}j(\Delta x)^3 + \cdots \end{aligned}$$

(using $j^2 = -1$, $j^3 = -j$)

Taking the imaginary component $\rightarrow$ the derivative

$$\text{Im}\left(f(x + j\Delta x)\right) \approx f'(x)(j\Delta x) - \frac{f'''(x)}{3!}j(\Delta x)^3 + \cdots$$

$$f'(x) = \frac{\text{Im}\left(f(x + j\Delta x)\right)}{\Delta x} + \frac{f'''(x)}{3!}(\Delta x)^2 + \cdots$$

as $\Delta x \to 0$:

$$\boxed{f'(x) = \frac{\text{Im}\left(f(x + j\Delta x)\right)}{\Delta x} + o((\Delta x)^2)}$$

Taking the real component → the function value

$$\text{Re}\left(f(x + j\Delta x)\right) = f(x) - \frac{f''(x)}{2!}(\Delta x)^2 + \cdots$$

$$f(x) = \text{Re}\left(f(x + j\Delta x)\right) + \frac{f''(x)}{2!}(\Delta x)^2 - \cdots$$

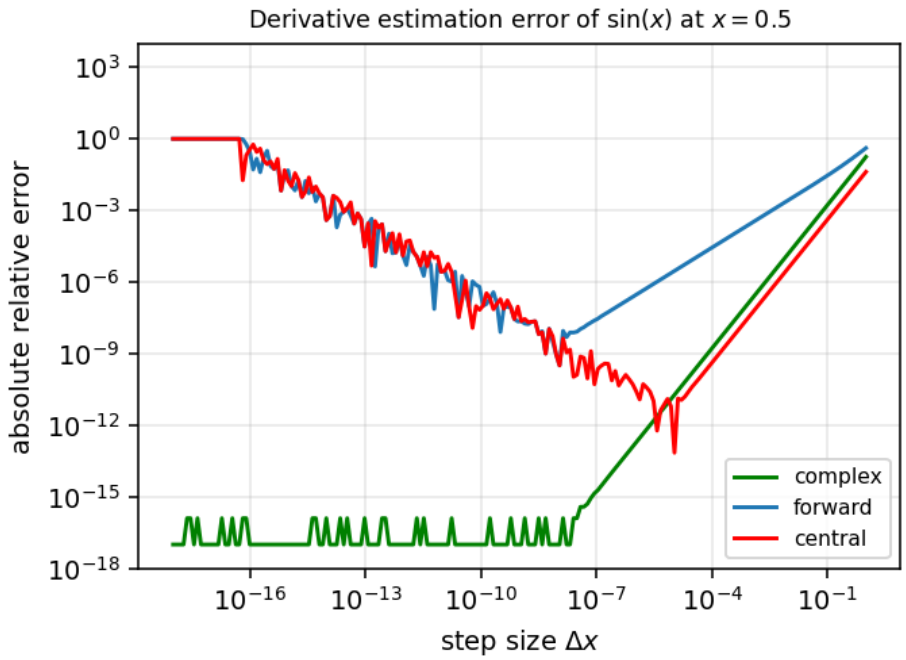
So a **single complex evaluation gives both** the function value (real part) and its derivative (imaginary part).

Differentiation error and numerical cancellation

The absolute relative differentiation error as a function of the step size Δx for the derivative of $\sin(x)$ at $x = 0.5$ shows:

Method	Behaviour
Forward	Error falls to $\approx 10^{-8}$ near $\Delta x \approx 10^{-8}$, then **rises sharply** from round-off
Central	Better - reaches $\approx 10^{-11}$ near $\Delta x \approx 10^{-6}$, then rises
Complex	Falls to $\approx 10^{-17}$ and **stays flat all the way down to** $\Delta x = 10^{-18}$ - no cancellation, no blow-up

There is **no subtraction of nearly equal numbers** in the complex step formula, so shrinking Δx never hurts.



The V-shape of the forward and central curves is the truncation-vs-round-off trade-off. The complex step (green) has no round-off arm at all - it is flat at machine precision across twelve orders of magnitude of Δx.

```python
import numpy as np

f = np.sin
x, h = 0.5, 1e-20

forward = (f(x + 1e-8) - f(x)) / 1e-8
central = (f(x + 5e-7) - f(x - 5e-7)) / 1e-6
complex_step = np.imag(np.sin(x + 1j*h)) / h   # exact to machine precision

print(forward, central, complex_step, np.cos(x))
```

References for the complex step

J. R. R. A. Martins, P. Sturdza, and J. J. Alonso, "The Complex-Step Derivative Approximation," *ACM Transactions on Mathematical Software*, vol. 29, no. 3, pp. 245–262, 2003. Kochenderfer, Mykel J., and Tim A. Wheeler. *Algorithms for Optimization*. MIT Press, 2019.

2.13 Resources

Matrix calculus and identities

- The Matrix Cookbook
- Review of Simple Matrix Derivatives (sfu.ca)
- Matrix Calculus (stanford.edu)
- Matrix Identities (nyu.edu)
- Notes On Norms (utexas.edu)
- Matrix Vector Norms

2.14 Key Takeaways

Summary

The **gradient** ∇f points along steepest ascent and vanishes at every critical point; the **Hessian** $\boldsymbol{H}$ describes curvature and decides what kind of critical point you found. **FONC + SONC find candidates; only SOSC (positive definite Hessian) confirms a local minimum.** When you cannot differentiate by hand, use finite differences - or better, the **complex step method**, which is immune to subtractive cancellation.

Self-check

1. In this course's notation, is $[a,\ b,\ c]$ a row or column vector?
2. Formulate the shopping problem: what are the decision variables, objective, and constraints? Why does "must purchase all four items" cause trouble?
3. Explain why the formulations that treat y as a fixed parameter and as a decision variable are *not* the same problem, and why Problem 2 is unbounded.
4. What does a contour plot show, and how are the contours of $x^2 + y^2$ and $x^2 - y^2$ different?
5. State the forward, central, and backward difference formulas for $f'(x)$.
6. In which direction is the directional derivative maximized? Minimized? What algorithm does this justify?
7. Write the Hessian of $f = x_1^3 x_2 + x_1 x_2^3$ and evaluate it at $[1,\ 2]$.
8. Under what condition is the Hessian guaranteed to be symmetric?
9. Give one function satisfying both FONC and SONC at a point that is **not** a minimum.
10. State SOSC for a local minimum and for a local maximum.
11. For $\boldsymbol{H} = \begin{bmatrix} 5 & 8 \\ 8 & 10 \end{bmatrix}$, classify the critical point without computing eigenvalues.
12. Why does the complex step method outperform central differences for small Δx?

CHAPTER 3

Solution Methods for Unconstrained Optimization Problems

Overview

Numerical optimization constructs a sequence of points intended to approach a minimizer. Two complementary tasks are:

1. **One-D search methods** - bracketing, Fibonacci search, golden section search, and the bisection method: how to trap a minimum in a shrinking interval, with or without derivatives.
2. **Multivariate methods** - the descent-direction framework, exact and approximate **line search** (the Wolfe conditions and backtracking), **Newton's method**, the **secant** and quasi-Newton (DFP) methods, and **trust-region** methods.

Core idea: every method here is the same loop - pick a direction, pick a step size, move, repeat until a termination condition fires. What separates the methods is how much derivative information they use to make those two choices.

Further reading

Kochenderfer, Mykel J., and Tim A. Wheeler. *Algorithms for Optimization*. MIT Press, 2019.

3.1 Recap and Two Formulation Exercises

The two problem forms

Constrained:

$$\begin{aligned} \text{minimize} \quad & f_0(\boldsymbol{x}) \\ \text{subject to} \quad & f_i(\boldsymbol{x}) \leq b_i, \quad i = 1, \ldots, m \\ & g_i(\boldsymbol{x}) = 0, \quad i = 1, \ldots, n \end{aligned}$$

Unconstrained:

$$\text{minimize} \quad f_0(\boldsymbol{x})$$

with $f_0 : \mathbb{R}^n \to \mathbb{R}$ and decision variables $\boldsymbol{x} = [x_1 \ \cdots \ x_n]^T \in \mathbb{R}^n$. The methods developed here address the unconstrained problem.

3.1.1 Exercise A - the cardboard box

Problem

Given a sheet of cardboard with dimensions l (length) and w (width). A box is formed by **cutting squares of equal size from each corner and folding up the sides**. The top and bottom of the box are open. It is required to **maximize the volume** of this box. The height of the box **should not exceed half**

of its width and half of its length. Write down the optimization problem.

Formulation

Let x be the side of the cut-out square - this is the **height** of the folded box. The base then measures $(l-2x)\times(w-2x)$, so the volume is $x(l-2x)(w-2x)$.

$$\begin{aligned} \underset{x}{\text{maximize}} \quad & x\times(l-2x)\times(w-2x) \\ \text{subject to} \quad & x>0 \\ & l-2x>0 \\ & w-2x>0 \\ & x\le\frac{l-2x}{2} \\ & x\le\frac{w-2x}{2} \end{aligned}$$

The first three constraints keep all three dimensions **physically positive**; the last two encode "height $\le$ half the length **and** half the width".

Reduction to one dimension

This is a **univariate** ($x\in\mathbb{R}$) problem - exactly the shape the one-D search methods below are built to solve.

3.1.2 Exercise B - the largest triangle in an ellipse

Problem

Given an ellipse $\frac{x^2}{a^2}+\frac{y^2}{b^2}=c^2$ for $a,b>0$, and two points $(-4ac,0)$ and $(ac,0)$ in (x,y) coordinates, formulate an optimization problem to find a **third point on the ellipse** that maximizes the area of the triangle formed by those three points.

Formulation and solution

The two given points lie on the x-axis, so the base of the triangle has fixed length $|ac-(-4ac)|=5ac$. The **height** is the vertical distance $|d|$ from the third point to the x-axis - that is, its y-coordinate.

$$\begin{aligned} \max_{x,y} \quad & \tfrac{1}{2}\,5ac\times d \\ \text{s.t.} \quad & \frac{x^2}{a^2}+\frac{y^2}{b^2}=c^2 \\ & -bc\le d\le 0 \quad \text{or} \quad 0\le d\le bc \end{aligned}$$

The ellipse's extreme vertical extent is $\pm bc$, so $d^*=bc$ or $d^*=-bc$, giving two optimal solutions:

$$(x^*,y_1^*)=(0,\ bc), \qquad (x^*,y_2^*)=(0,\ -bc)$$

Both are the top and bottom of the ellipse - the points farthest from the base.

3.2 How Optimization Problems Get Solved

Four general approaches

If a problem has multiple solutions, optimization involves identifying the **best** solution based on specific performance criteria. Several general approaches are available:

Approach	Nature
Analytical methods	Solve $\nabla f = 0$ by hand
Graphical methods	Read the answer off a plot
Experimental methods	Try candidate designs physically
Numerical methods	Iterative computation - the focus of numerical optimization

The numerical-method pattern

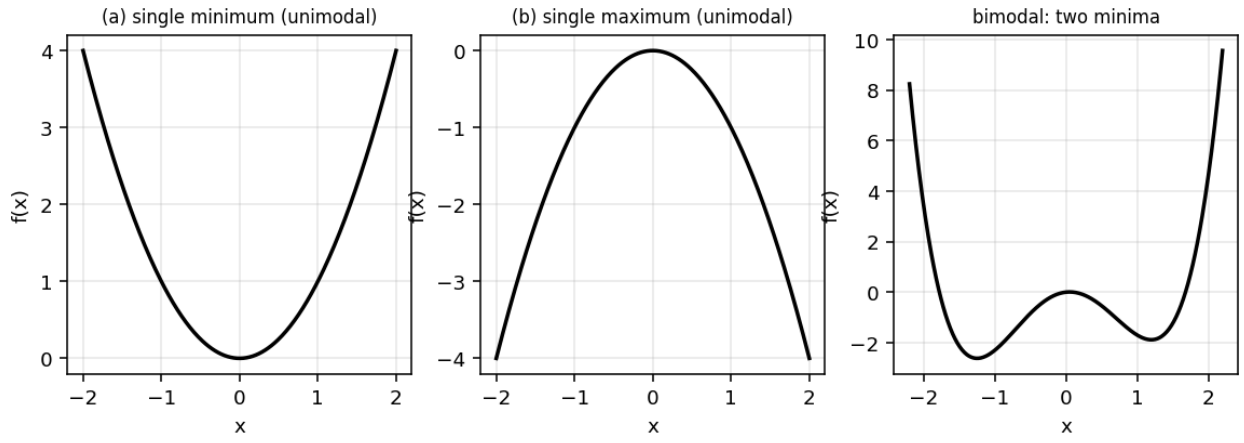
- These methods employ **iterative numerical procedures** to produce progressively refined solutions.
- The optimization process begins with an **initial estimate** for the solution.
- Iterations continue until a **convergence criterion** is met.

(Antoniou, Andreas, and Wu-Sheng Lu. Practical Optimization: Algorithms and Engineering Applications, Springer, 2007.)

3.2.1 Vocabulary for one-D methods

Univariate, unimodal, bimodal

- **Univariate function:** a function involving only a **single variable**.
- **Unimodal function:** a function that has a **single maximum or minimum point**.
- Formally, $f(x)$ is unimodal when there exists a unique value x^* such that $f(x)$ is **monotonically decreasing for** $x \leq x^*$ and **monotonically increasing for** $x \geq x^*$.
- This implies there is a **unique global minimum at** x^* (no other local minima).
- **Bimodal function:** a function that has **two maxima or minima**.



Unimodality is the assumption that makes interval-shrinking search valid: once you know the minimum is in $[a, b]$ and the function only falls then rises, discarding a sub-interval can never discard the answer. A bimodal function breaks that guarantee.

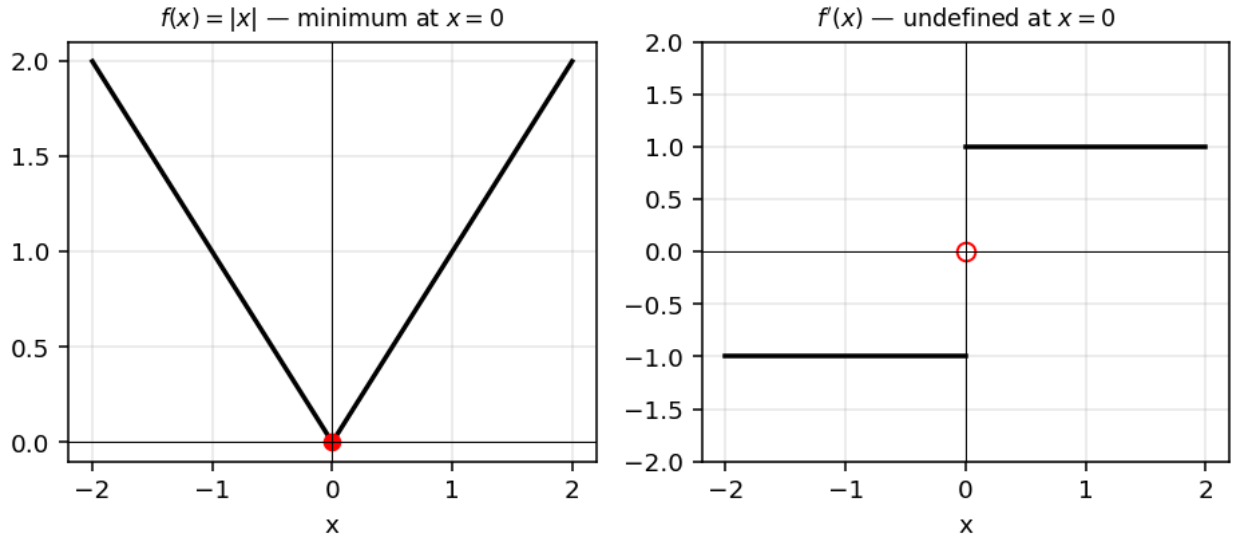
3.3 Bracketing

What bracketing is

- The process of **identifying an interval or range in which the minimum or maximum of a function is likely to exist**.
- Once the interval is identified, **shrinking the interval successively**.

Why do we need bracketing?

- Derivatives provide a way of identifying possible candidates for maxima or minima (local).
- For certain functions, the **derivative may be unavailable or nonexistent**.
- In such cases, optimization methods **without relying on derivatives** are necessary.



*The motivating example: $f(x) = |x|$ has a perfectly good minimum at $x = 0$, but its derivative is **not defined** there. FONC cannot be applied - yet a bracketing search finds the minimum without ever differentiating.*

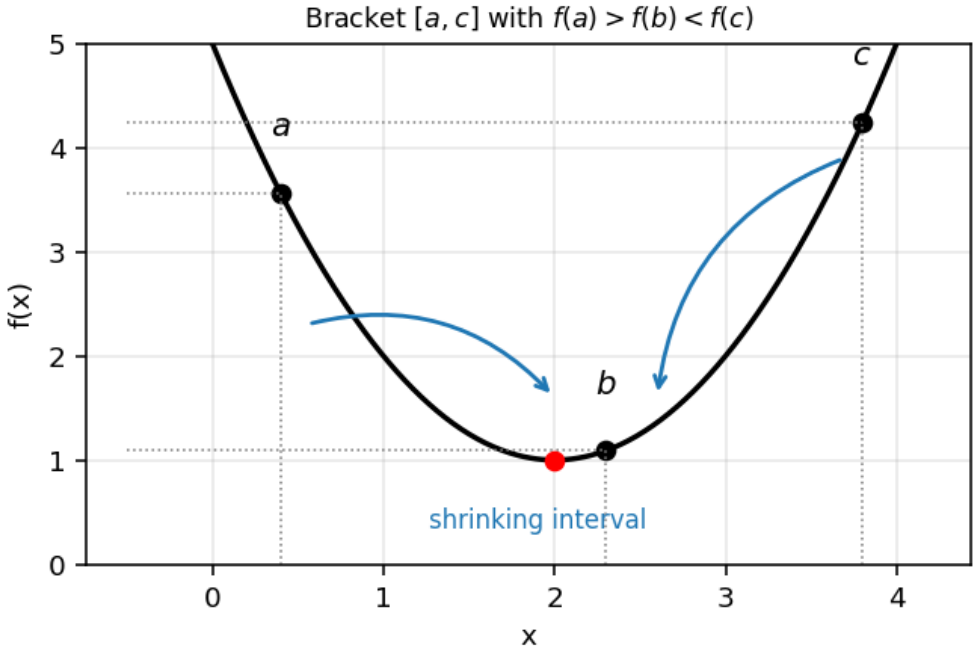
3.3.1 The bracketing condition

Three points that trap a minimum

Consider a **unimodal** function. We bracket an interval $[a, c]$ which contains the global minimum if we are able to identify **three points** $a < b < c$ such that

$$f(a) > f(b) < f(c)$$

The middle point is lower than both ends, so by unimodality the minimum must lie strictly between a and c.



Procedure

- Begins by **bracketing an interval with a local minimum**.
- **Successive reductions** in the size of the bracketed interval.
- The goal is to **identify the local minimum**.

3.3.2 Finding the initial bracket

How to identify an initial interval

- **Initialize with a single point** a (typically $a = 0$).
- **Move toward the positive direction in** x to get a new point b (i.e. $b = a + s$). Here s is the **step size** (a hyper-parameter).
- **If** $f(b) > f(a)$**, swap** a **and** b - the function increases in the $+x$ direction, so the minimum lies the other way.
- If a swap occurred, **move toward the negative direction** (i.e. $s = -s$) and consider a new point $c = b + s$.
- **Multiply the step size** s **by a factor** k to increase it for the next evaluation.
- Keep going **until** $f(c) > f(b)$. When $f(c) > f(b)$, **return** a **and** c.

Initialize
$a = x,\ b = a + s$. If $f(b) > f(a)$, swap a, b and reverse s.

↓

Expand
Compute $c = b + s$. If $f(c) > f(b)$, return the ordered outer endpoints a, c.

↓

Repeat
Otherwise set $a \leftarrow b$, $b \leftarrow c$, enlarge $s \leftarrow ks$, and test again.

Algorithm 1: Bracketing Algorithm

Data: Function f, initial point x, step size s, scaling factor k **Result:** Bracketed interval containing a local minimum $([a,\ b])$

```
a, f(a) ← x, f(x);
b, f(b) ← a + s, f(a + s);
if f(b) > f(a) then
    a, b ← b, a;
    f(a), f(b) ← f(b), f(a);
    s ← −s;

while true do
    c, f(c) ← b + s, f(b + s);
    if f(c) > f(b) then
        return a < c ? (a, c) : (c, a);
    a, f(a), b, f(b) ← b, f(b), c, f(c);
    s ← s × k;
```

3.3.3 Worked example

Trace the algorithm

Consider the function $f(x) = x^2 - 4x + 4$, initial point $x = 0$, step size $s = 0.1$, scaling factor $k = 2$.

- What is the initial interval $[a,\ b]$?
- What is the interval after two iterations?

Solution

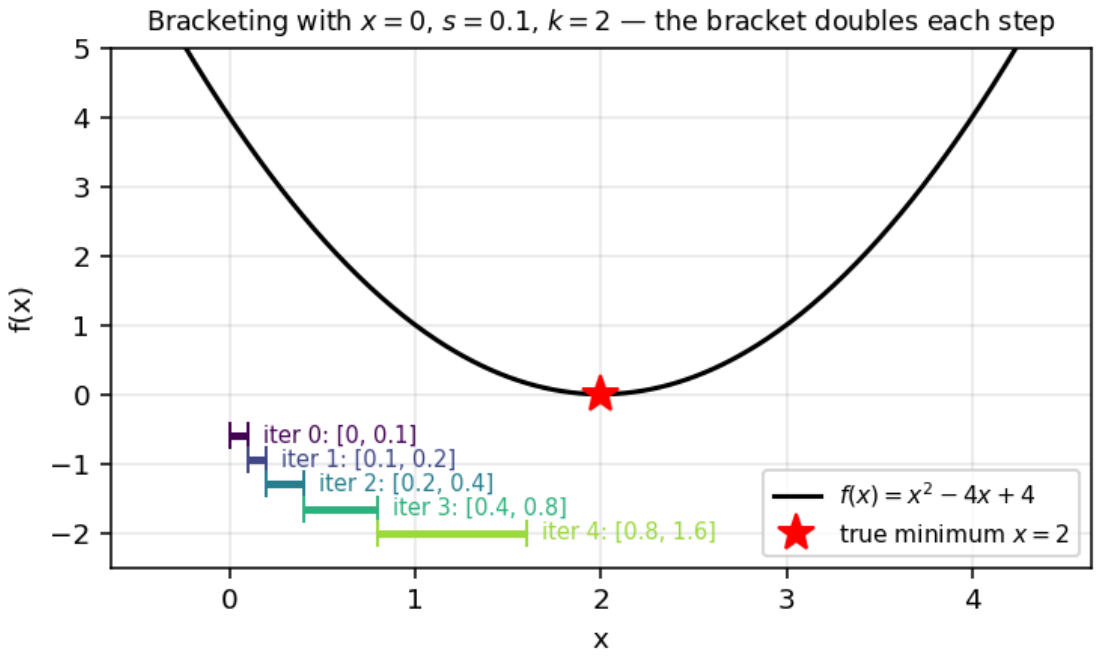
$f(x) = (x-2)^2$, so the true minimum is at $x = 2$.
Initialization: $a = 0$, $f(a) = 4$; $b = 0 + 0.1 = 0.1$, $f(b) = (0.1-2)^2 = 3.61$. Since $f(b) = 3.61 \not> f(a) = 4$, **no swap** - we are already walking downhill.

$$\boxed{\text{Initial interval } [a, b] = [0,\ 0.1]}$$

Iteration	$c = b + s$	$f(c)$	$f(c) > f(b)$?	New $[a, b]$	New s
1	$0.1 + 0.1 = 0.2$	3.24	No ($3.24 < 3.61$)	$[0.1,\ 0.2]$	0.2
2	$0.2 + 0.2 = 0.4$	2.56	No ($2.56 < 3.24$)	$[0.2,\ 0.4]$	0.4

$$\boxed{\text{After two iterations } [a, b] = [0.2,\ 0.4]}$$

The loop would continue $[0.4, 0.8] \to [0.8, 1.6] \to \ldots$ until it steps past $x = 2$ and $f(c)$ finally rises.



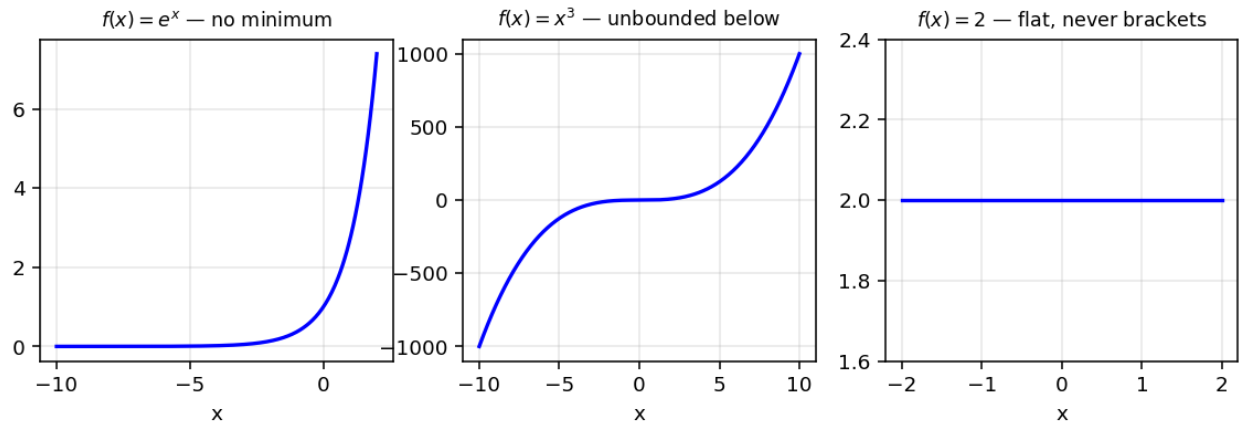
*Because s is multiplied by $k = 2$ every iteration, the bracket **grows geometrically** - it takes only a handful of function evaluations to reach the neighbourhood of the minimum, however far away it starts.*

3.3.4 When bracketing fails

Will Algorithm 1 work for these?

Function	What happens
$f(x) = e^x$	**No.** Strictly decreasing towards $-\infty$; $f(c)$ never exceeds $f(b)$, so the loop never terminates. There is no minimum to bracket.
$f(x) = x^3$	**No.** Unbounded below; the search runs off to $-\infty$ and never returns.
$f(x) = 2$	**No.** Constant, so $f(c) = f(b)$ always and the strict test $f(c) > f(b)$ is never true - the loop runs forever.

All three violate the assumption the method rests on: bracketing requires a unimodal function that actually **has** a minimum.



3.4 Fibonacci Search

The setting

Suppose the function f is

- **unimodal**,
- **bracketed by the interval** $[a, b]$,
- and the **number of query times is limited** (each evaluation of f is expensive).

The **Fibonacci search ensures (guarantees) the optimal reduction of the bracketed interval** for that fixed budget of queries.

3.4.1 The two-query rule

Which half survives

Place two interior points $a_1 < b_1$ inside $[a, b]$ and evaluate both:

- If $f(a_1) > f(b_1) \Rightarrow$ the minimizer is in the range $[a_1,\ b]$ - discard $[a, a_1]$.
- If $f(b_1) > f(a_1) \Rightarrow$ the minimizer is in the range $[a,\ b_1]$ - discard $[b_1, b]$.

With **two queries** of f, we are able to **remove one-third of the interval - irrespective of** f (given unimodality).

Tighter bracket

Shifting our guesses towards the centre of the interval gives a bigger cut: as $\epsilon \to 0$ (the two points nearly coincide at the midpoint), this **guarantees a reduction of the interval by a factor of two**. The catch is that when the two points coincide, the *next* iteration learns nothing - which is exactly the trade-off Fibonacci search resolves optimally.

Three-query example

- **First step:** one-third and two-third segments selection.
- **Second step:** the two-third segment is selected.
- **Third step:** half from the two-third segment is selected.
- With three queries, one-third of the interval can be selected (**shrink the interval by three**).

3.4.2 The Fibonacci sequence

Definition

The relationship between the number of queries (n) and interval lengths follows the **Fibonacci sequence**. It begins with $1, 1$, then continues $2, 3, 5, 8, \ldots$; for $n > 2$ each subsequent term is the sum of the two preceding terms:

$$F_n = \begin{cases} 1 & \text{if } n \leq 2 \\ F_{n-1} + F_{n-2} & \text{otherwise} \end{cases}$$

F_1	F_2	F_3	F_4	F_5	F_6	F_7	…	F_n
1	1	2	3	5	8	13	…	$F_{n-1} + F_{n-2}$

Binet's formula

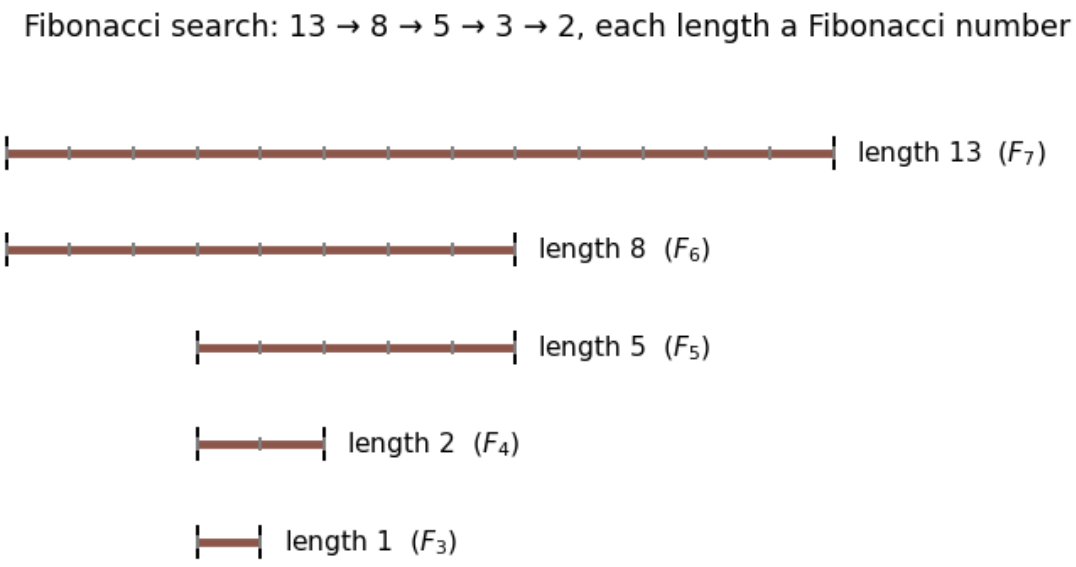
The Fibonacci sequence can be determined analytically:

$$F_n = \frac{\phi^n - (1-\phi)^n}{\sqrt{5}}, \quad \text{where the golden ratio } \phi = \frac{1+\sqrt{5}}{2} \approx 1.61803$$

The ratio between successive values is

$$\frac{F_n}{F_{n-1}} = \phi \frac{1 - s^{n+1}}{1 - s^n}, \quad \text{where } s = \frac{1-\sqrt{5}}{1+\sqrt{5}} \approx -0.382$$

3.4.3 Why Fibonacci numbers are the right lengths



Each interval's length in Fibonacci search is a ***Fibonacci number multiplied by the length of the final (smallest) interval****. Reading the lengths backwards* $13 \to 8 \to 5 \to 3 \to 2$ *shows why every already-evaluated interior point can be* ***reused*** *in the next iteration - this is what makes the method optimal for a fixed query budget.*

The nesting relations

If the final (smallest) interval has length I_n:

- The second smallest interval has a length of $I_{n-1} = F_2 I_n$
- The third smallest interval has a length of $I_{n-2} = F_3 I_n$

E.g. let the smallest interval be I_4:

$$I_3 = F_2 I_4, \qquad I_2 = I_3 + I_4 = F_3 I_4, \qquad I_1 = I_2 + I_3 = F_4 I_4$$

Reduction factors ρ_k

For a unit-length interval ($b - a = 1$), the fraction kept at iteration k is:

$$\rho_1 = 1 - \frac{F_N}{F_{N+1}}, \quad \rho_2 = 1 - \frac{F_{N-1}}{F_N}, \quad \rho_k = 1 - \frac{F_{N-k+1}}{F_{N-k+2}}, \quad \rho_N = 1 - \frac{F_1}{F_2}$$

Total reduction after n queries

$$(1-\rho_1)(1-\rho_2)(1-\rho_3)\cdots(1-\rho_{N-1})(1-\rho_N)$$
$$= \frac{F_N}{F_{N+1}} \times \frac{F_{N-1}}{F_N} \times \frac{F_{N-2}}{F_{N-1}} \times \cdots \times \frac{F_2}{F_3} \times \frac{F_1}{F_2} = \boxed{\frac{1}{F_{N+1}}}$$

This telescoping product is the headline result: N **queries shrink the interval by a factor of** F_{N+1}**.**

3.4.4 The last-iteration problem

The two interior points coincide

At the last iteration, the two intermediate points **coincide at the midpoint**:

$$x_1 = x_2 = \frac{a+b}{2}$$

To keep them distinct, use a small perturbation, performing the last iteration as $\frac{1}{2} - \epsilon$ with $\epsilon > 0$. Hence the **worst-case reduction factor** is

$$\frac{1+2\epsilon}{F_{N+1}} \approx \frac{1}{F_{N+1}}$$

Proof of the worst-case factor

Consider the worst case. The current interval length L_k is proportional to F_3 (which is 2). Its proportion relative to the initial interval L_0 is

$$L_k = \frac{F_3}{F_{N+1}} L_0 = \frac{2}{F_{N+1}} L_0$$

The two test points divide the current interval L_k using the coefficients $(1/2 - \epsilon)$ and $(1/2 + \epsilon)$. Once the function is evaluated at these offset points, the algorithm discards one side. **In the worst case, the algorithm retains the *larger* of the remaining segments:**

$$L_f = \left(\frac{1}{2} + \epsilon\right) L_k = \left(\frac{1}{2} + \epsilon\right)\left(\frac{2}{F_{N+1}}\right) L_0$$

Multiplying out:

$$L_f = \left(\frac{1}{2}\frac{2}{F_{N+1}} + \epsilon\frac{2}{F_{N+1}}\right) L_0 = \left(\frac{1}{F_{N+1}} + \frac{2\epsilon}{F_{N+1}}\right) L_0 = \frac{1+2\epsilon}{F_{N+1}} L_0$$

Dividing by L_0 yields the final reduction factor $\frac{1+2\epsilon}{F_{N+1}}$.

3.4.5 Choosing the number of queries N

The design rule

Say the required range of x is p. Then

$$\frac{1+2\epsilon}{F_{N+1}} \leq \frac{\text{final range}}{\text{initial range}} = \frac{p}{|b-a|}$$

Worked example

Initial range $[-2,\ 4]$, $\epsilon = 0.1$, required range of x is 0.2.

$$\frac{1.2}{F_{N+1}} \leq \frac{0.2}{6} \implies F_{N+1} \geq 36$$

From the sequence $1, 1, 2, 3, 5, 8, 13, 21, 34, 55$: the first Fibonacci number ≥ 36 is $F_{10} = 55$.

$$F_{N+1} = F_{10} \implies N+1 = 10 \implies \boxed{N = 9 \text{ is sufficient}}$$

Fibonacci search on $f(x) = x^2$ over $[a, b] = [-2,\ 6]$

Let $N = 5$. Then

$$\rho_1 = 1 - \frac{F_5}{F_6} = 1 - \frac{5}{8} = \frac{3}{8}$$

Evaluate at the two interior points ($b - a = 8$):

$$f\left(a + (b-a)\left(1 - \frac{F_5}{F_6}\right)\right) = f(1) = 1$$

$$f\left(a + (b-a)\left(\frac{F_5}{F_6}\right)\right) = f(3) = 9$$

Since $f(1) = 1$ is lower, the **new interval is** $[a, b] = [-2,\ 3]$ - the segment $[3, 6]$ is discarded.
Next two points: $a + (b-a)\left(1 - \frac{F_4}{F_5}\right)$ and $a + (b-a)\left(\frac{F_4}{F_5}\right)$. Then select the new interval (say $[a_0, b_0]$), whose next two points are $a_0 + (b_0 - a_0)\left(1 - \frac{F_3}{F_4}\right)$ and $a_0 + (b_0 - a_0)\left(\frac{F_3}{F_4}\right)$ - and so on, the ratio index dropping by one each iteration.

3.5 Golden Section Search

The idea

Fibonacci search uses a **different ρ from iteration to iteration**, and requires knowing N in advance. Golden section search replaces every ρ_k with the **limit** of the Fibonacci ratios - one fixed number, no budget needed up front.

	F_1/F_0	F_2/F_1	F_3/F_2	F_4/F_3	F_5/F_4	F_6/F_5	…
ratio	1	2	1.5	1.666	1.6	1.625	…

The golden ratio

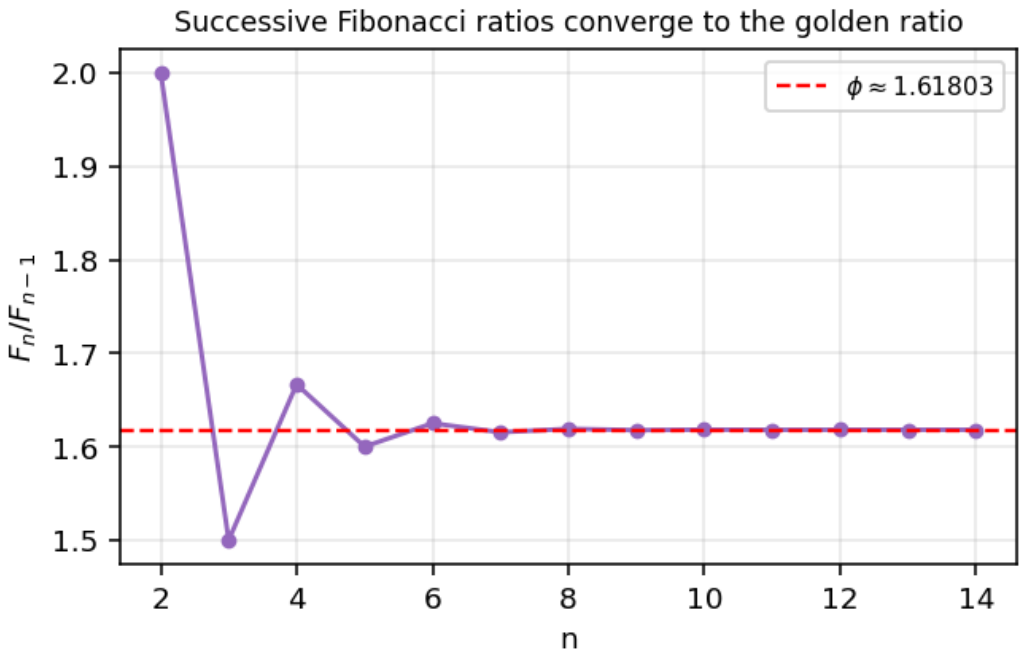
$$\lim_{n\to\infty}\frac{F_n}{F_{n-1}}=\phi,\quad \text{where } \phi=\frac{1+\sqrt{5}}{2}\approx 1.61803$$

Hence

$$1-\rho\approx 0.61803,\qquad \rho\approx 0.382$$

N steps reduce the range by a factor of approximately $(0.61803)^N$:

$$(0.61803)^N\le\frac{\text{final range}}{\text{initial range}}=\frac{p}{|b-a|}$$



The successive Fibonacci ratios oscillate above and below ϕ and settle onto it within a few terms - which is why golden section search performs almost identically to Fibonacci search while being far simpler to implement.

Fibonacci vs golden section - the trade-off

	Fibonacci search	Golden section search
ρ per iteration	Different each iteration	Fixed ≈ 0.382
Needs N in advance	**Yes**	No
Total reduction	$1/F_{N+1}$ **(optimal)**	$\approx (0.618)^N$ (near-optimal)
Implementation	More bookkeeping	Simple

3.6 The Bisection Method

From root-finding to optimization

- The bisection method is a **root-finding method - roots** are points where the function is zero.
- **How to use this method for optimization?** By **finding roots of the derivative of the objective function**, since FONC says $f'(x^*) = 0$ at any interior minimum.

Bracketing a root

Consider a function $f(x)$ which is continuous in the bracket $[a, b]$. **What interval contains roots?** The one where the function changes sign:

$$f(a) \times f(b) \leq 0$$

Algorithm

1. Select an interval $[a,\ b]$ such that the function has **different signs at** a **and** b.
2. Compute the **midpoint** $c = (a + b)/2$.
3. If $f(c) = 0$, **return** c as the root and terminate the algorithm.
4. Otherwise, **update the interval**:
   - If $f(a)$ and $f(c)$ have **opposite signs**, set $b = c$.
   - If $f(b)$ and $f(c)$ have **opposite signs**, set $a = c$.

Bracket a continuous root
$f(a)f(b) < 0$; set $c = (a + b)/2$.

↓

Keep the sign change
If $f(c) = 0$, stop. Otherwise retain $[a, c]$ or $[c, b]$ with opposite endpoint signs.

↓

Repeat
Stop when the interval width meets the tolerance; otherwise bisect again.

Convergence rate

This method will converge to within ε of x^* within

$$\log_2\left(\frac{|b-a|}{\varepsilon}\right) \text{ iterations}$$

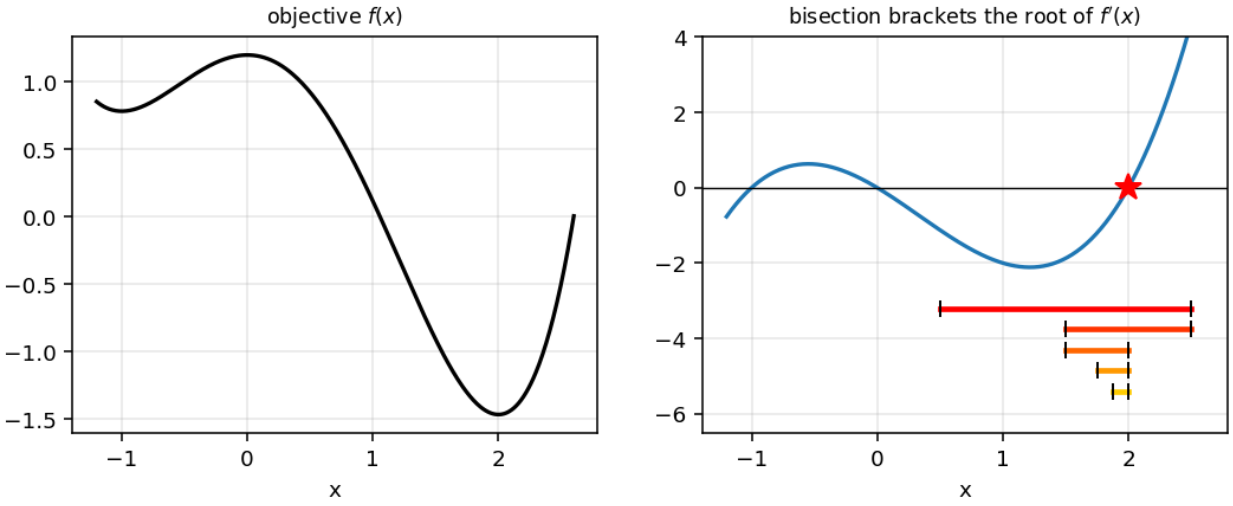
The interval halves every step, so the accuracy gained is **linear in the number of iterations** (one bit per iteration) - slower than Newton, but utterly reliable.

3.6.1 Using bisection for optimization

Apply it to f', not f

- In optimization, we are looking to **find roots of** $f'(x)$.
- At the initial interval, we need to make sure that

$$\text{sign}\left(f'(a)\right) \neq \text{sign}\left(f'(b)\right)$$



Left: the objective. Right: its derivative, with successive bisection brackets stacked below the axis, each half the width of the one above, closing on the root of f' - which is the minimum of f.

What if the signs are the same?

A simple approach:

- Begin with an interval $[a,\ b]$.
- If the interval is **invalid, increase its width by a constant factor** (say 2).

A sign-changing derivative bracket may not exist. For $f(x) = e^x$, the derivative $f'(x) = e^x > 0$ everywhere, so widening the interval cannot bracket a stationary point. Bracket expansion therefore needs a maximum number of evaluations or a domain limit.

Brent–Dekker method

- An **extension of the bisection method**.
- It **combines the secant method and inverse quadratic interpolation**.
- Known for its **reliable and fast convergence** properties.
- **Widely used in many optimization packages** (it is what SciPy's `brentq`/`minimize_scalar` use by default).
- References: Brent, Richard P. *Algorithms for Minimization without Derivatives*, Courier Corporation, 2013; Dekker, T. J. "Finding a zero by means of successive linear interpolation," *Constructive Aspects of the Fundamental Theorem of Algebra*, 1969.

3.7 Descent Direction - The Multivariate Framework

The universal update rule

Iteratively update the design point $\boldsymbol{x}$ by taking steps that minimize the objective function (based on a local model):

$$\boldsymbol{x}^{(k+1)} \leftarrow \boldsymbol{x}^{(k)} + \alpha^{(k)}\boldsymbol{d}^{(k)}$$

- $\alpha^{(k)}$ **- step size / learning rate**
- $\boldsymbol{d}^{(k)}$ **- descent direction**

Initialize and test termination
Choose $x^{(0)}$ and tolerances.

↓

Choose a direction and step
Find a descent direction $d^{(k)}$ and a suitable $\alpha^{(k)}$.

↓

Update and repeat
$x^{(k+1)} = x^{(k)} + \alpha^{(k)}d^{(k)}$; return to the termination test.

Direction, step size, and iteration

Every algorithm from here to the end of the course is a choice of **how to pick** $\boldsymbol{d}^{(k)}$ (gradient descent, Newton, conjugate gradient, quasi-Newton) and **how to pick** $\alpha^{(k)}$ (exact line search, backtracking, fixed schedule).

3.8 Line Search

The step-size subproblem

Assume that the descent direction $\boldsymbol{d}$ is known; we need to determine the step size α:

$$\min_{\alpha} f(\boldsymbol{x} + \alpha\boldsymbol{d})$$

- This is a **univariate optimization problem** - it is possible to use the **1-D methods** such as Fibonacci search or golden-section search.
- This method can be **computationally expensive** (determining the step size α in every iteration).
- As an alternative: **find a satisfactory value for α and proceed**.
- **Fixed scale factor** for step size: $\alpha^{(k+1)} = \beta\alpha^{(k)}$ with $\beta > 1$.
- **Decaying scale factor:** $\alpha^{(k+1)} = \beta\alpha^{(k)}$ with $\beta \in [0,\ 1]$.

Exact and approximate line search

Picture a car driving down a winding valley. **(a)** Find the *optimal* value for α at each point and proceed - accurate, but you stop and survey at every step. **(b)** Find a *satisfactory* value for α and proceed - you cover the same ground with far less surveying. Approximate line search is option (b).

3.8.1 Worked example

Setting up the line-search objective

$f(x_1, x_2, x_3) = (x_1 x_2)^2 + (x_2 + x_3)^2 - x_3$, with $\boldsymbol{d} = [0,\ -1,\ -1]$ and $\boldsymbol{x} = [1,\ 2,\ 3]$.

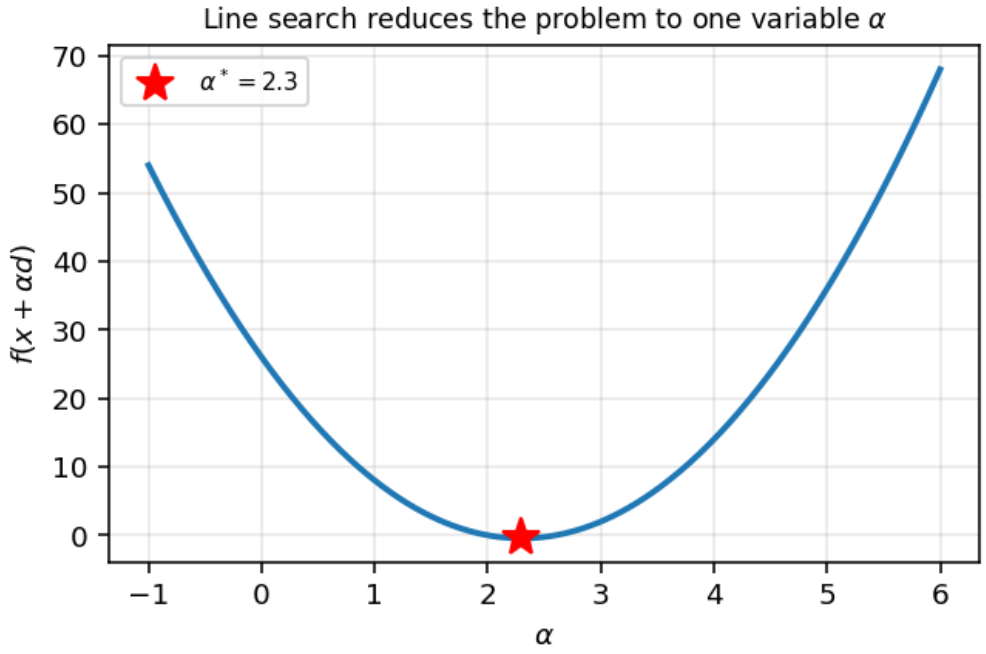
$$\boldsymbol{x} + \alpha \boldsymbol{d} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \alpha \begin{bmatrix} 0 \\ -1 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 - \alpha \\ 3 - \alpha \end{bmatrix}$$

Substituting into f:

$$\min_{\alpha} \ ((x_1 + 0\alpha)(x_2 - 1\alpha))^2 + (x_2 - 1\alpha + x_3 - 1\alpha)^2 - (x_3 - 1\alpha)$$

$$\boxed{\min_{\alpha} (2 - \alpha)^2 + (5 - 2\alpha)^2 - (3 - \alpha)}$$

A single-variable problem in α.



The multivariate objective restricted to the line through $\boldsymbol{x}$ in direction $\boldsymbol{d}$ - a clean 1-D parabola whose minimum is the step size.

Solving and iterating

Setting $\frac{d}{d\alpha}\left[(2 - \alpha)^2 + (5 - 2\alpha)^2 - (3 - \alpha)\right] = -2(2 - \alpha) - 4(5 - 2\alpha) + 1 = 10\alpha - 23 = 0$:

$$\alpha = 2.3$$

Updated $\boldsymbol{x} = [x_1,\ x_2 - \alpha,\ x_3 - \alpha] = [1,\ -0.3,\ 0.7]$.
Next iteration: say $\boldsymbol{d} = [-1,\ -1,\ -1]$. We must solve a **new** optimization problem to estimate α:

$$\min_{\alpha} \ ((1 - \alpha)(-0.3 - \alpha))^2 + (-0.3 - \alpha + 0.7 - \alpha)^2 - (0.7 - \alpha)$$

3.8.2 Exercises

Exercise 1

$f(x_1, x_2, x_3) = (x_1 x_3)^2 + \exp(x_2 + x_1)$, $\boldsymbol{d} = [-1,\ -1,\ -1]$ and $\boldsymbol{x} = [0,\ 1,\ 2]$. **Write down the objective function of line search.**

Solution 1

$\boldsymbol{x} + \alpha \boldsymbol{d} = [-\alpha,\ 1-\alpha,\ 2-\alpha]$. Substituting:

$$\min_{\alpha}\ ((-\alpha)(2-\alpha))^2 + \exp((1-\alpha)+(-\alpha)) \ = \ \min_{\alpha}\ (\alpha^2 - 2\alpha)^2 + e^{1-2\alpha}$$

Exercise 2

$f(x_1, x_2, x_3) = (x_1 x_2)^2 + x_3^2$, $\boldsymbol{d} = [0,\ -2,\ -1]$ and $\boldsymbol{x} = [1,\ 2,\ 1]$.

1. Write down the objective function of line search.
2. What is the optimal value for step size α?
3. What is the next design point?
4. In the next iteration, suppose $\boldsymbol{d} = [-1,\ -1,\ -1]$; write down the new optimization problem used to estimate α.

Solution 2

1. $\boldsymbol{x} + \alpha \boldsymbol{d} = [1,\ 2-2\alpha,\ 1-\alpha]$, so

$$\min_{\alpha}\ (1\cdot(2-2\alpha))^2 + (1-\alpha)^2 = \min_{\alpha}\ 4(1-\alpha)^2 + (1-\alpha)^2 = \min_{\alpha}\ 5(1-\alpha)^2$$

2. $5(1-\alpha)^2$ is minimized when $(1-\alpha)^2 = 0$, so $\boxed{\alpha^* = 1}$ (objective value 0). **3.** Next design point $= [1,\ 2-2(1),\ 1-1] = \boxed{[1,\ 0,\ 0]}$. **4.** With $\boldsymbol{d} = [-1,-1,-1]$ at $\boldsymbol{x} = [1,0,0]$: $\boldsymbol{x} + \alpha \boldsymbol{d} = [1-\alpha,\ -\alpha,\ -\alpha]$, giving

$$\min_{\alpha}\ ((1-\alpha)(-\alpha))^2 + (-\alpha)^2 = \min_{\alpha}\ (\alpha^2 - \alpha)^2 + \alpha^2$$

3.9 Approximate Line Search and the Wolfe Conditions

Why approximate

- **More iterations of a descent method can be computationally better than exact line search** (when function and derivative calculations are costly).
- **Approximate line search:** find a *suitable* step size with a **small number of evaluations**.
- **What is a good step size?** Our objective is to **decrease the objective function as we move**:

$$f(\boldsymbol{x}^{(k)}) - f(\boldsymbol{x}^{(k+1)}) \propto \alpha$$

$$f(\boldsymbol{x}^{(k)}) - f(\boldsymbol{x}^{(k+1)}) \geq \gamma\alpha \text{ with } \gamma > 0$$

$$f(\boldsymbol{x}^{(k+1)}) \leq f(\boldsymbol{x}^{(k)}) - \gamma\alpha$$

3.9.1 The first Wolfe condition (sufficient decrease / Armijo)

Statement

When we consider the local slope $\gamma = -\beta \nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k)})$, the **first Wolfe condition** is

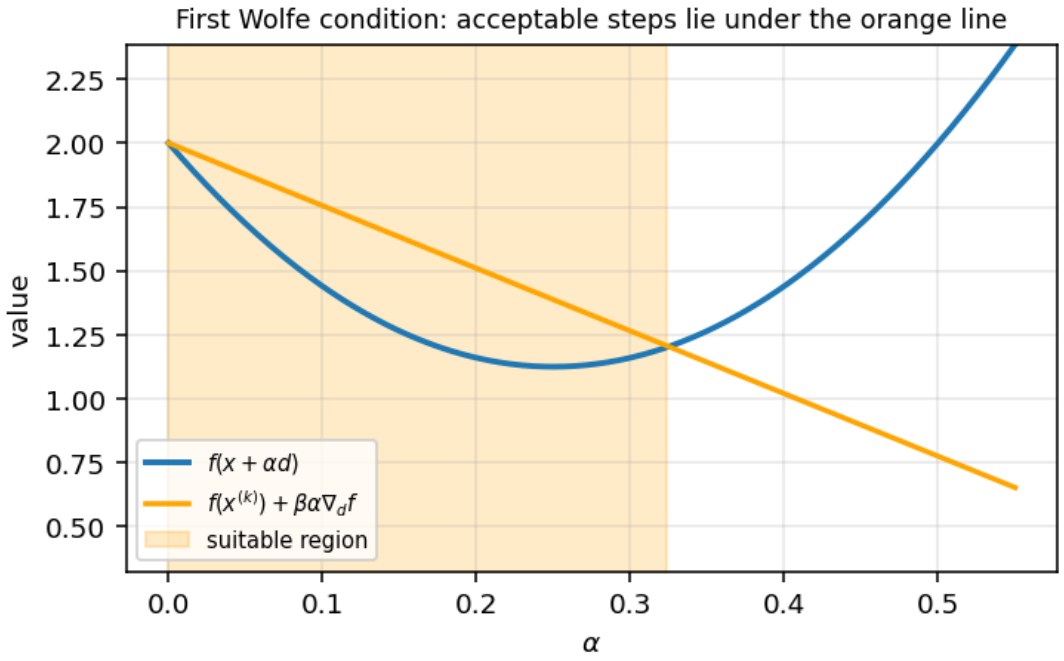
$$f(\boldsymbol{x}^{(k+1)}) \leq f(\boldsymbol{x}^{(k)}) + \beta\alpha\nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k)}) \quad \text{with } \beta \in [0,\ 1]$$

(Typically β is set as 10^{-4}.)

How β behaves

- $\beta = 0 \rightarrow$ **any decrease is acceptable**.
- $\beta = 1 \rightarrow$ the decrease needs to be **at least as much as the prediction of a first-order approximation** ($f(x) \approx f(x_0) + f'(x_0)(x - x_0)$).
- When $\beta \rightarrow 0$, **bigger step sizes are permitted**.
- When $\beta \rightarrow 1$, **small step sizes are permitted**.

The first Wolfe condition rejects steps which are too large.



*The blue curve is the true 1-D slice $f(\boldsymbol{x} + \alpha\boldsymbol{d})$; the orange line is the sufficient-decrease threshold $f(\boldsymbol{x}^{(k)}) + \beta\alpha\nabla_{\boldsymbol{d}} f$. Any α where the curve sits **below** the line is acceptable - the shaded "suitable region".*

3.9.2 Backtracking line search

The search strategy

Suppose $\boldsymbol{d}$ is a valid direction $\rightarrow$ there **must be a small step size that meets the sufficient decrease condition**. **Backtracking line search: begin with a large step size and decrease it consistently (by a factor) until the sufficient decrease condition is met.**
(Also known as the Armijo line search - Armijo, Larry. "Minimization of functions having Lipschitz

continuous first partial derivatives." Pacific Journal of Mathematics 16.1 (1966): 1–3.)

Algorithm 1: Backtracking Line Search

Input: f, ∇f, $\boldsymbol{x}$, $\boldsymbol{d}$, α, p, β **Output:** α

```
y ← f(x)
g ← ∇f(x)
while f(x + α·d) > y + βα··(ᵀg·d) do
    α ← α · p
return α
```

Why backtrack rather than start small? Extremely tiny step sizes might satisfy the sufficient decrease condition but cause **premature convergence**. Backtracking prevents this by accepting the **largest** step size through gradual reduction.

3.9.3 The second Wolfe condition (curvature)

Statement

- $\alpha = 0 \rightarrow \nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k)}) < 0$ (at the starting point, for the direction to be valid, we need to move in a descent direction).
- Say we reached a local minimum: $\nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k)} + \alpha \boldsymbol{d}) = \nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k+1)}) = 0$.
- Directional derivative increment: $|\nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k)})|$.
- **To avoid short steps**, we make sure that a factor of the above is achieved:

$$\frac{\nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k+1)})}{\nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k)})} \leq \sigma$$

  Since $\nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k)}) < 0$, multiplying through **flips the inequality**:

$$\boxed{\nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k+1)}) \geq \sigma \nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k)})}$$

- $\sigma \rightarrow 0$: **large step sizes are permitted**.
- $\sigma \rightarrow 1$: **small step sizes are permitted**.

Can the ratio be negative?

Yes!

- The **denominator is always negative** ($\boldsymbol{d}^{(k)}$ is a descent direction).
- The **numerator can be negative** (still on a downward slope) **or positive** (past the minimum, on an upward slope).
- Dividing a positive numerator by a negative denominator yields a **negative ratio**, which satisfies $\leq \sigma$ for $\sigma > 0$.

Strong Wolfe condition

The parameter σ determines the level of reduction required for the directional derivative at the next

iteration. The **strong Wolfe condition** is an alternative to the curvature condition (more restrictive):

$$\left|\nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k+1)})\right| \leq -\sigma \nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k)})$$

- The curvature condition only prevents the derivative from being **too negative** (too steep downhill). However, it **allows the step to overshoot the minimum massively and head steeply uphill** (where the derivative becomes a very large positive number).
- The strong Wolfe condition uses an **absolute value** $|\cdot|$ to prevent the derivative from being too large **in either direction** (neither too steep downhill nor too steep uphill). It forces the new point to be close to a local minimum where the directional derivative is near zero.
- The combination of the sufficient decrease condition with the curvature condition forms the **strong Wolfe condition**.
- *Typical settings:* $\sigma = 0.1$ and $0 < \beta < \sigma < 1$.

The two conditions summarised

Condition	Formula	What it rejects
First Wolfe (sufficient decrease)	$f(\boldsymbol{x}^{(k+1)}) \leq f(\boldsymbol{x}^{(k)}) + \beta\alpha\nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k)})$	Steps which are **too large**
Second Wolfe (curvature)	$\nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k+1)}) \geq \sigma\nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k)})$	Steps which are **too small** (guarantees sufficient progress)

Parameters ordered on the unit line: $0 < \beta < \sigma < 1$. *(Note: σ needs to be strictly greater than β.)*

3.9.4 Worked example - backtracking through the first Wolfe condition

Setup

$f(\boldsymbol{x}) = x_1 + x_1x_2 + x_1^2$, $\boldsymbol{x} = [1,\ 2]$, $\boldsymbol{d} = [-1,\ -2]$, maximum step size 10, reduction factor 0.5, $\beta = 1 \times 10^{-4}$, $\sigma = 0.9$.
Useful quantities: $\nabla_{\boldsymbol{x}} = [1 + x_2 + 2x_1,\ x_1]$, so at $\boldsymbol{x} = [1, 2]$: $\nabla f = [5,\ 1]^T$ and $f(\boldsymbol{x}) = f([1, 2]) = 4$.
The directional derivative is $\nabla_{\boldsymbol{d}} f = [5, 1][-1, -2]^T = -7$.

The backtracking trace

Condition tested each time: $f(\boldsymbol{x}^{(k+1)}) \leq f(\boldsymbol{x}^{(k)}) + \beta\alpha\nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k)})$

α	$\boldsymbol{x} + \alpha\boldsymbol{d}$	$f(\boldsymbol{x}^{(k+1)})$	Threshold $4 + \beta\alpha(-7)$	Satisfied?
10	$[-9,\ -18]$	234	3.993	×
5	$[-4,\ -8]$	44	3.9965	×
2.5	$[-1.5,\ -3]$	5.25	3.99825	×
1.25	$[-0.25,\ -0.5]$	-0.0625	3.999125	✓

The first Wolfe condition is satisfied at $\alpha = 1.25$, and the candidate design point is

$$\boldsymbol{x} + \alpha\boldsymbol{d} = [-0.25,\ -0.50]$$

Checking the second Wolfe condition at that point

At $\boldsymbol{x}^{(k+1)} = [-0.25,\ -0.5]$: $\nabla f = [1 + (-0.5) + 2(-0.25),\ -0.25] = [0,\ -0.25]$.

$$\nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k+1)}) \geq \sigma \nabla_{\boldsymbol{d}^{(k)}} f(\boldsymbol{x}^{(k)})$$

$$[0,\ -0.25]^T[-1,\ -2] \geq 0.9\,[5,\ 1]^T[-1,\ -2]$$

$$0.5 \geq -6.3$$

The second Wolfe condition is satisfied. ✓

3.9.5 Exercises

Exercise 3

$f(\boldsymbol{x}) = x_1 + x_1 x_2 + x_1^2$, $\boldsymbol{x} = [1,\ 2]$, $\boldsymbol{d} = [-1\ \ -1]$, maximum step size 10, reduction factor 0.5, $\beta = 1 \times 10^{-2}$, $\sigma = 0.9$.

1. Determine the step size α that satisfies the first Wolfe condition.
2. What is the design point?
3. Compare your result with $\beta = 1 \times 10^{-4}$.

*(Point of the comparison: a larger β demands a steeper decrease, so it **rejects more step sizes** and forces more backtracking - smaller accepted α.)*

Exercise 4 (from Kochenderfer & Wheeler, 2019)

$f(\boldsymbol{x}) = x_1^2 + x_1 x_2 + x_2^2$, $\boldsymbol{x} = [1,\ 2]$, $\boldsymbol{d} = [-1\ \ -1]$, maximum step size 10, reduction factor 0.5, $\beta = 1 \times 10^{-4}$, $\sigma = 0.9$.

1. Determine the step size α that satisfies the first Wolfe condition.
2. What is the design point?
3. Show that this design point satisfies the second Wolfe condition.

Answer to Exercise 4

Step size $\alpha = 2.5$, design point $\boldsymbol{x} = [-1.5,\ -0.5]$. This design point satisfies the second Wolfe condition.
Check: $f([1,2]) = 1 + 2 + 4 = 7$; $\nabla f = [2x_1 + x_2,\ x_1 + 2x_2] = [4,\ 5]$; $\nabla_{\boldsymbol{d}} f = -9$.

- $\alpha = 10 \Rightarrow [-9, -8]$, $f = 81 + 72 + 64 = 217 > 7 - 0.009$ ×
- $\alpha = 5 \Rightarrow [-4, -3]$, $f = 16 + 12 + 9 = 37$ ×
- $\alpha = 2.5 \Rightarrow [-1.5, -0.5]$, $f = 2.25 + 0.75 + 0.25 = 3.25 \leq 6.99775$ ✓

Curvature: at $[-1.5, -0.5]$, $\nabla f = [-3.5,\ -2.5]$, so $\nabla_{\boldsymbol{d}} f = 3.5 + 2.5 = 6.0 \geq 0.9(-9) = -8.1$ ✓

3.10 Newton's Method

The idea

- A **root-finding method** - roots are points where the function is zero.
- **It's applied to the derivative of a function** (since we want $f'(x) = 0$).
- **It iteratively enhances a design point.**

Mechanically: we use the **tangent line at** $(x,\ f'(x))$, find the **intersection of this line with the x-axis**, and the x value of this intersection becomes the **next design point**.

Derivation from the tangent line

Tangent line at x_0:

$$f'(x_0) = \frac{f(x_0) - 0}{x_0 - x_1} \implies x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

In general (also known as **Newton's method of tangents**):

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$$

Replacing $f(x)$ **by** $f'(x)$ to find a stationary point:

$$\boxed{x^{(k+1)} = x^{(k)} - \frac{f'(x^{(k)})}{f''(x^{(k)})}}$$

Multivariate optimization:

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \frac{f(\boldsymbol{x}^{(k)})}{\boldsymbol{H}(\boldsymbol{x}^{(k)})} = \boldsymbol{x}^{(k)} - \frac{\boldsymbol{g}^{(k)}}{\boldsymbol{H}^{(k)}} = \boldsymbol{x}^{(k)} - \left(\boldsymbol{H}^{(k)}\right)^{-1} \boldsymbol{g}^{(k)}$$

where $\boldsymbol{g}^{(k)} = \nabla f(\boldsymbol{x}^{(k)})$ and $\boldsymbol{H}^{(k)} = \nabla^2 f(\boldsymbol{x}^{(k)})$. Note that f **needs to be twice differentiable**.

3.10.1 Geometric interpretation: quadratic approximation

The model being minimized

Taylor expansion of $f(x)$ around a point x_0:

$$q(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \frac{f'''(x_0)}{3!}(x - x_0)^3 + \dots$$

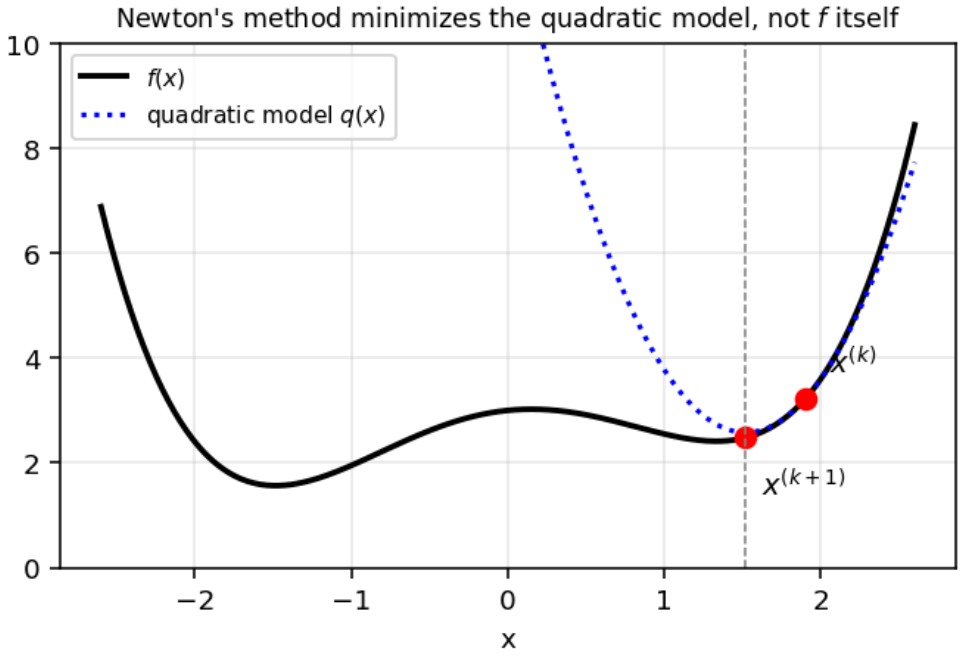
Second-order Taylor expansion (truncate after the quadratic term):

$$q(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2$$

We minimize the quadratic approximation rather than the original function at x_0.
Setting the derivative of $q(x)$ to zero gives the next design point:

$$\frac{\partial q(x)}{\partial x} = f'(x^{(k)}) + f''(x^{(k)})(x - x^{(k)})$$

$$\frac{\partial q(x)}{\partial x} = 0 \rightarrow x^{(k+1)} = x^{(k)} - \frac{f'(x^{(k)})}{f''(x^{(k)})}$$



*Newton's method fits a parabola matching f's value, slope and curvature at $x^{(k)}$, then jumps to that parabola's minimum. When f really is quadratic, the parabola **is** f and the jump lands exactly on the answer.*

The multivariate model

Second-order Taylor expansion:

$$q(\boldsymbol{x}) = f(\boldsymbol{x}^{(k)}) + \left(\boldsymbol{g}^{(k)}\right)^T (\boldsymbol{x} - \boldsymbol{x}^{(k)}) + \tfrac{1}{2}(\boldsymbol{x} - \boldsymbol{x}^{(k)})^T \boldsymbol{H}^{(k)} (\boldsymbol{x} - \boldsymbol{x}^{(k)})$$

$$\nabla q(\boldsymbol{x}^{(k)}) = 0 \rightarrow \boldsymbol{g}^{(k)} + \boldsymbol{H}^{(k)}(\boldsymbol{x} - \boldsymbol{x}^{(k)}) = 0$$

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \left(\boldsymbol{H}^{(k)}\right)^{-1} \boldsymbol{g}^{(k)}$$

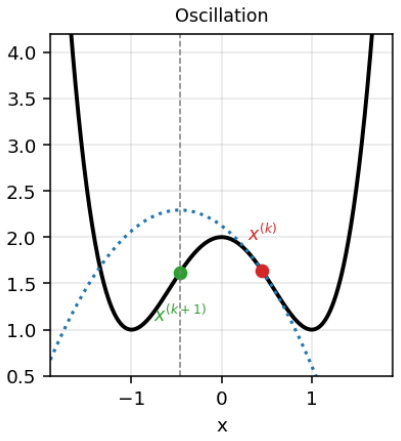
3.10.2 When Newton's method goes wrong

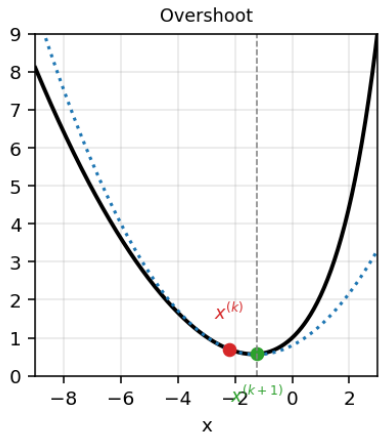
The division-by-curvature problem

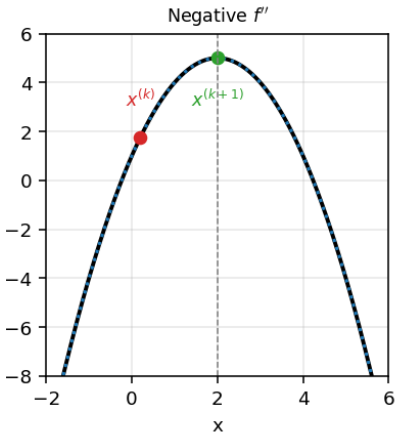
- Newton's method update rule includes the **second derivative in the denominator**.
- If $f''(x^{(k)}) = 0$, **the update becomes undefined** (the quadratic approximation forms a horizontal line).
- **Instability also arises when** $f''(x^{(k)}) \to 0$**.**
- In such cases, the next design point is **far away, where the quadratic approximation does not hold**.

Poor initialization may cause Newton's method to perform poorly

Failure mode	What happens
Oscillation	The iterate is thrown to a mirror-image point and bounces back and forth without converging
Overshoot	The function and the quadratic approximation are **largely different at the minimum of the parabola**, so the step lands somewhere useless
Negative f''	Instead of minimizing the parabola, we get a point that **maximizes** the parabola - we do not reach a local minimum







Fixes - how Newton is made robust in practice

- **Line search:** new design point $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha \boldsymbol{d}^{(k)}$.
- **Newton's method:** new design point $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \left(\boldsymbol{H}^{(k)}\right)^{-1} \boldsymbol{g}^{(k)}$ - i.e. Newton's method is the special case $\alpha = 1$ with descent direction $\boldsymbol{d}^{(k)} = -\left(\boldsymbol{H}^{(k)}\right)^{-1} \boldsymbol{g}^{(k)}$.
- **Valid scenario:** $\alpha = 1$ satisfies the Wolfe conditions **and** $\left(\boldsymbol{H}^{(k)}\right)^{-1}$ is positive definite.
- To improve robustness, a **step factor can be introduced** $(0 < \alpha < 1)$, with $\alpha = 1$ used when it satisfies the Wolfe conditions.

- If $\left(\boldsymbol{H}^{(k)}\right)^{-1}$ is **not positive definite**, some modification is required. A possible modification (**inflating**) is

$$\boldsymbol{H}^{(k)} \leftarrow \boldsymbol{H}^{(k)} + \tau \boldsymbol{I}$$

3.10.3 Convergence

Quadratic convergence

Convergence of Newton's method is **quadratic** - the difference between the minimum (x^*) and the iterate is **approximately squared with every iteration**:

$$\left|x^{(k+1)} - x^*\right| \approx c\left|x^{(k)} - x^*\right|^2, \quad c \text{ a constant}$$

This rate holds when (supposing we start within δ distance from the root x^*):

1. $f''(x) \neq 0$ for $x \in [x^* - \delta,\ x^* + \delta]$
2. $f'''(x)$ continuous on $[x^* - \delta,\ x^* + \delta]$
3. $\frac{1}{2}\left|\frac{f'''(x^{(1)})}{f''(x^{(1)})}\right| < c\left|\frac{f'''(x^*)}{f''(x^*)}\right|$ for constant $c < \infty$ - this condition avoids the overshoot

Stopping criterion: generally $\left|x^{(k+1)} - x^{(k)}\right| \leq$ Tolerance.

3.10.4 Worked examples

Example - a quadratic solved in one step

$f(\boldsymbol{x}) = (x_1 + 2x_2 - 1)^2 + (2x_1 + x_2 - 3)^2$ with $\boldsymbol{x}^{(1)} = [8,\ 8]$.

$$\nabla f(\boldsymbol{x}) = \boldsymbol{g} = \left[\frac{\partial f}{\partial x_1},\quad \frac{\partial f}{\partial x_2}\right]$$

$$\boldsymbol{g} = \Big[2(x_1 + 2x_2 - 1) + 2\cdot 2(2x_1 + x_2 - 3),\quad 2\cdot 2(x_1 + 2x_2 - 1) + 2(2x_1 + x_2 - 3)\Big]$$

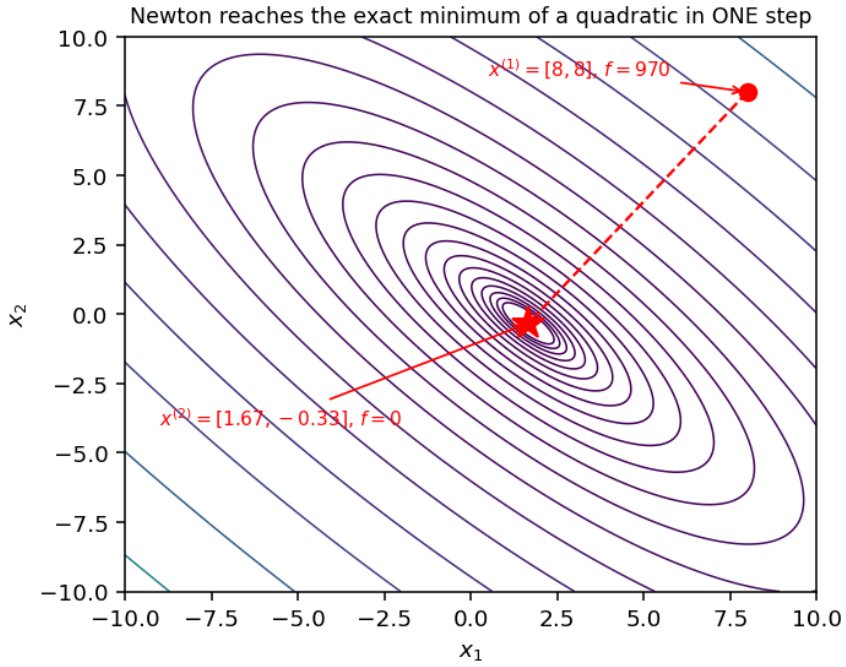
$$= \Big[10x_1 + 8x_2 - 14,\quad 8x_1 + 10x_2 - 10\Big]$$

$$\nabla^2 f(\boldsymbol{x}) = \boldsymbol{H} = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial}{\partial x_1}\left(\frac{\partial f}{\partial x_2}\right) \\ \frac{\partial}{\partial x_2}\left(\frac{\partial f}{\partial x_1}\right) & \frac{\partial^2 f}{\partial x_2^2} \end{bmatrix} = \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}$$

At $\boldsymbol{x}^{(1)} = [8, 8]$: $\boldsymbol{g}^{(1)} = [80 + 64 - 14,\ 64 + 80 - 10] = [130,\ 134]$.

$$\boldsymbol{x}^{(2)} = \boldsymbol{x}^{(1)} - \left(\boldsymbol{H}^{(1)}\right)^{-1}\boldsymbol{g}^{(1)} = [8,\ 8] - \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}^{-1}[130,\ 134] = [1.67,\ -0.33]$$

$$f(\boldsymbol{x}^{(1)}) = 970 \quad \text{and} \quad f(\boldsymbol{x}^{(2)}) = 0$$



*Because the objective is exactly quadratic, its second-order Taylor model is the function itself - so **one Newton step lands precisely on the minimum**, no matter how bad the starting guess. This is the best-case behaviour that motivates the method.*

Exercise 5 - Newton on a univariate function

$f(x) = x^2 + \cos(x)$ with $x^{(1)} = 1.0$, tolerance $= 10^{-4}$ (i.e. the algorithm stops when $|x^{(k+1)} - x^{(k)}| \leq 10^{-4}$).

$$f'(x) = 2x - \sin(x), \qquad f''(x) = 2 - \cos(x), \qquad x^{(k+1)} = x^{(k)} - \frac{f'(x^{(k)})}{f''(x^{(k)})}$$

Solution - watch the quadratic convergence

k	$x^{(k)}$	$\left\|x^{(k)} - x^{(k-1)}\right\|$
2	0.2063	0.7936
3	0.002854	0.2034
4	7.754×10^{-9}	2.854×10^{-3}
5	0.0	7.754×10^{-9}

The error goes $10^{-1} \to 10^{-3} \to 10^{-9}$ - **the number of correct digits roughly doubles each iteration**, exactly as quadratic convergence predicts.

Exercise 6

$f(\boldsymbol{x}) = (x_1 + 2x_2 - 7)^2 + (2x_1 + x_2 - 5)^2$ with $\boldsymbol{x}^{(1)} = [5,\ 4]$. Show that $\boldsymbol{x}^{(2)} = [1,\ 3]$.

Solution

$$\boldsymbol{g} = \Big[2(x_1 + 2x_2 - 7) + 2 \cdot 2(2x_1 + x_2 - 5), \quad 2 \cdot 2(x_1 + 2x_2 - 7) + 2(2x_1 + x_2 - 5)\Big]$$

$$= \begin{bmatrix} 10x_1 + 8x_2 - 34, & 8x_1 + 10x_2 - 38 \end{bmatrix}, \qquad \boldsymbol{H} = \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}$$

At $[5, 4]$: $\boldsymbol{g}^{(1)} = [50 + 32 - 34,\ 40 + 40 - 38] = [48,\ 42]$.

$$\boldsymbol{x}^{(2)} = [5,\ 4] - \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}^{-1} [48,\ 42] = [5, 4] - [4,\ 1] = \boxed{[1,\ 3]}$$

Using $\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \dfrac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$, here $\boldsymbol{H}^{-1} = \dfrac{1}{36} \begin{bmatrix} 10 & -8 \\ -8 & 10 \end{bmatrix}$.

3.11 The Secant Method and Quasi-Newton (DFP)

Approximating the second derivative

If $f''(x)$ is **not available**, approximate it using the first derivative:

$$f''(x^{(k)}) = \frac{f'(x^{(k)}) - f'(x^{(k-1)})}{x^{(k)} - x^{(k-1)}}$$

Newton's method:

$$x^{(k+1)} = x^{(k)} - \frac{f'(x^{(k)})}{f''(x^{(k)})}$$

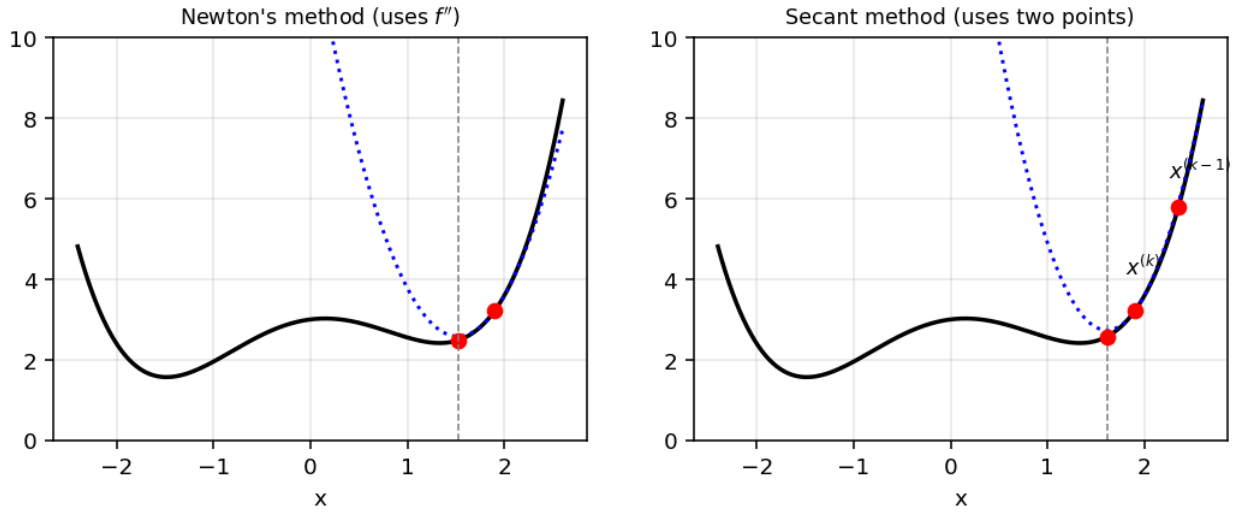
Secant method (substituting the approximation):

$$x^{(k+1)} = x^{(k)} - \frac{x^{(k)} - x^{(k-1)}}{f'(x^{(k)}) - f'(x^{(k-1)})} f'(x^{(k)})$$

equivalently

$$x^{(k+1)} = \frac{f'(x^{(k)})x^{(k-1)} - f'(x^{(k-1)})x^{(k)}}{f'(x^{(k)}) - f'(x^{(k-1)})}$$

The algorithm **requires two initial points** ($x^{(0)}$ and $x^{(1)}$).



*Newton uses the exact curvature at one point; the secant method uses the **"secant" between** $x^{(k)}$ **and** $x^{(k-1)}$ to estimate that curvature. Cheaper per iteration (no second derivative), at the cost of a slightly slower convergence rate.*

3.11.1 Quasi-Newton methods

Approximating the inverse Hessian

- Quasi-Newton methods **approximate the inverse of the Hessian**.
- These methods usually start by setting an **initial guess for the inverse of the Hessian matrix to the identity matrix**.
- They then **iteratively update this matrix** based on the information learned in each iteration.

Davidon–Fletcher–Powell (DFP) method

Let Q_k approximate the inverse Hessian, $s_k = x_{k+1} - x_k$, and $y_k = g_{k+1} - g_k$. The search direction is $d_k = -Q_k g_k$, and the DFP update is

$$Q_{k+1} = Q_k + \frac{s_k s_k^T}{s_k^T y_k} - \frac{Q_k y_k y_k^T Q_k}{y_k^T Q_k y_k}.$$

The update satisfies the secant equation $Q_{k+1} y_k = s_k$. If $Q_k \succ 0$ and $s_k^T y_k > 0$, the update preserves positive definiteness. The denominators must be nonzero; in practice, curvature tests and line search protect the update.

Learning curvature from successive iterates

DFP and BFGS are members of the **quasi-Newton family**. They estimate curvature from changes in iterates and gradients, reducing the need to evaluate and factorize the exact Hessian at every step.

3.12 Trust-Region Methods

Motivation

Optimizing high-dimensional functions often involves **breaking the problem into smaller, manageable tasks**:

- **Line search:** $\min_\alpha f(\boldsymbol{x} + \alpha \boldsymbol{d})$, then $\boldsymbol{x}^{(k+1)} \leftarrow \boldsymbol{x}^{(k)} + \alpha^{(k)} \boldsymbol{d}^{(k)}$ - pick the **direction first**, then the step size.
- **Newton's method:** approximate the function at a design point and minimize the approximated function.

The trust-region idea

- Trust-region methods approximate the objective function with a **simpler function (model function) within a given range**.
- This approximation is **effective within a neighborhood of the current point** $\boldsymbol{x}^{(k)}$.
- Trust-region methods first determine the **size** of the step, then the **direction** - the exact reverse of line search.

Solve a local model
Minimize $m_k(s)$ subject to $\|s\| \leq \Delta_k$.

↓

Compare actual and predicted decrease
Poor agreement: reject the step and shrink the radius, then solve again.

↓

Accept and continue
Good agreement: accept the step, possibly enlarge the radius, and repeat until convergence.

The constrained subproblem

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \boldsymbol{s}^{(k)}, \qquad \boldsymbol{s}^{(k)} = \underset{\boldsymbol{s}\in\mathbb{R}^d:\ \|\boldsymbol{s}\|\leq\Delta_k}{\arg\min} \ m_k(\boldsymbol{s})$$

Note this is itself a **constrained** optimization problem - the constraint $\|\boldsymbol{s}\| \leq \Delta_k$ *is* the trust region.

Choosing the model function $m_k(\cdot)$

1. Linear approximation

$$m_k(\boldsymbol{x}) = f(\boldsymbol{x}^{(k)}) + \left(\boldsymbol{x} - \boldsymbol{x}^{(k)}\right)^T \nabla f(\boldsymbol{x}^{(k)})$$

or

$$m_k(\boldsymbol{s}) = f(\boldsymbol{x}^{(k)}) + \boldsymbol{s}^T \nabla f(\boldsymbol{x}^{(k)})$$

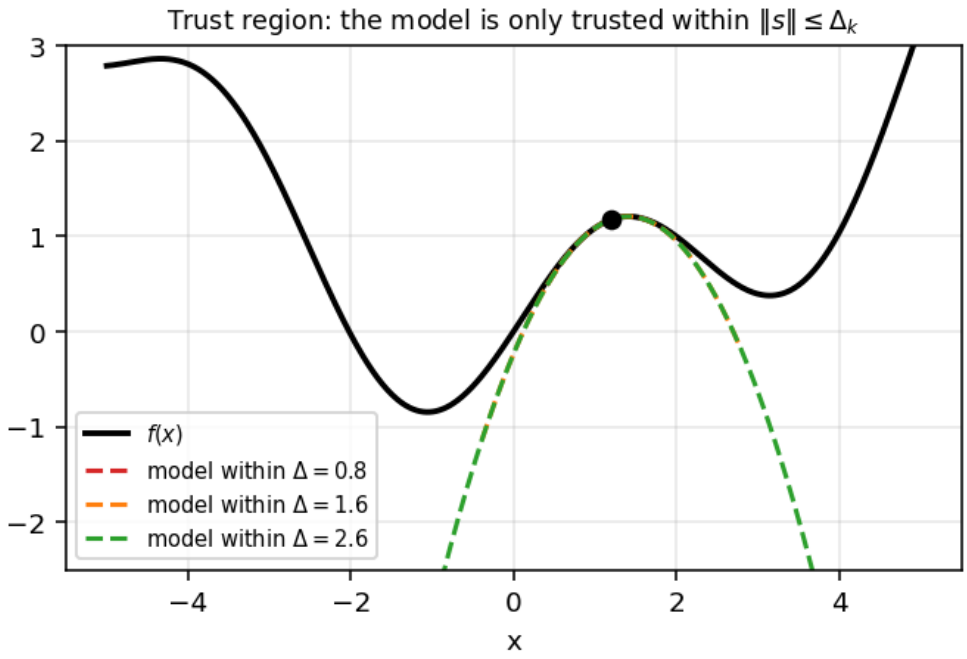
2. Quadratic approximation

$$m_k(\boldsymbol{x}) = f(\boldsymbol{x}^{(k)}) + (\boldsymbol{x} - \boldsymbol{x}^{(k)})^T \nabla f(\boldsymbol{x}^{(k)}) + \tfrac{1}{2}\left(\boldsymbol{x} - \boldsymbol{x}^{(k)}\right)^T \boldsymbol{H}^{(k)} (\boldsymbol{x} - \boldsymbol{x}^{(k)})$$

or

$$m_k(\boldsymbol{s}) = f(\boldsymbol{x}^{(k)}) + \boldsymbol{s}^T \nabla f(\boldsymbol{x}^{(k)}) + \tfrac{1}{2}\boldsymbol{s}^T \boldsymbol{H}^{(k)} \boldsymbol{s}$$

(It is also possible to choose a symmetric matrix for $\boldsymbol{H}$.)



The same quadratic model, trusted over three different radii. Close to $x^{(k)}$ the model tracks f well; far from it, the model and the function part company completely - which is exactly why the step must be capped at Δ_k.

3.12.1 Adjusting the trust-region radius

The ratio test

$$\eta = \frac{\text{actual improvement}}{\text{predicted improvement}} = \frac{f(\boldsymbol{x}^{(k)}) - f(\boldsymbol{x}^{(k+1)})}{f(\boldsymbol{x}^{(k)}) - m_k(\boldsymbol{x}^{(k+1)})}$$

- $|\eta - 1| \approx 0 \rightarrow$ the **predicted step size matches the actual step size** (the model is trustworthy).
- $\eta < \eta_1 \rightarrow$ **not desired**; the trust region radius is **decreased** by a factor $\gamma_1 < 1$.
- $\eta > \eta_2 \rightarrow$ **accurate prediction**; the trust region radius is **increased** by a factor $\gamma_2 > 1$.

3.12.2 Worked example - where Newton stalls and trust region does not

$f(\boldsymbol{x}) = x_1^2 - x_2^2$ with $\boldsymbol{x}^{(1)} = [0,\ 0]$

$$\nabla f(\boldsymbol{x}) = \boldsymbol{g} = \begin{bmatrix} 2x_1, & -2x_2 \end{bmatrix}$$

At $[0, 0]$: $\boldsymbol{g}^{(1)} = [0,\ 0]$.
Newton's method:

$$\boldsymbol{x}^{(2)} = \boldsymbol{x}^{(1)} - \left(\boldsymbol{H}^{(1)}\right)^{-1} \boldsymbol{g}^{(1)} = [0,\ 0] - \left(\boldsymbol{H}^{(1)}\right)^{-1} [0, 0] = [0,\ 0]$$

Newton's method stayed at $[0,\ 0]$ - the origin is a **saddle point** because its Hessian is $\text{diag}(2, -2)$, where the gradient vanishes, so Newton has nothing to move on.

The trust-region step at the same point

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \boldsymbol{s}^{(k)}, \qquad \boldsymbol{s}^{(k)} = \underset{\boldsymbol{s}\in\mathbb{R}^d:\ \|\boldsymbol{s}\|\leq\Delta_k}{\arg\min} \ m_k(\boldsymbol{s})$$

with the quadratic model

$$m_k(\boldsymbol{s}) = f(\boldsymbol{x}^{(k)}) + \boldsymbol{s}^T\nabla f(\boldsymbol{x}^{(k)}) + \tfrac{1}{2}\boldsymbol{s}^T\boldsymbol{H}^{(k)}\boldsymbol{s}$$

$$= f([0,0]) + [s_1,s_2]^T\nabla f([0,0]) + \tfrac{1}{2}[s_1,s_2]^T\begin{bmatrix}2 & 0\\ 0 & -2\end{bmatrix}[s_1,s_2]^T$$

$$= s_1^2 - s_2^2$$

Now the optimization problem is

$$\boldsymbol{s}^{(k)} = \underset{\boldsymbol{s}\in\mathbb{R}^2:\ s_1^2+s_2^2\leq\Delta_k}{\arg\min} \ \left(s_1^2 - s_2^2\right)$$

which has the obvious answer $s_1 = 0$, $s_2 = \pm\Delta_k$ - **the method escapes the saddle** by moving along the descent direction to the edge of its trust region. The constraint $\|\boldsymbol{s}\| \leq \Delta_k$ is what makes the otherwise-unbounded model problem well-posed.

3.13 Termination Conditions of Iterative Algorithms

Four common termination conditions

1. **Maximum iterations:** terminate when the number of iterations k exceeds a threshold k_{max}.
2. **Absolute improvement:** terminate if the change in function value between subsequent steps is smaller than a given threshold ε:
$$f(\boldsymbol{x}^{(k)}) - f(\boldsymbol{x}^{(k+1)}) < \varepsilon$$
3. **Relative improvement:** terminate if the change in function value **relative to the current function value** is smaller than a given threshold ε_r:
$$f(\boldsymbol{x}^{(k)}) - f(\boldsymbol{x}^{(k+1)}) < \varepsilon_r \left|f(\boldsymbol{x}^{(k)})\right|$$
4. **Gradient magnitude:** terminate if the magnitude of the gradient at the next step is smaller than a given threshold ε_g:
$$\left\|\nabla f(\boldsymbol{x}^{(k+1)})\right\| < \varepsilon_g$$

Random restarts

After meeting termination conditions, **it is better to do random restarts** by initiating the local descent method from randomly selected initial points. This is **particularly useful in cases where multiple local minima are likely to exist** - a non-convex problem gives no information about how far a local solution is from the global one.

3.14 Method Comparison

Choosing a method

Method	Derivatives needed	Works on	Key strength	Key weakness
Bracketing	None	Univariate, unimodal	Finds an interval from a single starting point; no derivatives	Fails on functions with no minimum (e^x, x^3, constants)
Fibonacci search	None	Univariate, unimodal, bracketed	**Optimal** reduction $1/F_{N+1}$ for a fixed query budget	Must know N in advance
Golden section	None	Univariate, unimodal, bracketed	Near-optimal, fixed $\rho \approx 0.382$, simple	Slightly worse than Fibonacci
Bisection	f'	Univariate, continuous f'	Guaranteed, $\log_2(\|b-a\|/\varepsilon)$ iterations	Needs a sign change in f'; linear convergence
Exact line search	Depends	Multivariate (step size)	Best possible α each iteration	Expensive - a full 1-D solve every iteration
Backtracking / Wolfe	∇f	Multivariate (step size)	Cheap, few evaluations, provably sufficient progress	Only a *satisfactory* α, not optimal
Newton's method	∇f, $\boldsymbol{H}$	Multivariate, twice differentiable	**Quadratic** convergence; exact for quadratics in one step	Needs and inverts the Hessian; fails at $f'' \to 0$, saddles, and with poor initialization
Secant	∇f only	Univariate	No second derivative required	Needs two starting points; slower than Newton
Quasi-Newton (DFP)	∇f only	Multivariate	Newton-like behaviour without forming $\boldsymbol{H}$	Approximation quality builds up over iterations
Trust region	∇f (+ $\boldsymbol{H}$)	Multivariate	Robust where Newton fails - **escapes saddle points**	Solves a constrained subproblem each iteration

3.15 Key Takeaways

Summary

One-D methods **trap** a minimum in a shrinking interval - optimally with **Fibonacci** ($1/F_{N+1}$), practically with **golden section** ($\approx 0.618^N$), reliably with **bisection** on f'. Multivariate methods **step**: $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha^{(k)}\boldsymbol{d}^{(k)}$, where the **Wolfe conditions** certify α (first rejects too-large steps, second rejects too-small ones) and **Newton** picks $\boldsymbol{d} = -\boldsymbol{H}^{-1}\boldsymbol{g}$ for quadratic convergence - with **trust regions** as the fix for everywhere Newton breaks.

Self-check

1. Formulate the cardboard-box problem. Why are there **five** constraints and not three?
2. What three conditions must hold for $a < b < c$ to bracket a minimum?
3. Trace Algorithm 1 on $f(x) = x^2 - 4x + 4$ with $x = 0$, $s = 0.1$, $k = 2$: give the initial interval and the interval after two iterations.
4. Give three functions on which the bracketing algorithm fails, and say why in each case.
5. How much of the interval can two queries remove, irrespective of f? Three queries?
6. Derive the total reduction factor $1/F_{N+1}$ from the individual ρ_k.
7. Why do the two interior points coincide at the last Fibonacci iteration, and what is the fix?
8. Initial range $[-2, 4]$, $\epsilon = 0.1$, required final range 0.2. Find N.
9. What is ρ in golden section search, and how is it related to the Fibonacci ρ_k?
10. Why is bisection applied to f' rather than f? What must be true of $f'(a)$ and $f'(b)$?
11. State the general descent update rule and name both of its ingredients.
12. For $f = (x_1x_2)^2 + x_3^2$ with $\boldsymbol{x} = [1, 2, 1]$, $\boldsymbol{d} = [0, -2, -1]$, find α^* and the next design point.
13. Write both Wolfe conditions. Which rejects too-large steps, and which too-small?
14. In the backtracking example, why does $\alpha = 2.5$ fail but $\alpha = 1.25$ succeed?
15. Can the curvature ratio be negative? Explain using the signs of numerator and denominator.
16. Derive $x^{(k+1)} = x^{(k)} - f'(x^{(k)})/f''(x^{(k)})$ from a second-order Taylor expansion.
17. Name Newton's three failure modes and the cause of each.
18. Show that one Newton step solves $(x_1 + 2x_2 - 7)^2 + (2x_1 + x_2 - 5)^2$ from $[5, 4]$.
19. How does the secant method avoid needing f''?
20. Why does Newton's method stall on $f = x_1^2 - x_2^2$ at the origin, and how does a trust region escape?
21. Define η in a trust-region method and state what happens when it is small or large.
22. List the four termination conditions, and say when random restarts are worthwhile.

CHAPTER 4

Unconstrained Optimization - First and Second Order Methods

Overview

An iterative descent method updates $x_{k+1} = x_k + \alpha_k d_k$. Choosing the direction d_k determines which derivative information the method uses:

- **First-order methods** use only the gradient: gradient descent, conjugate gradient, momentum and Nesterov momentum, and the adaptive family - **Adagrad, RMSProp, Adadelta, Adam** - plus hypergradient descent.
- **Second-order methods** use curvature: Newton's method, the classical modified Newton method, and the **quasi-Newton** family built from rank-one and rank-two updates (**DFP, BFGS, L-BFGS**).

Core idea: every method in this chapter is $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \alpha^{(k)} \boldsymbol{S}^{(k)} \boldsymbol{g}^{(k)}$ - they differ only in what matrix $\boldsymbol{S}^{(k)}$ they put in front of the gradient. Gradient descent uses $\boldsymbol{I}$; Newton uses $\boldsymbol{H}^{-1}$; everything else is an attempt to get the benefit of $\boldsymbol{H}^{-1}$ without paying for it.

Further reading

Kochenderfer, Mykel J., and Tim A. Wheeler. *Algorithms for Optimization*. MIT Press, 2019.

4.1 Choosing Directions and Step Sizes

Step-size selection and direction selection

A descent iteration has two separate decisions. A direction rule uses gradient or curvature information to select d_k. A step-size rule chooses α_k, using a fixed schedule, line search, or a trust-region model. Improving either decision can improve convergence, but the two rules must remain compatible.

Descent-direction test

For a smooth objective, a direction d is a strict descent direction at x when $\nabla f(x)^T d < 0$. This guarantees improvement for sufficiently small positive steps; it does not guarantee improvement for an arbitrarily large fixed step.

4.2 Gradient Descent

The method

1. Descent direction $\boldsymbol{d} \rightarrow$ **direction of steepest descent** (direction **opposite the gradient**).
2. Notation: the gradient of the function f at $\boldsymbol{x} = \boldsymbol{x}^{(k)}$ is denoted by $\boldsymbol{g}^{(k)} = \nabla f\left(\boldsymbol{x}^{(k)}\right)$.
3. Direction of steepest descent (typically a **normalized** direction is used):

$$\boldsymbol{d}^{(k)} = -\frac{\boldsymbol{g}^{(k)}}{\left\|\boldsymbol{g}^{(k)}\right\|}$$

4. New design point:

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha \boldsymbol{d}^{(k)}$$

Three conditions for guaranteed improvement

Following the direction of steepest descent guarantees improvement, **however under certain conditions**:

- The objective function should be **smooth**. *(If the function is not smooth, it may have discontinuities or sharp changes, making it difficult to compute the gradient.)*
- The **step size must be adequately small**.
- We are **not currently at a point where the gradient is zero** (not a stationary point).

4.2.1 Computing the steepest-descent direction

Example

For $f(\boldsymbol{x}) = x_1^2 x_2 + 1$, suppose $\boldsymbol{x}^{(k)} = [2,\ 4]$. What is the direction of steepest descent?

$$\boldsymbol{g}^{(k)} = \left[2x_1x_2,\ x_1^2\right] = [16,\ 4]$$

$$\boldsymbol{d}^{(k)} = -\frac{\boldsymbol{g}^{(k)}}{\left\|\boldsymbol{g}^{(k)}\right\|} = -\frac{[16,\ 4]}{\|[16,\ 4]\|} = \left[\frac{-16}{\sqrt{272}},\ \frac{-4}{\sqrt{272}}\right]$$

since $\|[16, 4]\| = \sqrt{16^2 + 4^2} = \sqrt{256 + 16} = \sqrt{272}$.

Task

For $f(\boldsymbol{x}) = x_1 x_2^2 + 1$, suppose $\boldsymbol{x}^{(k)} = [-2,\ -2]$. What is the direction of steepest descent?

Solution

$$\boldsymbol{g}^{(k)} = \left[x_2^2,\ 2x_1x_2\right] = \left[(-2)^2,\ 2(-2)(-2)\right] = [4,\ 8]$$

$$\left\|\boldsymbol{g}^{(k)}\right\| = \sqrt{16 + 64} = \sqrt{80}$$

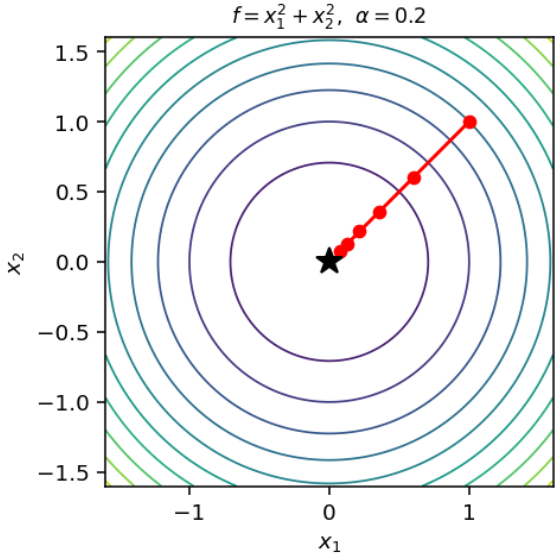
$$\boldsymbol{d}^{(k)} = -\frac{[4,\ 8]}{\sqrt{80}} = \left[\frac{-4}{\sqrt{80}},\ \frac{-8}{\sqrt{80}}\right] \approx [-0.447,\ -0.894]$$

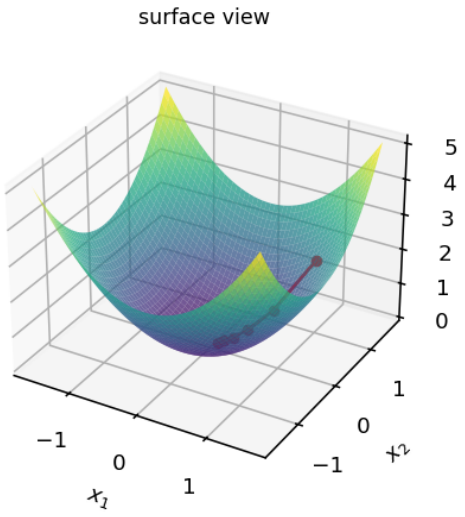
4.2.2 A full run

$f(\boldsymbol{x}) = x_1^2 + x_2^2$ with $\alpha = 0.2$

- $\boldsymbol{x}^{(0)} = [1,\ 1]$ and $f(\boldsymbol{x}^{(0)}) = 1^2 + 1^2 = 2$
- $\boldsymbol{g}^{(0)} = [2x_1,\ 2x_2] = [2,\ 2]$
- $\boldsymbol{x}^{(1)} = \boldsymbol{x}^{(0)} + \alpha \boldsymbol{d}^{(0)} = \boldsymbol{x}^{(0)} - \alpha \boldsymbol{g}^{(0)}$ - **the direction is used without normalization here**
- $\boldsymbol{x}^{(1)} = [1,\ 1] - 0.2[2,\ 2] = [0.6,\ 0.6]$
- $f(\boldsymbol{x}^{(1)}) = 0.6^2 + 0.6^2 = 0.72$

Iteration	$\boldsymbol{x}^{(k)}$	$f(\boldsymbol{x}^{(k)})$	$\boldsymbol{x}^{(k+1)}$	$f(\boldsymbol{x}^{(k+1)})$
1	[1.000, 1.000]	2.0000	[0.600, 0.600]	0.7200
2	[0.600, 0.600]	0.7200	[0.360, 0.360]	0.2592
3	[0.360, 0.360]	0.2592	[0.216, 0.216]	0.0933
4	[0.216, 0.216]	0.0933	[0.130, 0.130]	0.0336
5	[0.130, 0.130]	0.0336	[0.077, 0.077]	0.0121





*Each step multiplies both coordinates by $(1 - 2\alpha) = 0.6$, so the path is a straight line into the minimum with geometrically shrinking steps. On a perfectly circular bowl, steepest descent points **straight at** the minimum - this is the easiest possible case.*

Normalized or not?

Two common conventions are the normalized direction $d = -g/\|g\|$ and the unnormalized direction $d = -g$. With these conventions, α means "distance moved" (normalized) or "gradient multiplier" (unnormalized). Check which convention a question uses before substituting.

4.2.3 Exercise: termination conditions

Match the method to the termination behaviour

Compare **Function A:** $f(x) = x^2$ against **Function B:** $f(x) = 0.1x^2$ under four different termination conditions. Parameters: $x^{(0)} = 2$, $\alpha = 0.2$, $\varepsilon = 0.1$, $\varepsilon_r = 0.1$, $k_{\max} = 10$, $\varepsilon_g = 0.1$.

No	Function	Termination condition
1	$f(x) = x^2$	Number of iterations k exceeds threshold $k_{\max} = 10$
2	$f(x) = 0.1x^2$	Number of iterations k exceeds threshold $k_{\max} = 10$
3	$f(x) = x^2$	$\left\|f(x^{(k)}) - f(x^{(k+1)})\right\| < \varepsilon$
4	$f(x) = 0.1x^2$	$\left\|f(x^{(k)}) - f(x^{(k+1)})\right\| < \varepsilon$
5	$f(x) = x^2$	$f(\boldsymbol{x}^{(k)}) - f(x^{(k+1)}) < \varepsilon_r \left\|f(x^{(k)})\right\|$
6	$f(x) = 0.1x^2$	$f(\boldsymbol{x}^{(k)}) - f(x^{(k+1)}) < \varepsilon_r \left\|f(x^{(k)})\right\|$
7	$f(x) = x^2$	$\left\|\nabla f(x^{(k+1)})\right\| < \varepsilon_g$
8	$f(x) = 0.1x^2$	$\left\|\nabla f(x^{(k+1)})\right\| < \varepsilon_g$

Sensitivity of stopping criteria to scale

The two functions have the **same minimizer** ($x = 0$) but very different **scales**. With $\alpha = 0.2$:

- On $f = x^2$: $x^{(k+1)} = x^{(k)}(1 - 0.4) = 0.6\,x^{(k)}$ - fast decay, the design points sprint to the origin.
- On $f = 0.1x^2$: $x^{(k+1)} = x^{(k)}(1 - 0.04) = 0.96\,x^{(k)}$ - the design points barely crawl.

Scaling the objective changes the magnitude of its gradient: on $0.1x^2$ the design points bunch up near the starting point, while on x^2 they have already reached the bottom. **The same termination condition therefore fires at a completely different place on the two functions** - an absolute threshold ε is generous on a small-valued function and strict on a large-valued one, which is precisely why the **relative** condition (ε_r) and the **gradient magnitude** condition exist.

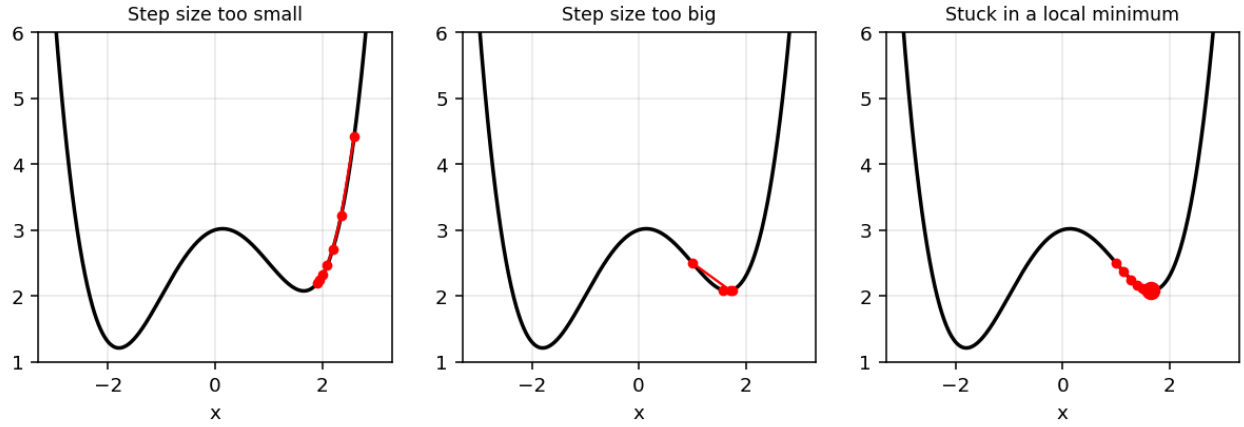
4.3 The Step Size (Learning Rate) α

Recap

$$\boldsymbol{d}^{(k)} = -\frac{\boldsymbol{g}^{(k)}}{\left\|\boldsymbol{g}^{(k)}\right\|}, \qquad \boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha \boldsymbol{d}^{(k)}$$

Three ways a fixed α goes wrong

Pathology	What happens
Step size too small	Tiny moves; enormous iteration counts to make progress
Step size too big	The iterate leaps across the valley and can diverge entirely
Stuck in local minimum	The method settles in a shallow local basin and never sees the global one



What is the best step size?

Recall line search:

$$\alpha = \arg\min_{\alpha} f(\boldsymbol{x} + \alpha \boldsymbol{d})$$

This is a **one-dimensional optimization problem**. Scalar search algorithms can solve this subproblem.

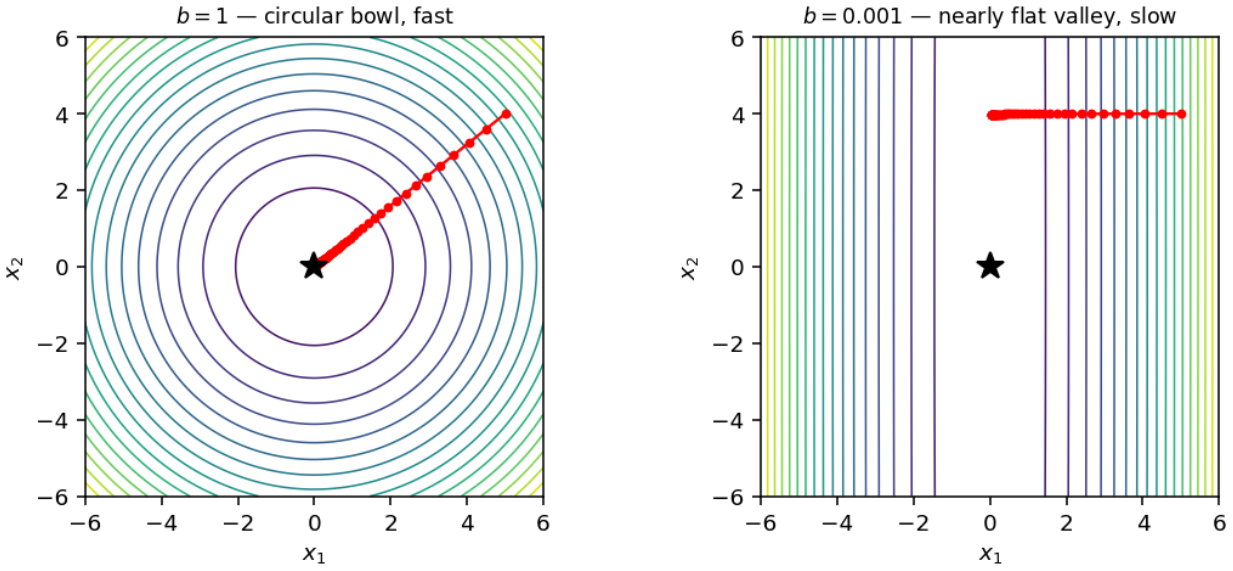
4.3.1 The effect of conditioning

$f(\boldsymbol{x}) = \frac{1}{2}\left(x_1^2 + bx_2^2\right)$ - a quadratic test problem

Varying b changes how *round* the bowl is, and this single parameter drives everything in the rest of the chapter.

Setting	Convergence behaviour
$b = 1$, $\boldsymbol{x}^{(0)} = [5, 5]$, $\alpha = 0.1$	Circular bowl - steady, direct descent to the minimum
$b = 1$, $\boldsymbol{x}^{(0)} = [5, 5]$, $\alpha = 0.5$	Much faster: reaches $\approx [4.9 \times 10^{-5},\ 4.9 \times 10^{-5}]$ in the same number of steps
$b = 0.001$, $\boldsymbol{x}^{(0)} = [5, 5]$, $\alpha = 0.1$	x_1 collapses quickly but x_2 **hardly moves** ($5 \to 4.9995 \to 4.999\ldots$)
$b = 0.001$, $\alpha = 0.5$, **2000 iterations**	$\boldsymbol{x}^{(0)} = [5, 5]$, $\boldsymbol{x}^{(500)} = [0,\ 3.89]$, $\boldsymbol{x}^{(1000)} = [0,\ 3.032]$, $\boldsymbol{x}^{(1500)} = [0,\ 2.36]$, $\boldsymbol{x}^{(2000)} = [0,\ 1.83]$
$b = 0.001$, $\boldsymbol{x}^{(0)} = [5, 1]$, $\alpha = 1.5$, 20 iterations	Step size too large in x_1 - the iterate **oscillates with growing amplitude**

Even after **2000 iterations** with an aggressive $\alpha = 0.5$, x_2 has only travelled from 5 to 1.83. This is the central failure of plain gradient descent, and the motivation for everything that follows.



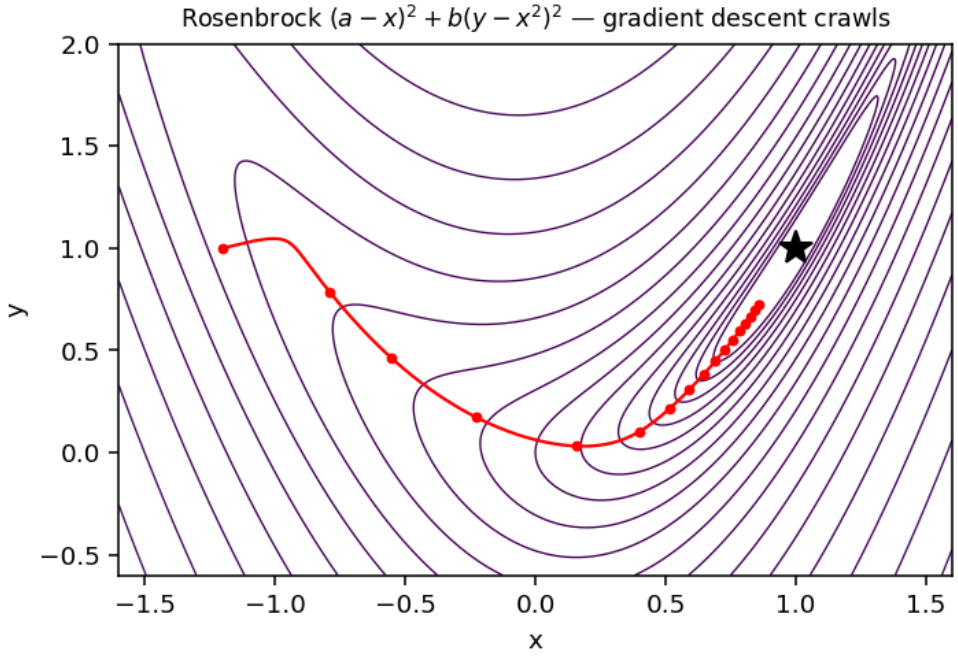
Left: with $b = 1$ the contours are circles and the gradient points at the minimum. Right: with $b = 0.001$ the contours are enormously elongated - the gradient is almost entirely x_1, so the method fixes x_1 almost immediately and then inches along the flat x_2 floor.

4.3.2 Narrow valleys

The Rosenbrock function

$$f(x, y) = (a - x)^2 + b(y - x^2)^2$$

When navigating through narrow valleys, gradient descent may require many iterations to make meaningful progress. The gradient points across the valley rather than along it, so the iterate zig-zags between the walls while creeping forward.



Four hundred gradient-descent iterations on Rosenbrock: the method finds the valley floor in a handful of steps and then spends the rest of its budget crawling along it toward the minimum at $(1, 1)$.

4.4 Conjugate Gradient

How to address the narrow-valley problem

Conjugate gradient modifies the descent direction.

1. It **starts with the direction of steepest descent** $\boldsymbol{d}^{(1)} = -\boldsymbol{g}^{(1)}$.
2. **Line search is used to find the next design point:**

$$\alpha^{(1)} = \arg\min_{\alpha} f\left(\boldsymbol{x} + \alpha \boldsymbol{d}^{(1)}\right), \qquad \boldsymbol{x}^{(2)} = \boldsymbol{x}^{(1)} + \alpha^{(1)} \boldsymbol{d}^{(1)}$$

The quadratic model

Suppose the quadratic function approximation

$$f(\boldsymbol{x}) = \tfrac{1}{2} \boldsymbol{x}^T \boldsymbol{A} \boldsymbol{x} + \boldsymbol{b}^T \boldsymbol{x} + c$$

with **symmetric and positive definite matrix** $\boldsymbol{A}$ - which implies $f(\boldsymbol{x})$ has a **unique local minimum**.

4.4.1 Deriving the exact step size

Step size for the quadratic model

$$\alpha = \arg\min_{\alpha} f(\boldsymbol{x} + \alpha \boldsymbol{d}) = \arg\min_{\alpha} \left(\tfrac{1}{2} (\boldsymbol{x} + \alpha \boldsymbol{d})^T \boldsymbol{A} (\boldsymbol{x} + \alpha \boldsymbol{d}) + \boldsymbol{b}^T (\boldsymbol{x} + \alpha \boldsymbol{d}) + c \right)$$

Taking the derivative with respect to α yields

$$\frac{\partial f(\boldsymbol{x} + \alpha \boldsymbol{d})}{\partial \alpha} = \frac{\partial}{\partial \alpha} \left(\tfrac{1}{2} (\boldsymbol{x} + \alpha \boldsymbol{d})^T \boldsymbol{A} (\boldsymbol{x} + \alpha \boldsymbol{d}) + \boldsymbol{b}^T (\boldsymbol{x} + \alpha \boldsymbol{d}) + c \right) = \boldsymbol{d}^T \boldsymbol{A} (\boldsymbol{x} + \alpha \boldsymbol{d}) + \boldsymbol{d}^T \boldsymbol{b}$$

Finally, the step size can be evaluated as

$$\frac{\partial f(\boldsymbol{x} + \alpha \boldsymbol{d})}{\partial \alpha} = 0 \implies \boxed{\alpha = -\frac{\boldsymbol{d}^T (\boldsymbol{A} \boldsymbol{x} + \boldsymbol{b})}{\boldsymbol{d}^T \boldsymbol{A} \boldsymbol{d}}}$$

4.4.2 Deriving β - the conjugacy condition

Direction update

Direction update (gradient and previous direction):

$$\boldsymbol{d}^{(k+1)} = -\boldsymbol{g}^{(k+1)} + \beta^{(k)} \boldsymbol{d}^{(k)} \quad \text{with scalar parameter } \beta$$

Larger $\beta \rightarrow$ **previous direction contribution is higher.**
In conjugate gradient, directions are **mutually conjugate with respect to** $\boldsymbol{A}$, i.e.

$$\boldsymbol{d}^{(i)T} \boldsymbol{A} \boldsymbol{d}^{(j)} = 0 \quad \text{for } i \neq j$$

Now β can be estimated as:

$$\boldsymbol{d}^{(k+1)T}\boldsymbol{A}\boldsymbol{d}^{(k)} = 0$$

$$\left(-\boldsymbol{g}^{(k+1)} + \beta^{(k)}\boldsymbol{d}^{(k)}\right)^T \boldsymbol{A}\boldsymbol{d}^{(k)} = 0$$

$$-\boldsymbol{g}^{(k+1)T}\boldsymbol{A}\boldsymbol{d}^{(k)} + \beta^{(k)}\boldsymbol{d}^{(k)T}\boldsymbol{A}\boldsymbol{d}^{(k)} = 0$$

$$\boxed{\beta^{(k)} = \frac{\boldsymbol{g}^{(k+1)T}\boldsymbol{A}\boldsymbol{d}^{(k)}}{\boldsymbol{d}^{(k)T}\boldsymbol{A}\boldsymbol{d}^{(k)}}}$$

4.4.3 Practical choices for β

The matrix A is unknown

The matrix $\boldsymbol{A}$, that approximates f around $\boldsymbol{x}^{(k)}$, **is unknown**. As a workaround, various selections for $\beta^{(k)}$ provide satisfactory results - both of which use **only gradients**, never $\boldsymbol{A}$.

Fletcher–Reeves

$$\beta^{(k)} = \frac{\boldsymbol{g}^{(k)T}\boldsymbol{g}^{(k)}}{\boldsymbol{g}^{(k-1)T}\boldsymbol{g}^{(k-1)}}$$

Polak–Ribière

$$\beta^{(k)} = \frac{\boldsymbol{g}^{(k)T}\left(\boldsymbol{g}^{(k)} - \boldsymbol{g}^{(k-1)}\right)}{\boldsymbol{g}^{(k-1)T}\boldsymbol{g}^{(k-1)}}$$

with the safeguard

$$\beta \leftarrow \max(\beta,\ 0)$$

References

Fletcher, Reeves, and Colin M. Reeves. “Function minimization by conjugate gradients.” *The Computer Journal* 7.2 (1964): 149–154. Polak, Elijah, and Gerard Ribiere. “Note sur la convergence de methodes de directions conjuguées.” *Revue française d’informatique et de recherche opérationnelle*, Série rouge 3.16 (1969): 35–43.

Why conjugacy is powerful

For an n-dimensional quadratic, conjugate gradient reaches the exact minimum in **at most n steps** - compared with the unbounded zig-zagging of plain gradient descent. The $\max(\beta, 0)$ reset in Polak–Ribière is what keeps it stable when f is not actually quadratic.

4.5 Momentum

Motivation

For a nearly flat surface, gradient descent may perform a large number of iterations to reach the minimum - exactly the $b = 0.001$ behaviour above.

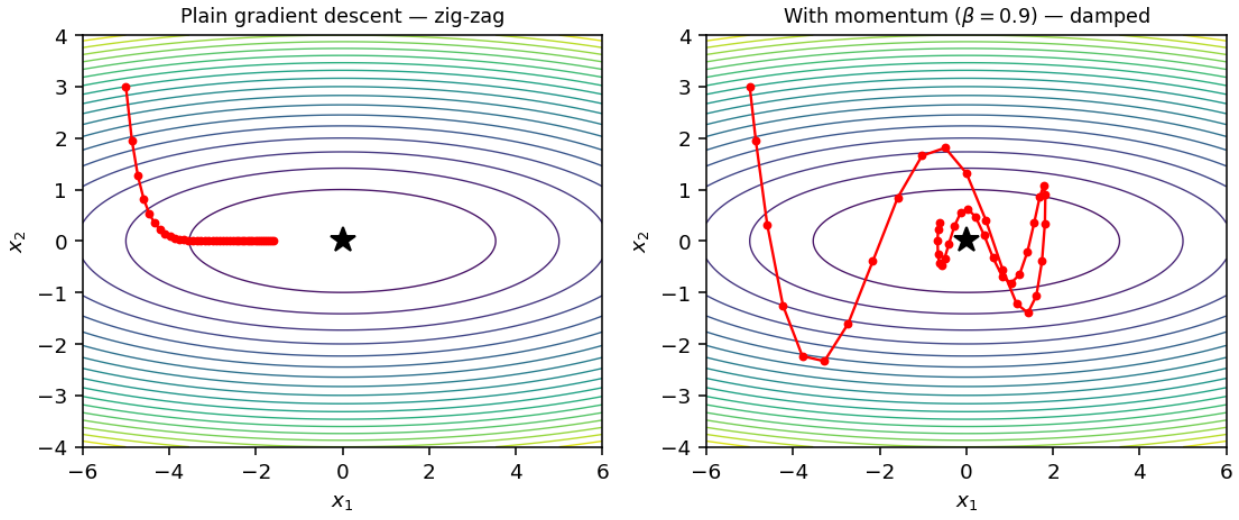
Momentum

1. $\boldsymbol{v}^{(k+1)} = \beta \boldsymbol{v}^{(k)} - \alpha \boldsymbol{g}^{(k)}$
2. $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \boldsymbol{v}^{(k+1)} = \boldsymbol{x}^{(k)} + \left(\beta \boldsymbol{v}^{(k)} - \alpha \boldsymbol{g}^{(k)}\right) = \boldsymbol{x}^{(k)} - \alpha \boldsymbol{g}^{(k)} + \beta \boldsymbol{v}^{(k)}$
3. **When** $\beta = 0 \rightarrow$ **gradient descent.**

The red term is the entire modification: a memory of where you were already going.

Momentum as accumulated velocity

A rolling ball provides an analogy: accumulated velocity carries motion through small changes in slope, while friction gradually dissipates it. Velocity **accumulates** in directions of persistent descent and **cancels** in directions that keep reversing - which is exactly why momentum damps the zig-zag across a narrow valley while accelerating along it.



4.5.1 Nesterov momentum

Overshoot due to high momentum

Too much accumulated velocity carries the iterate straight past the minimum and up the other side.

Nesterov momentum

Modify the momentum algorithm to consider the gradient at the anticipated (projected) future position:

1. $\boldsymbol{v}^{(k+1)} = \beta \boldsymbol{v}^{(k)} - \alpha \nabla f\left(\boldsymbol{x}^{(k)} + \beta \boldsymbol{v}^{(k)}\right)$
2. $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \boldsymbol{v}^{(k+1)}$

The gradient is evaluated at $\boldsymbol{x}^{(k)} + \beta \boldsymbol{v}^{(k)}$ - **where momentum is about to take us** - rather than where we currently are, so the method can "see" the far wall coming and brake early.
(Nesterov, Yurii. "A method of solving a convex programming problem with convergence rate $O(1/k^2)$." Doklady Akademii Nauk SSSR 269.3 (1983): 543.)

4.6 Adaptive Step Sizes

Is a fixed step size the way to go?

No - the same three pathologies (too small, too big, stuck) recur. The adaptive family gives **each variable its own step size**, derived from the history of its own gradients.

4.6.1 Adagrad - Adaptive sub-gradient method

Update rule

Suppose $\boldsymbol{x} = [x_1, x_2, \ldots, x_i, \ldots, x_n]$. The learning rate / step size for the i-th variable (feature):

$$x_i^{(k+1)} = x_i^{(k)} - \frac{\alpha}{\epsilon + \sqrt{s_i^{(k)}}} g_i^{(k)}, \qquad \text{where } s_i^{(k)} = \sum_{j=1}^{k} \left(g_i^{(j)}\right)^2$$

How it behaves

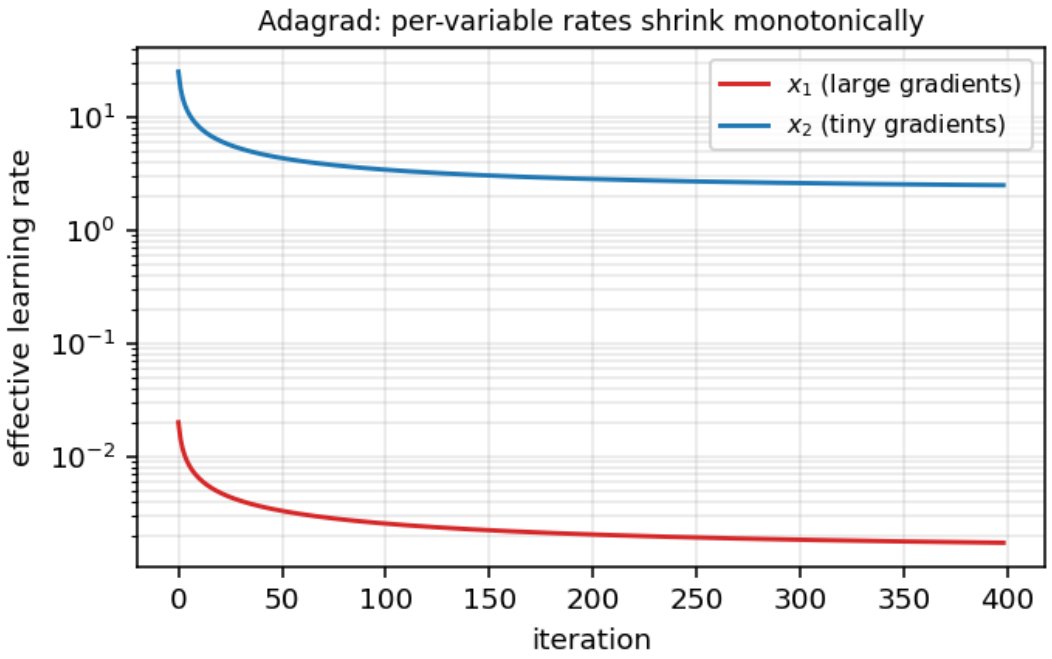
Effective learning rate:

$$\frac{\alpha}{\epsilon + \sqrt{\sum_{j=1}^{k} \left(g_i^{(j)}\right)^2}}$$

- The **impact of α on the effective learning rate is less** (typically set $\alpha = 0.01$).
- ϵ - a small positive number (e.g. 1×10^{-8}).
- **Scale the learning rate inversely proportional to the accumulated sum of squared gradients for each parameter.**
- **Prevent variables with consistently higher gradients from dominating the update process.**

Weakness

Learning rates become infinitesimally small before the optimization process converges. Because s_i is a running sum of squares it can only ever grow, so the denominator marches monotonically upward and the method grinds to a halt whether or not it has arrived.
(Duchi, John, Elad Hazan, and Yoram Singer. "Adaptive subgradient methods for online learning and stochastic optimization." Journal of Machine Learning Research 12.7, 2011.)



*Adagrad's per-variable effective rates on the $b = 0.001$ problem. The x_1 rate (large gradients) collapses fastest, but **both** decay without bound - the flaw the next two methods exist to fix.*

4.6.2 RMSProp

Update rule

Addresses the "learning rates become infinitesimally small before the optimization process converges" issue of Adagrad. Average gradient update:

$$s_i^{(k+1)} = \gamma \underbrace{s_i^{(k)}}_{\text{Moving average}} + (1-\gamma) \underbrace{\left(\boldsymbol{g}^{(k)} \odot \boldsymbol{g}^{(k)}\right)}_{\text{Current squared gradient}}$$

where $\odot$ denotes **element-wise multiplication** and $\gamma \in [0,\ 1]$.
Learning rate / step size for the i-th variable:

$$x_i^{(k+1)} = x_i^{(k)} - \frac{\alpha}{\epsilon + \sqrt{s_i^{(k)}}} g_i^{(k)} = x_i^{(k)} - \frac{\alpha}{\epsilon + \mathrm{RMS}(g_i)} g_i^{(k)}$$

Here **"RMS(x) refers to the decaying root mean square of the time series of x"**.

The one-word difference from Adagrad

Adagrad **sums**; RMSProp **averages with decay**. Replacing $\sum$ with an exponentially weighted moving average means old gradients are forgotten, so the denominator can go **down** as well as up - and the method keeps moving.

4.6.3 Adadelta

Update rule

Addresses the same Adagrad issue. Learning rate / step size for the i-th variable:

$$x_i^{(k+1)} = x_i^{(k)} - \frac{\text{RMS}(\Delta x_i)}{\epsilon + \text{RMS}(g_i)} g_i^{(k)}$$

where

$$\text{RMS}(\Delta x_i) = \sqrt{\left(E\left((\Delta x_i)^2_{(k)}\right) + \epsilon\right)}, \qquad (\Delta x_i)^2_{(k)} = \gamma E(\Delta x_i)^2_{(k-1)} + (1-\gamma)(\Delta x_i)^2_{(k)}$$

There is no α.

What makes Adadelta unusual

The numerator is a decaying RMS of **past parameter changes**, so the update is dimensionally consistent and the method needs **no learning-rate hyperparameter at all**. The resulting updates can be conservative when the accumulated change estimate is small.
(Zeiler, Matthew D. "Adadelta: an adaptive learning rate method." arXiv preprint arXiv:1212.5701, 2012.)

4.6.4 Adam - Adaptive moment estimation

What Adam is

- **Combines the advantages of both AdaGrad and RMSProp.**
- **Incorporates momentum into its updates.**
- **Individual adaptive learning rates for each feature.**

(Kingma, Diederik P., and Jimmy Ba. "Adam: A method for stochastic optimization." arXiv, 2014.)

The five steps

1. Biased decaying momentum (weighted moving average of the past gradients):

$$\boldsymbol{v}^{(k+1)} = \gamma_v \boldsymbol{v}^{(k)} + (1-\gamma_v)\boldsymbol{g}^{(k)}$$

2. Biased decaying squared gradient:

$$\boldsymbol{s}^{(k+1)} = \gamma_s \boldsymbol{s}^{(k)} + (1-\gamma_s)\left(\boldsymbol{g}^{(k)} \odot \boldsymbol{g}^{(k)}\right)$$

Weighted moving average of the squared past gradients. It tracks the magnitude (or uncentered variance) of recent gradients. **3. Corrected decaying momentum:**

$$\hat{\boldsymbol{v}}^{(k+1)} = \frac{\boldsymbol{v}^{(k+1)}}{1-\gamma_v^k}$$

Because the momentum is initially set to zero, the estimates are artificially close to zero during the first few training iterations. This step applies a mathematical correction to "un-bias" the momentum estimate, making it accurate even at the very start of training. **4. Corrected decaying squared gradient:**

$$\hat{\boldsymbol{s}}^{(k+1)} = \frac{\boldsymbol{s}^{(k+1)}}{1-\gamma_s^k}$$

This ensures the algorithm doesn't make overly large parameter updates in the beginning due to an artificially small denominator. **5. Next iterate:**

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \frac{\alpha \hat{\boldsymbol{v}}^{(k+1)}}{\varepsilon + \sqrt{\hat{\boldsymbol{s}}^{(k+1)}}}$$

A small constant ε is added to the denominator purely to prevent a "division by zero" error if the gradients are zero.

4.6.5 Hypergradient descent

Optimizing the learning rate itself

Hypergradient descent applies gradient descent to the learning rate of a descent method:

$$\alpha^{(k+1)} = \alpha^{(k)} - \mu \frac{\partial f(\boldsymbol{x}^{(k)})}{\partial \alpha} = \alpha^{(k)} + \mu \left(\boldsymbol{g}^{(k)}\right)^T \boldsymbol{g}^{(k-1)}$$

Hypergradient learning-rate adaptation

$\left(\boldsymbol{g}^{(k)}\right)^T \boldsymbol{g}^{(k-1)}$ is positive when consecutive gradients **agree** (we are heading consistently downhill → **increase** α) and negative when they **oppose** (we overshot and reversed → **decrease** α). The learning rate tunes itself from the geometry.

4.7 First-Order Methods - Collected Update Rules

All seven in one place

1. Gradient Descent (GD)

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \alpha \boldsymbol{g}^{(k)}$$

2. Conjugate Gradient Descent

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha \left(-\boldsymbol{g}^{(k+1)} + \beta^{(k)} \boldsymbol{d}^{(k)}\right)$$

3. Gradient Descent with momentum

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \alpha \boldsymbol{g}^{(k)} + \beta \boldsymbol{v}^{(k)}$$

4. Adagrad

$$x_i^{(k+1)} = x_i^{(k)} - \frac{\alpha}{\epsilon + \sqrt{s_i^{(k)}}} g_i^{(k)}, \quad \text{where } s_i^{(k)} = \sum_{j=1}^{k} \left(g_i^{(k)}\right)^2$$

5. RMSProp

$$x_i^{(k+1)} = x_i^{(k)} - \frac{\alpha}{\epsilon + \mathrm{RMS}(g_i)} g_i^{(k)}$$

6. Adadelta

$$x_i^{(k+1)} = x_i^{(k)} - \frac{\mathrm{RMS}(\Delta x_i)}{\epsilon + \mathrm{RMS}(g_i)} g_i^{(k)}$$

7. Adam

$$\boldsymbol{v}^{(k+1)} = \gamma_v \boldsymbol{v}^{(k)} + (1 - \gamma_v) \boldsymbol{g}^{(k)}$$

$$\boldsymbol{s}^{(k+1)} = \gamma_s \boldsymbol{s}^{(k)} + (1-\gamma_s)\left(\boldsymbol{g}^{(k)} \odot \boldsymbol{g}^{(k)}\right)$$

$$\hat{\boldsymbol{v}}^{(k+1)} = \frac{\boldsymbol{v}^{(k+1)}}{1-\gamma_v^k}, \qquad \hat{\boldsymbol{s}}^{(k+1)} = \frac{\boldsymbol{s}^{(k+1)}}{1-\gamma_s^k}$$

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \frac{\alpha \hat{\boldsymbol{v}}^{(k+1)}}{\varepsilon + \sqrt{\hat{\boldsymbol{s}}^{(k+1)}}}$$

Sign of the gradient-descent update

Gradient descent moves against the gradient:

$$x_{k+1} = x_k - \alpha_k g_k, \qquad g_k = \nabla f(x_k), \quad \alpha_k > 0.$$

A plus sign instead defines gradient ascent. Keep the direction convention and the update sign consistent.

4.8 The Head-to-Head Comparison

Setup

$$f(\boldsymbol{x}) = \tfrac{1}{2}\left(x_1^2 + b x_2^2\right), \quad b = 0.001, \quad \boldsymbol{x}^{(0)} = [5,\ 4], \quad \alpha = 0.1, \quad 500 \text{ iterations}$$

First 50 iterations

	GD	GD+Momentum	Adagrad
$\boldsymbol{x}^{(0)}$	$[5,\ 4]$	$[5,\ 4]$	$[5,\ 4]$
$\boldsymbol{x}^{(10)}$	$[1.743,\ 3.996]$	$[-2.944,\ 3.983]$	$[4.508,\ 3.510]$
$\boldsymbol{x}^{(20)}$	$[0.608,\ 3.992]$	$[1.723,\ 3.952]$	$[4.264,\ 3.270]$
$\boldsymbol{x}^{(30)}$	$[0.212,\ 3.988]$	$[-1.003,\ 3.915]$	$[4.079,\ 3.089]$
$\boldsymbol{x}^{(40)}$	$[0.074,\ 3.984]$	$[0.580,\ 3.877]$	$[3.925,\ 2.939]$
$\boldsymbol{x}^{(50)}$	$[0.026,\ 3.980]$	$[-0.333,\ 3.838]$	$[3.791,\ 2.809]$

	RMSProp	Adadelta	Adam
$\boldsymbol{x}^{(0)}$	$[5,\ 4]$	$[5,\ 4]$	$[5,\ 4]$
$\boldsymbol{x}^{(10)}$	$[3.381,\ 2.414]$	$[4.997,\ 3.997]$	$[4.008,\ 3.010]$
$\boldsymbol{x}^{(20)}$	$[2.452,\ 1.550]$	$[4.993,\ 3.993]$	$[3.060,\ 2.080]$
$\boldsymbol{x}^{(30)}$	$[1.677,\ 0.881]$	$[4.989,\ 3.989]$	$[2.203,\ 1.276]$
$\boldsymbol{x}^{(40)}$	$[1.018,\ 0.388]$	$[4.985,\ 3.985]$	$[1.476,\ 0.651]$
$\boldsymbol{x}^{(50)}$	$[0.497,\ 0.104]$	$[4.981,\ 3.981]$	$[0.901,\ 0.230]$

Out to 500 iterations

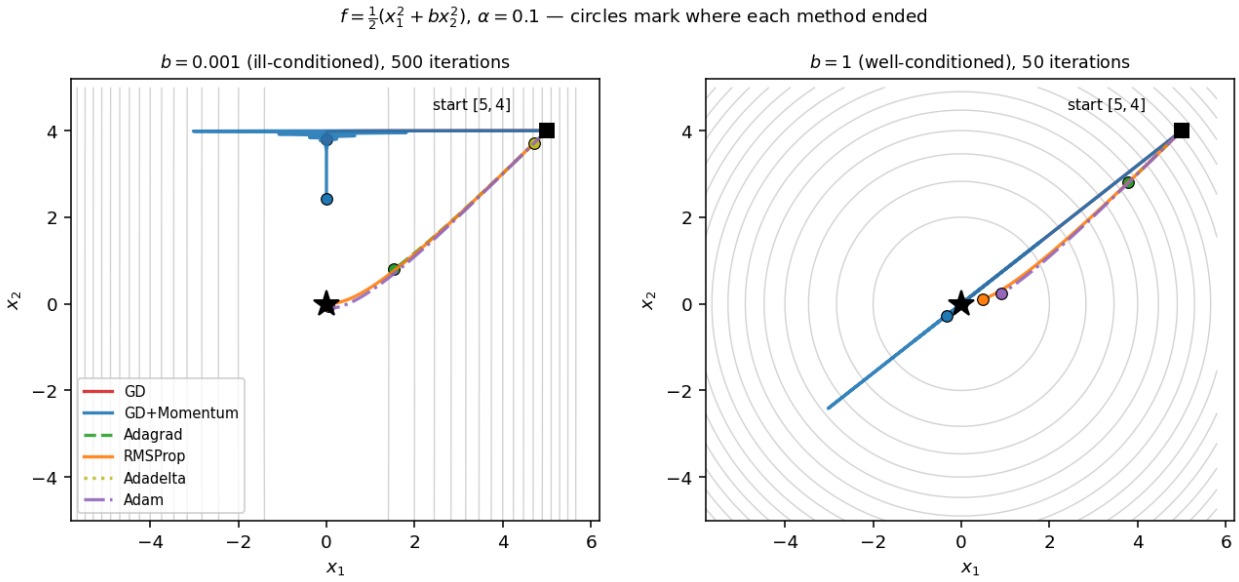
	GD	GD+Momentum	Adagrad
$\boldsymbol{x}^{(0)}$	[5, 4]	[5, 4]	[5, 4]
$\boldsymbol{x}^{(100)}$	[1.328e-04, 3.960]	[0.019, 3.649]	[3.283, 2.324]
$\boldsymbol{x}^{(200)}$	[3.528e-09, 3.921]	[6.974e-06, 3.299]	[2.620, 1.711]
$\boldsymbol{x}^{(300)}$	[9.370e-14, 3.882]	[−4.444e-07, 2.982]	[2.159, 1.307]
$\boldsymbol{x}^{(400)}$	[2.489e-18, 3.843]	[−3.508e-09, 2.696]	[1.807, 1.014]
$\boldsymbol{x}^{(500)}$	[6.610e-23, 3.805]	[−1.443e-11, 2.437]	[1.526, 0.793]

	RMSProp	Adadelta	Adam
$\boldsymbol{x}^{(0)}$	[5, 4]	[5, 4]	[5, 4]
$\boldsymbol{x}^{(100)}$	[2.372e-16, −1.754e-20]	[4.960, 3.960]	[−0.039, −0.021]
$\boldsymbol{x}^{(200)}$	[0.043, −0.046]	[4.908, 3.909]	[−0.00011, 0.00013]
$\boldsymbol{x}^{(300)}$	[0.050, −0.050]	[4.849, 3.850]	[−1.83e-06, −6.94e-07]
$\boldsymbol{x}^{(400)}$	[0.050, −0.050]	[4.782, 3.784]	[5.12e-09, 4.90e-09]
$\boldsymbol{x}^{(500)}$	[0.050, −0.050]	[4.709, 3.712]	[−2.68e-11, 2.42e-11]

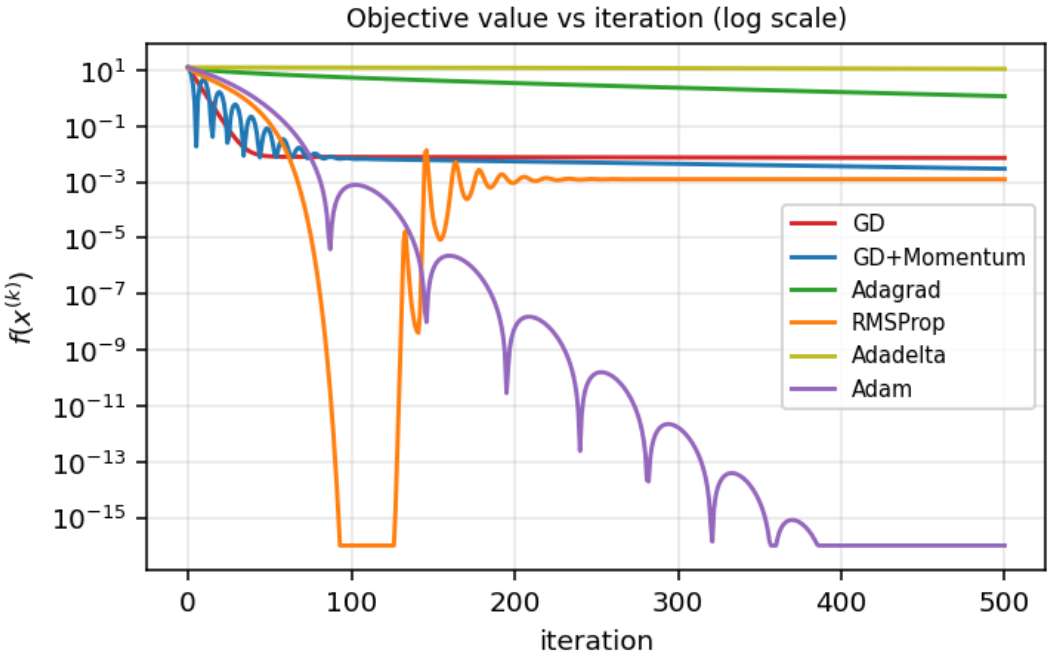
Convergence on an ill-conditioned quadratic

Method	Verdict on this problem
GD	Kills x_1 to 10^{-23} but x_2 only reaches 3.805 after 500 iterations - essentially no progress on the flat direction
GD+Momentum	Oscillates wildly in x_1 early on (note the sign flips $-2.944 \to 1.723 \to -1.003$), but drags x_2 down to 2.437 - **better than GD, thanks to accumulated velocity along the flat floor**
Adagrad	Steady progress on both coordinates, but visibly **slowing down** (x_2: $2.324 \to 1.711 \to 1.307 \to 1.014 \to 0.793$) - the monotonic denominator at work
RMSProp	Reaches the neighbourhood fastest of the adaptive methods, but then **parks at** $[0.050, -0.050]$ **and stops improving**
Adadelta	Barely moves at all ($[4.709, 3.712]$ after 500 iterations) - far too conservative here
Adam	Converges cleanly to $\approx [-2.7 \times 10^{-11}, 2.4 \times 10^{-11}]$ - the clear winner on this problem

Hyperparameters (recovered by reproducing the tables exactly): momentum $\beta = 0.9$; RMSProp and Adadelta $\gamma = 0.9$; Adam $\gamma_v = 0.9$, $\gamma_s = 0.999$; all with $\alpha = 0.1$.



*Left - the ill-conditioned case from the tables: GD (red) and Adagrad race along x_1 then stall at $x_2 \approx 3.8$; momentum (blue) thrashes back and forth in x_1 but is the only non-adaptive method to make real headway in x_2; Adam and RMSProp curve straight down to the origin. Right - with $b = 1$ the bowl is round and **every method behaves well**, which is the point: these algorithms are distinguished by how they handle bad conditioning, not good.*



The same runs as objective value against iteration. Adam falls to machine precision; RMSProp plateaus; Adagrad and Adadelta decay ever more slowly; GD flattens out once x_1 is exhausted and only the untouched x_2 term remains.

Comparison with $b = 1$, $\boldsymbol{x}^{(0)} = [5, 4]$, $\alpha = 0.1$, 50 iterations

On the well-conditioned bowl all six methods trace nearly the same straight path into the minimum. **Conditioning, not the algorithm, is what creates the difficulty.**

4.9 Second-Order Methods

4.9.1 Newton's method

The update

Multivariate function, second-order Taylor expansion:

$$q(\boldsymbol{x}) = f(\boldsymbol{x}^{(k)}) + \left(\boldsymbol{g}^{(k)}\right)^T (\boldsymbol{x} - \boldsymbol{x}^{(k)}) + \tfrac{1}{2}(\boldsymbol{x} - \boldsymbol{x}^{(k)})^T \boldsymbol{H}^{(k)} (\boldsymbol{x} - \boldsymbol{x}^{(k)})$$

$$\nabla q(\boldsymbol{x}^{(k)}) = 0 \rightarrow \boldsymbol{g}^{(k)} + \boldsymbol{H}^{(k)}(\boldsymbol{x} - \boldsymbol{x}^{(k)}) = 0$$

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \left(\boldsymbol{H}^{(k)}\right)^{-1} \boldsymbol{g}^{(k)}$$

Challenges

1. The **second derivative / Hessian may not be available** at $\boldsymbol{x}^{(k)}$, **or computing the Hessian is computationally high**.

Classical modified Newton's method

Use the Hessian matrix at $\boldsymbol{x}^{(0)}$ for all iterations:

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \left(\boldsymbol{H}^{(0)}\right)^{-1} \boldsymbol{g}^{(k)}$$

Compute and invert the Hessian **once**, then reuse it forever. **This approach may not be effective if the Hessian is changing rapidly.**

4.9.2 Quasi-Newton methods

The framework

Quasi-Newton methods **approximate the inverse of the Hessian**. These methods usually start by setting an initial guess for the inverse of the Hessian matrix to the **identity matrix**.

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \alpha^{(k)} \boldsymbol{Q}^{(k)} \boldsymbol{g}^{(k)}$$

- $\alpha^{(k)}$ - scaling factor
- $\boldsymbol{Q}^{(k)}$ approximates $\left(\boldsymbol{H}^{(k)}\right)^{-1}$
- Typically we start with an identity matrix: $\boldsymbol{Q}^{(1)} = \boldsymbol{I}$
- $\boldsymbol{Q}^{(k+1)} = \boldsymbol{Q}^{(k)} + \Delta$

Our objective is:

$$\lim_{k \to \infty} \boldsymbol{Q}^{(k)} = \boldsymbol{H}^{(k)}$$

- **Rank one update:** Δ is a rank one matrix
- **Rank two update:** Δ is a rank two matrix

Initial approximation
$Q^{(1)} = I$: the initial direction is steepest descent.

↓

Learn curvature
$Q^{(k+1)} = Q^{(k)} + \Delta Q^{(k)}$ using successive steps and gradients.

↓

Approximate Newton behaviour
$Q^{(k)}$ estimates the inverse Hessian without forming it directly.

Progressively incorporating curvature

$\boldsymbol{Q = I}$ **is** gradient descent. $\boldsymbol{Q = H^{-1}}$ **is** Newton's method. Quasi-Newton starts at the first and walks toward the second, learning curvature from the gradients it has already had to compute anyway.

4.9.3 Rank-one correction - the derivation

What "rank" means here

Rank of a matrix: the maximum number of linearly independent columns. A rank-one matrix can be written as an outer product $\boldsymbol{A} = \boldsymbol{u}\boldsymbol{v}^T$:

$$\begin{bmatrix} 1 & 3 \\ 2 & 6 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \begin{bmatrix} 1 & 3 \end{bmatrix}$$

Deriving the rank-one update

Rank one correction:

$$\boldsymbol{Q}^{(k+1)} = \boldsymbol{Q}^{(k)} + \Delta = \boldsymbol{Q}^{(k)} + a^{(k)}\boldsymbol{w}^{(k)} \left(\boldsymbol{w}^{(k)}\right)^T$$

Next design point $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \alpha^{(k)}\boldsymbol{Q}^{(k)}\boldsymbol{g}^{(k)}$ with $\boldsymbol{d}^{(k)} = \boldsymbol{Q}^{(k)}\boldsymbol{g}^{(k)}$. Now

$$\boldsymbol{d}^{(k+1)} = \boldsymbol{Q}^{(k)}\boldsymbol{g}^{(k+1)} + a^{(k)}\boldsymbol{w}^{(k)} \left(\boldsymbol{w}^{(k)}\right)^T \boldsymbol{g}^{(k+1)}$$

$$a^{(k)}\boldsymbol{w}^{(k)} = \frac{\boldsymbol{d}^{(k+1)} - \boldsymbol{Q}^{(k)}\boldsymbol{g}^{(k+1)}}{\left(\boldsymbol{w}^{(k)}\right)^T \boldsymbol{g}^{(k+1)}}$$

Here $\left(\boldsymbol{w}^{(k)}\right)^T \boldsymbol{g}^{(k+1)}$ is a **scalar**, so we may read off

$$a^{(k)} = \frac{1}{\left(\boldsymbol{w}^{(k)}\right)^T \boldsymbol{g}^{(k+1)}}, \qquad \boldsymbol{w}^{(k)} = \boldsymbol{d}^{(k+1)} - \boldsymbol{Q}^{(k)}\boldsymbol{g}^{(k+1)}$$

Substituting back:

$$a^{(k)} = \frac{1}{\left(\boldsymbol{d}^{(k+1)} - \boldsymbol{Q}^{(k)}\boldsymbol{g}^{(k+1)}\right)^T \boldsymbol{g}^{(k+1)}}$$

$$\Delta = a^{(k)}\boldsymbol{w}^{(k)} \left(\boldsymbol{w}^{(k)}\right)^T = \frac{\boldsymbol{w}^{(k)} \left(\boldsymbol{w}^{(k)}\right)^T}{\left(\boldsymbol{d}^{(k+1)} - \boldsymbol{Q}^{(k)}\boldsymbol{g}^{(k+1)}\right)^T \boldsymbol{g}^{(k+1)}}$$

$$\Delta = \frac{\left(\boldsymbol{d}^{(k+1)} - \boldsymbol{Q}^{(k)}\boldsymbol{g}^{(k+1)}\right)\left(\boldsymbol{d}^{(k+1)} - \boldsymbol{Q}^{(k)}\boldsymbol{g}^{(k+1)}\right)^T}{\left(\boldsymbol{d}^{(k+1)} - \boldsymbol{Q}^{(k)}\boldsymbol{g}^{(k+1)}\right)^T \boldsymbol{g}^{(k+1)}}$$

$$\boldsymbol{Q}^{(k+1)} = \boldsymbol{Q}^{(k)} + \Delta$$

In terms of γ and δ

Define

$$\boldsymbol{\gamma}^{(k+1)} = \boldsymbol{g}^{(k+1)} - \boldsymbol{g}^{(k)} \quad \text{(change in gradient)}$$

$$\boldsymbol{\delta}^{(k+1)} = \boldsymbol{x}^{(k+1)} - \boldsymbol{x}^{(k)} \quad \text{(change in design point)}$$

For the quadratic case $f(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^T\boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}^T\boldsymbol{x} + c$, we have $\boldsymbol{H} = \boldsymbol{A}^{-1}$ and $\boldsymbol{\gamma}^{(k+1)} = \boldsymbol{H}^{(k)}\boldsymbol{\delta}^{(k+1)}$. The update becomes

$$\boldsymbol{Q}^{(k+1)} = \boldsymbol{Q}^{(k)} + \frac{\left(\boldsymbol{\delta}^{(k)} - \boldsymbol{Q}^{(k)}\boldsymbol{\gamma}^{(k)}\right)\left(\boldsymbol{\delta}^{(k)} - \boldsymbol{Q}^{(k)}\boldsymbol{\gamma}^{(k)}\right)^T}{\left(\boldsymbol{\gamma}^{(k)}\right)^T\left(\boldsymbol{\delta}^{(k)} - \boldsymbol{Q}^{(k)}\boldsymbol{\gamma}^{(k)}\right)}$$

Two issues with the rank-one update

1. **Numerical instability when** $\left(\boldsymbol{\delta}^{(k)} - \boldsymbol{Q}^{(k)}\boldsymbol{\gamma}^{(k)}\right) \to 0$ - the denominator vanishes.
2. $\boldsymbol{Q}^{(k+1)}$ **may not be positive definite always** - and a non-positive-definite $\boldsymbol{Q}$ can produce an ascent direction.

Alternative: rank two update.

4.9.4 DFP - Davidon–Fletcher–Powell

Rank-two correction

$$\boldsymbol{Q}^{(k+1)} = \boldsymbol{Q}^{(k)} + \Delta = \boldsymbol{Q}^{(k)} + a^{(k)}\boldsymbol{w}^{(k)}\left(\boldsymbol{w}^{(k)}\right)^T + \bar{a}^{(k)}\bar{\boldsymbol{w}}^{(k)}\left(\bar{\boldsymbol{w}}^{(k)}\right)^T$$

DFP method:

$$\boldsymbol{Q}^{(k+1)} = \boldsymbol{Q}^{(k)} - \frac{\boldsymbol{Q}^{(k)}\boldsymbol{\gamma}^{(k)}\left(\boldsymbol{\gamma}^{(k)}\right)^T\boldsymbol{Q}^{(k)}}{\left(\boldsymbol{\gamma}^{(k)}\right)^T\boldsymbol{Q}^{(k)}\boldsymbol{\gamma}^{(k)}} + \frac{\boldsymbol{\delta}^{(k)}\left(\boldsymbol{\delta}^{(k)}\right)^T}{\left(\boldsymbol{\delta}^{(k)}\right)^T\boldsymbol{\gamma}^{(k)}}$$

Properties

- Also known as the **Variable Metric Method**.
- If $f(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^T\boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}^T\boldsymbol{x} + c$, then $\boldsymbol{Q} = \boldsymbol{A}^{-1}$.
- **If $\boldsymbol{Q}^{(0)}$ is positive definite, it can be shown that $\boldsymbol{Q}^{(k)}$ is positive definite** - this is exactly the guarantee the rank-one update lacked.
- **For high dimension:**
  1. May be significant computational requirement compared to first-order methods.
  2. An alternative: **Broyden–Fletcher–Goldfarb–Shanno (BFGS)**.

4.9.5 BFGS and L-BFGS

BFGS update

$$\boldsymbol{Q}^{(k+1)} = \boldsymbol{Q}^{(k)} - \frac{\boldsymbol{\delta}^{(k)} \left(\boldsymbol{\gamma}^{(k)}\right)^T \boldsymbol{Q}^{(k)} + \boldsymbol{Q}^{(k)} \boldsymbol{\gamma}^{(k)} \left(\boldsymbol{\delta}^{(k)}\right)^T}{\left(\boldsymbol{\delta}^{(k)}\right)^T \boldsymbol{\gamma}^{(k)}} + \left(1 + \frac{\left(\boldsymbol{\gamma}^{(k)}\right)^T \boldsymbol{Q}^{(k)} \boldsymbol{\gamma}^{(k)}}{\left(\boldsymbol{\delta}^{(k)}\right)^T \boldsymbol{\gamma}^{(k)}}\right) \frac{\boldsymbol{\delta}^{(k)} \left(\boldsymbol{\delta}^{(k)}\right)^T}{\left(\boldsymbol{\delta}^{(k)}\right)^T \boldsymbol{\gamma}^{(k)}}$$

When to use which

- **BFGS is better than DFP for high dimension problems.**
- For **very high dimensional problems, limited memory BFGS (L-BFGS)** can be used.
- **L-BFGS utilizes the last m values of $\boldsymbol{\delta}$ and $\boldsymbol{\gamma}$** - instead of storing the full $n \times n$ matrix $\boldsymbol{Q}$, it stores $2m$ vectors and reconstructs the action of $\boldsymbol{Q}$ on demand. This is what makes quasi-Newton feasible when n runs into the millions.

4.10 Second-Order Methods on the Test Problem

Same setup, now with Newton's method added

$f(\boldsymbol{x}) = \frac{1}{2}(x_1^2 + bx_2^2)$, $b = 0.001$, $\boldsymbol{x}^{(0)} = [5, 4]$, $\alpha = 0.1$, 500 iterations.

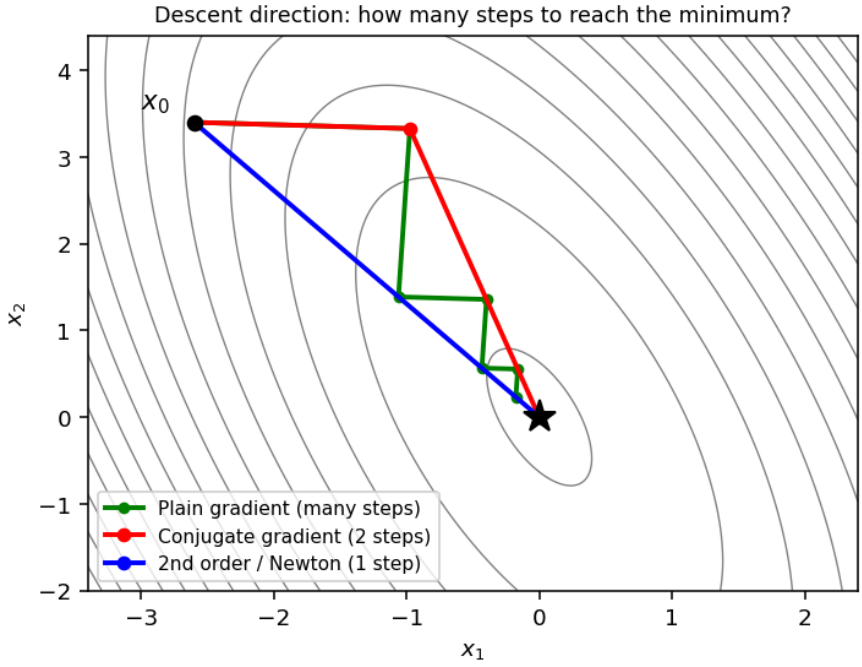
	RMSProp	Adadelta	Adam	**Newton's**
$\boldsymbol{x}^{(0)}$	$[5,\ 4]$	$[5,\ 4]$	$[5,\ 4]$	$[5,\ 4]$
$\boldsymbol{x}^{(10)}$	$[3.381,\ 2.414]$	$[4.997,\ 3.997]$	$[4.008,\ 3.010]$	$\boldsymbol{x}^{(1)} = [0,\ 0]$
$\boldsymbol{x}^{(20)}$	$[2.452,\ 1.550]$	$[4.993,\ 3.993]$	$[3.060,\ 2.080]$	
$\boldsymbol{x}^{(50)}$	$[0.497,\ 0.104]$	$[4.981,\ 3.981]$	$[0.901,\ 0.230]$	

Newton iteration on a quadratic

In the case where f is quadratic and its Hessian matrix is positive definite, Newton's method converges to the global minimum in just one step.

Here $\boldsymbol{H} = \begin{bmatrix} 1 & 0 \\ 0 & b \end{bmatrix}$ is constant and positive definite, so $\boldsymbol{x}^{(1)} = \boldsymbol{x}^{(0)} - \boldsymbol{H}^{-1}\boldsymbol{g}^{(0)} = [5, 4] - [5, 4] = [0, 0]$ - **exactly the minimum**, while Adam needs hundreds of iterations to get there and Adadelta never arrives. The conditioning that cripples every first-order method is *precisely the information the Hessian carries*.

4.10.1 Descent direction - the three families side by side



Plain gradient *(green) takes right-angle turns and needs many steps - consecutive steepest-descent directions with exact line search are orthogonal, which is what produces the zig-zag.* ***Conjugate gradient*** *(red) reaches the minimum of a 2-D quadratic in exactly* ***2*** *steps.* ***Second order*** *(blue) goes there in* ***1****. (Image concept adapted from "Approximate Second Order Methods" - Sargur N. Srihari.)*

4.11 Summary - One Formula for Everything

The unifying update

$$\boxed{\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \alpha^{(k)} \boldsymbol{s}^{(k)} \boldsymbol{g}^{(k)}}$$

Choice	Method
$\boldsymbol{s}^{(k)} = \boldsymbol{I}$	**Gradient descent**
$\boldsymbol{s}^{(k)} = \left(\boldsymbol{H}^{(k)}\right)^{-1}$ and $\alpha^{(k)} = 1$	**Newton's method**
$\boldsymbol{s}^{(k)} = \left(\boldsymbol{H}^{(k)}\right)^{-1}$, $\alpha^{(k)}$ chosen by line search	**Modified Newton method**
$\boldsymbol{s}^{(k)} = \boldsymbol{Q}^{(k)} \to \boldsymbol{H}^{-1}$	**Quasi-Newton (DFP, BFGS, L-BFGS)**
$\boldsymbol{s}^{(k)}$ = a diagonal matrix built from gradient history	**Adagrad, RMSProp, Adadelta, Adam**

Method comparison

Method	Uses	Cost per iteration	Strength	Weakness
Gradient descent	$\boldsymbol{g}$	Cheapest	Simple, always a descent direction	Crippled by ill-conditioning; zig-zags in valleys
Conjugate gradient	$\boldsymbol{g}$	Cheap	Exact on an n-D quadratic in $\leq n$ steps; no matrix storage	Needs β heuristics ($\boldsymbol{A}$ unknown); restarts needed off-quadratic
Momentum	$\boldsymbol{g}$	Cheap	Accelerates along flat directions, damps zig-zag	Can overshoot with high β
Nesterov momentum	$\boldsymbol{g}$	Cheap	Looks ahead - brakes before overshooting	Extra gradient evaluation point
Adagrad	$\boldsymbol{g}$	Cheap	Per-variable rates; tames dominant coordinates	Rates decay to zero **before** convergence
RMSProp	$\boldsymbol{g}$	Cheap	Fixes Adagrad's decay with a moving average	Can plateau short of the minimum
Adadelta	$\boldsymbol{g}$	Cheap	**No learning rate to tune**	Can be far too conservative
Adam	$\boldsymbol{g}$	Cheap	Momentum + adaptive rates + bias correction; best here	More hyperparameters ($\alpha, \gamma_v, \gamma_s, \varepsilon$)
Newton	$\boldsymbol{g}, \boldsymbol{H}$	Expensive ($\boldsymbol{H}^{-1}$)	**One step** on a positive-definite quadratic	Hessian may be unavailable, costly, or not positive definite
Modified Newton	$\boldsymbol{g}, \boldsymbol{H}^{(0)}$	One inversion total	Amortises the Hessian cost	Fails if the Hessian changes rapidly
DFP / BFGS	$\boldsymbol{g}$	Moderate ($n \times n$)	Newton-like without computing $\boldsymbol{H}$; preserves positive definiteness	$O(n^2)$ storage
L-BFGS	$\boldsymbol{g}$	Low	Scales to very high dimension using only the last m vectors	Approximation limited by the memory m

4.12 Key Takeaways

Summary

All of these methods are $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \alpha \boldsymbol{S}\boldsymbol{g}$. **First-order** methods set $\boldsymbol{S}$ from gradient history - conjugate directions, momentum, or per-variable adaptive rates (**Adam** = momentum + RMSProp + bias correction). **Second-order** methods set $\boldsymbol{S} = \boldsymbol{H}^{-1}$, which solves a positive-definite quadratic in **one step**, and **quasi-Newton** (DFP → BFGS → L-BFGS) builds that inverse from gradients alone when the real Hessian is out of reach.

Self-check

1. Write the steepest-descent direction, and compute it for $f = x_1 x_2^2 + 1$ at $[-2, -2]$.
2. What three conditions must hold for a steepest-descent step to guarantee improvement?
3. Run two iterations of gradient descent on $f = x_1^2 + x_2^2$ from $[1, 1]$ with $\alpha = 0.2$.
4. Why does the same absolute termination threshold behave differently on x^2 and $0.1x^2$?
5. Sketch the three step-size pathologies.
6. For $f = \frac{1}{2}(x_1^2 + bx_2^2)$ with $b = 0.001$, why does gradient descent stall even after 2000 iterations?
7. Derive $\alpha = -\dfrac{\boldsymbol{d}^T(\boldsymbol{A}\boldsymbol{x} + \boldsymbol{b})}{\boldsymbol{d}^T\boldsymbol{A}\boldsymbol{d}}$ for a quadratic.
8. Starting from $\boldsymbol{d}^{(k+1)T}\boldsymbol{A}\boldsymbol{d}^{(k)} = 0$, derive $\beta^{(k)}$.
9. Why are Fletcher–Reeves and Polak–Ribière needed at all? What does $\max(\beta, 0)$ do?
10. Write the momentum update. What does it reduce to when $\beta = 0$?
11. How does Nesterov momentum differ from plain momentum, and what problem does it fix?
12. State Adagrad's update. What is its weakness, and how do RMSProp and Adadelta each fix it?
13. What is unusual about Adadelta's hyperparameters?
14. List Adam's five steps. Why are steps 3 and 4 (bias correction) necessary?
15. Explain the sign of $\left(\boldsymbol{g}^{(k)}\right)^T \boldsymbol{g}^{(k-1)}$ in hypergradient descent.
16. From the comparison tables, which method wins and which two fail? Explain each in terms of its update rule.
17. What is the classical modified Newton's method, and when does it fail?
18. What is $\boldsymbol{Q}^{(1)}$ in a quasi-Newton method, and what is $\lim_{k\to\infty} \boldsymbol{Q}^{(k)}$ meant to be?
19. What is a rank-one matrix? Give the two failure modes of the rank-one update.
20. What guarantee does DFP provide that the rank-one update does not?
21. When would you choose BFGS over DFP, and L-BFGS over BFGS?
22. Why does Newton's method solve $\frac{1}{2}(x_1^2 + bx_2^2)$ in exactly one step?
23. On a 2-D quadratic, how many steps do plain gradient, conjugate gradient, and Newton each need?
24. Fill in $\boldsymbol{s}^{(k)}$ in $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \alpha^{(k)}\boldsymbol{s}^{(k)}\boldsymbol{g}^{(k)}$ for gradient descent, Newton, and quasi-Newton.

CHAPTER 5

Coding Examples - Unconstrained Optimization in Python

Overview

Python libraries provide practical implementations of scalar search, Newton and quasi-Newton methods, trust-region algorithms, and adaptive gradient methods:

- **`scipy.optimize`** for the classical methods - `bracket`, `bisect`, `line_search`, `newton`, and `minimize` with `Newton-CG`, `BFGS`, and `trust-ncg`.
- **PyTorch** for the first-order adaptive methods, using **automatic differentiation** so no gradient is ever written by hand.

The essential implementation choices are the solver, its required derivative information, the initial point, and the stopping criteria. Always inspect the termination status and residual quantities as well as the returned point.

Interpreting example outputs

The numerical outputs illustrate solver behaviour for the specified functions and initial points. Iteration counts, evaluation counts, last digits, and stopping messages can vary with the library version, floating-point arithmetic, and tolerances. Check feasibility, objective value, gradient norm, and solver status rather than expecting an exact match to a printed transcript.

5.1 Environment and Tooling

Python environment

- **SciPy:** "Fundamental algorithms for scientific computing in Python"
- **Python:** https://www.python.org
- **Beginners guide:** https://www.python.org/about/gettingstarted/
- **SciPy installation** - python command:

```
python -m pip install scipy
```

- **Google Colab:** a cloud-based platform provided by Google that allows users to write, execute, and share Python code in a **Jupyter notebook** environment.
- The relevant module is **`scipy.optimize`** - *Optimization and root finding*.

Notebook execution environment

The examples can run in a local Python environment or a hosted notebook. Install the required packages, execute the imports, and run each example with its stated initial values and tolerances. An accelerator is optional for these small optimization problems.

5.2 Bracketing - `scipy.optimize.bracket`

Method and assumptions

Bracketing: identify three points $a < b < c$ such that

$$f(a) > f(b) < f(c)$$

The bracketing search proceeds by **initializing at a starting point** and walking outward until it reaches an **end point** where the function has risen again.

Python code

Based on `scipy.optimize.bracket`

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import bracket

def f(x):
    return 10*x**2 + 3*x + 5

x = np.linspace(-2, 2)
y = f(x)
init_xa, init_xb = 0.1, 1
xa, xb, xc, fa, fb, fc, funcalls = bracket(f, xa=init_xa, xb=init_xb,
                                           args=())
plt.axvline(x=init_xa, color="k", linestyle="--")
plt.axvline(x=init_xb, color="k", linestyle="--")
plt.plot(x, y, "-k")
plt.plot(xa, fa, "bx")
plt.plot(xb, fb, "rx")
plt.plot(xc, fc, "bx")
plt.show()
```

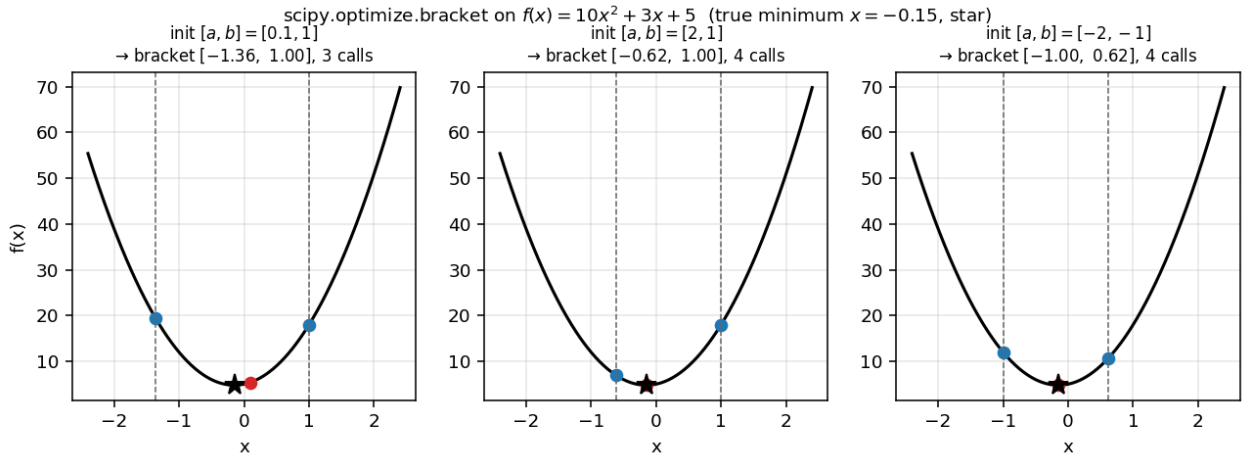
What `bracket` returns

A 7-tuple: **`xa, xb, xc`** (the three bracketing abscissae, with $f(x_b)$ the lowest), **`fa, fb, fc`** (the function values there), and **`funcalls`** (how many times f was evaluated).

Example output for three initial pairs

$f(x) = 10x^2 + 3x + 5$, whose true minimum is at $x = -\dfrac{3}{20} = -0.15$.

Initial $[a, b]$	x_a	x_b (lowest)	x_c	$f(x_b)$	function calls
$[0.1,\ 1]$	1.00000	0.10000	−1.35623	5.400	3
$[2,\ 1]$	1.00000	−0.15000	−0.61803	4.775	4
$[-2,\ -1]$	−1.00000	−0.15000	0.61803	4.775	4



Three things to notice

1. **The returned points are not in ascending order.** `bracket` returns them in *search* order, so $x_a > x_b > x_c$ in the first row. The bracket is the interval between x_a and x_c; x_b is the interior point.
2. **The initial pair does not have to straddle the minimum.** The search compares the objective values at its first two points. If the trial direction is uphill, it swaps the points and reverses direction before expanding the search.
3. The number 0.61803 appearing in the last two rows is the **golden ratio conjugate** - SciPy's `bracket` grows the step by a golden-ratio factor, $(\sqrt{5} - 1)/2$.

5.3 Bisection - `scipy.optimize.bisect`

Method and assumptions

Consider a function $f(x)$ which is **continuous in the bracket** $[a, b]$. **What interval contains roots?** The one where the sign changes:

$$f(a) \times f(b) \leq 0$$

Python code

Based on `scipy.optimize.bisect`

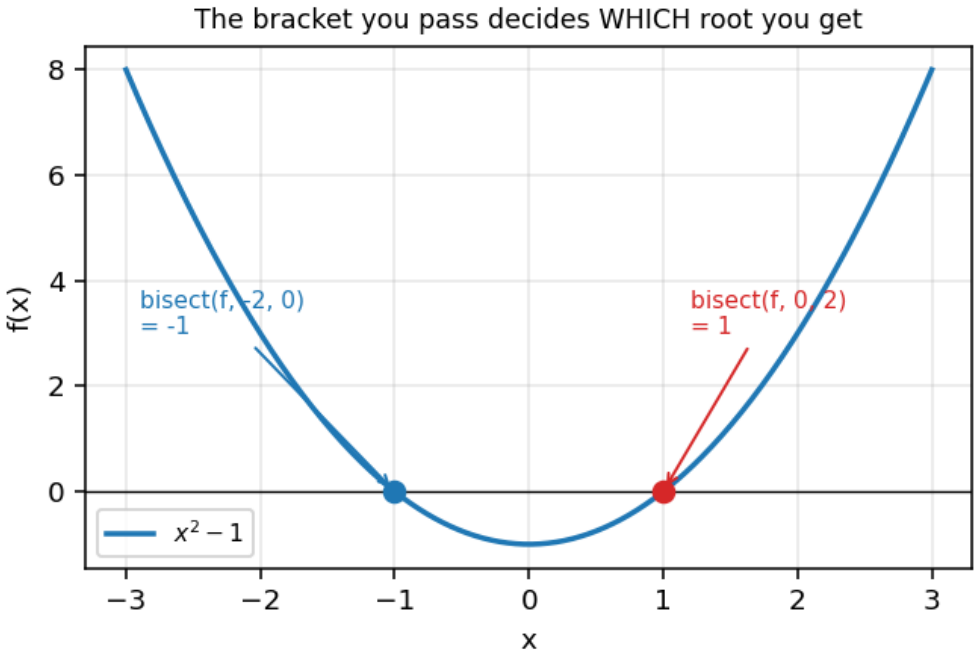
```
import numpy as np
import matplotlib.pyplot as plt
from scipy import optimize

def f(x):
    return (x**2 - 1)

# scipy.optimize.bisect(f, a, b, args=(), xtol=2e-12,
#                       rtol=8.881784197001252e-16,
#                       maxiter=100, full_output=False, disp=True)
root = optimize.bisect(f, 0, 2)    # get  1
root = optimize.bisect(f, -2, 0)   # get -1
```

Example output

```
optimize.bisect(f, 0, 2)   → 1.0
optimize.bisect(f, -2, 0)  → -1.0
```



The bracket you pass decides which root you get

$x^2 - 1$ has **two** roots. `bisect` is not searching globally - it is exploiting the sign change inside the interval you supplied, so **you choose the answer by choosing the bracket**. Passing $[-2,\ 2]$ would fail outright, because $f(-2) = f(2) = 3$ and there is no sign change.

The keyword arguments worth knowing

Argument	Meaning
`xtol=2e-12`	Absolute tolerance on the root's position
`rtol=8.88e-16`	Relative tolerance (machine epsilon scale)
`maxiter=100`	Iteration cap - recall bisection needs $\log_2(\|b-a\|/\varepsilon)$ iterations
`full_output=False`	Set `True` to also get a `RootResults` object with the iteration count
`disp=True`	Raise an error if convergence fails rather than returning silently

Using it for optimization

to minimize, pass f' rather than f. `bisect` finds roots; the roots of the derivative are the stationary points.

5.4 Line Search - `scipy.optimize.line_search`

Method and assumptions

- Assume that the **descent direction d is known**.
- Need to determine the **step size** α:

$$\min_{\alpha} f(\boldsymbol{x} + \alpha \boldsymbol{d})$$

- This is a **univariate optimization problem**, and **it is possible to use 1-D methods**.

Python code

Based on `scipy.optimize.line_search`

```
import numpy as np
from scipy.optimize import line_search

def obj_func(x):
    return (x[0])**2 + (x[1])**2

def obj_grad(x):
    return [2*x[0], 2*x[1]]

start_point = np.array([1.8, 1.7])
search_gradient = np.array([-1.0, -1.0])
line_search(obj_func, obj_grad, start_point, search_gradient)
```

Example output

```
(1.0, 2, 1, 1.13, 6.13, [1.6, 1.4])
```

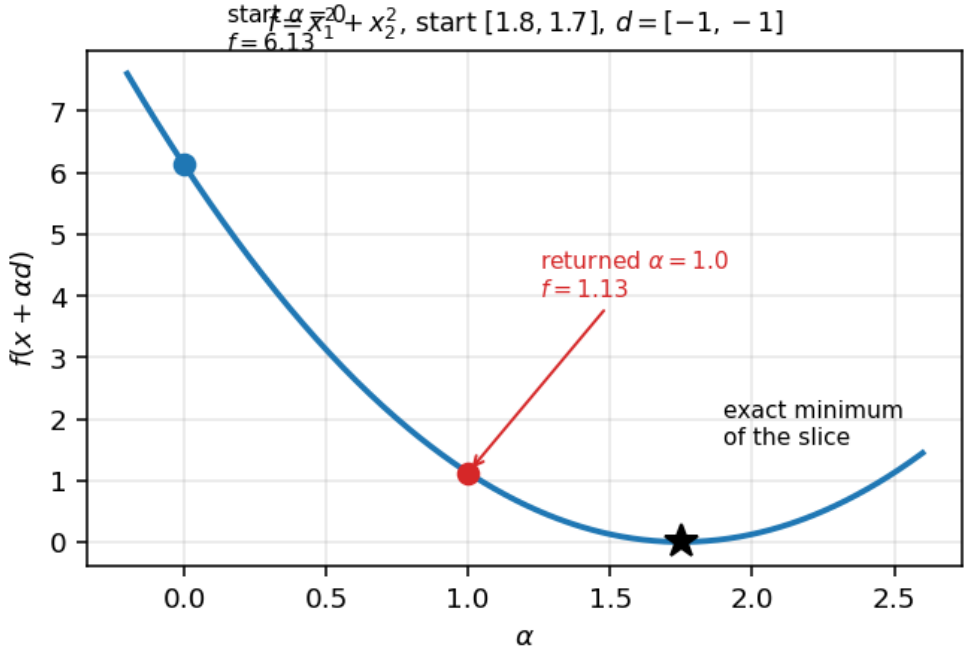
Output of the code - (step size, Number of function evaluations, Number of gradient evaluations, New function value, Old function value, new slope):

Position	Value	Meaning
`alpha`	1.0	The accepted step size
`fc`	2	Number of function evaluations used
`gc`	1	Number of gradient evaluations used
`new_fval`	1.13	f at the new point
`old_fval`	6.13	f at the starting point
`new_slope`	$[1.6,\ 1.4]$	Directional-derivative information at the new point

Checking the numbers by hand

Start $\boldsymbol{x} = [1.8,\ 1.7]$, direction $\boldsymbol{d} = [-1,\ -1]$.

- Old value: $1.8^2 + 1.7^2 = 3.24 + 2.89 = \boxed{6.13}$ ✓
- With $\alpha = 1.0$: new point $= [1.8 - 1,\ 1.7 - 1] = [0.8,\ 0.7]$
- New value: $0.8^2 + 0.7^2 = 0.64 + 0.49 = \boxed{1.13}$ ✓
- Gradient there: $[2(0.8),\ 2(0.7)] = \boxed{[1.6,\ 1.4]}$ ✓ - this is what appears in the sixth slot.



`line_search` is *approximate*, not exact

The slice $f(\boldsymbol{x} + \alpha \boldsymbol{d})$ is minimized at $\alpha = 1.75$ (where the point reaches the origin), yet the function returns $\alpha = 1.0$. **This is correct behaviour** - `line_search` implements the **Wolfe conditions** and stops at the first *satisfactory* step, using only 2 function and 1 gradient evaluation. It is doing approximate line search, exactly as designed.

Note on the sixth return value

The tuple's last slot is documented as `new_slope`, a scalar directional derivative $\nabla f(\boldsymbol{x}_{new})^T \boldsymbol{d}$ - which here would be $[1.6, 1.4] \cdot [-1, -1] = -3.0$. The printed value $[1.6,\ 1.4]$ is the **gradient vector** at the new point, because `obj_grad` returns a Python **list** rather than a NumPy array, so the internal product is not contracted as expected. Return `np.array([2*x[0], 2*x[1]])` from your gradient function and the slot behaves as documented.

5.5 Newton's Method - `scipy.optimize.newton`

Newton's method as a root-finding method

Recall : Newton is fundamentally a **root finder**, applied to f' when used for optimization.

Python code - without a derivative

Based on `scipy.optimize.newton`

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import optimize

def f(x):
    return (x**3 - 1)
```

```
root = optimize.newton(f, 1.5)
print(root)
```

If gradient and Hessian are not given, the secant method is used.

Python code - with a derivative

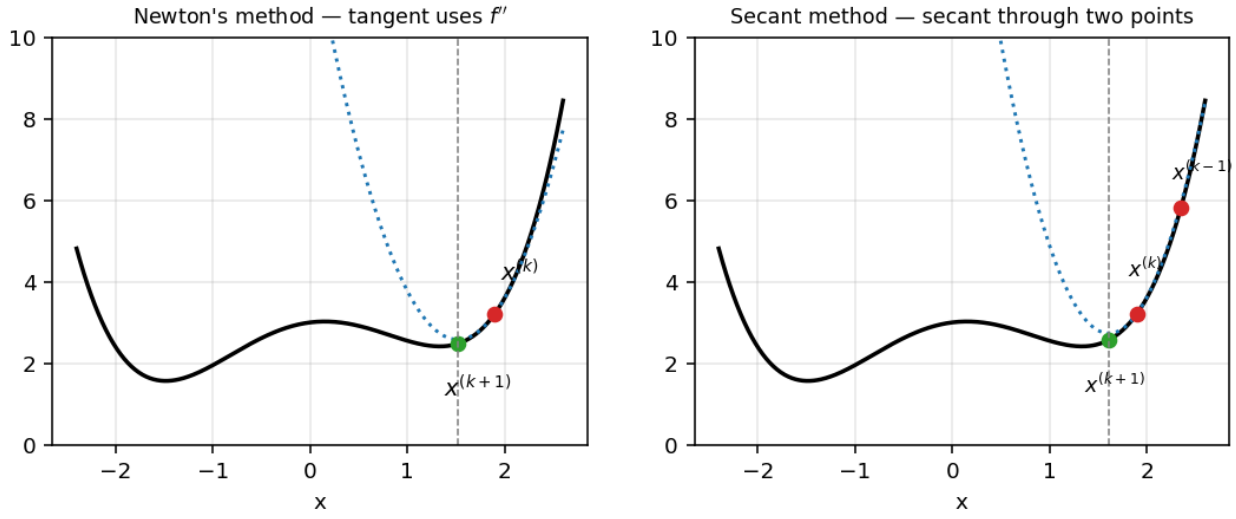
```
def f(x):
    return (x**3 - 1)

root = optimize.newton(f, 1.5, fprime=lambda x: 3 * x**2)
print(root)
```

Example output

Call	Method actually used	Result
`newton(f, 1.5)`	**Secant** (no `fprime` supplied)	1.0000000000000016
`newton(f, 1.5, fprime=...)`	**Newton** (true tangent)	1.0

The last digits can differ because of derivative information and stopping tolerances: an analytic derivative can improve local convergence, while secant convergence depends on successive function values and numerical precision.



The distinction the API is making. ***Newton*** *builds its model from the tangent at one point using f''; the* ***secant method*** *estimates that curvature from the line through the two most recent points, so it needs no second derivative but converges a little more slowly.*

Secant update

The "secant" between $x^{(k)}$ and $x^{(k-1)}$ is used to determine the next point $x^{(k+1)}$:

$$x^{(k+1)} = x^{(k)} - \frac{x^{(k)} - x^{(k-1)}}{f'(x^{(k)}) - f'(x^{(k-1)})} f'(x^{(k)})$$

5.6 Multivariate Newton - `minimize(method='Newton-CG')`

The theory

Newton's method: new design point

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \left(\boldsymbol{H}^{(k)}\right)^{-1} \boldsymbol{g}^{(k)}$$

Quasi-Newton Methods: Newton-Conjugate Gradient (Newton-CG). Here, **Newton-CG avoids explicit computation and inversion of the Hessian matrix by approximating it using conjugate gradient iterations.**

The test problem - Rosenbrock in n dimensions

$$f(\boldsymbol{x}, a, b) = \sum_{i=1}^{n-1} \left(a \left(x_{i+1} - x_i^2 \right)^2 + (1 - x_i)^2 \right) + b$$

With the standard $a = 100, b = 0$ the minimum is at $\boldsymbol{x} = [1, 1, \ldots, 1]$ with $f = 0$. This is a **narrow-valley** function that defeats plain gradient descent.

Python code - supplying gradient and Hessian

Based on the SciPy optimization tutorial

```
import numpy as np
from scipy.optimize import minimize

def rosen(x):
    """The Rosenbrock function"""
    return sum(100.0*(x[1:]-x[:-1]**2.0)**2.0 + (1-x[:-1])**2.0)

# Python function which computes gradient
def rosen_der(x):
    xm = x[1:-1]
    xm_m1 = x[:-2]
    xm_p1 = x[2:]
    der = np.zeros_like(x)
    der[1:-1] = 200*(xm-xm_m1**2) - 400*(xm_p1 - xm**2)*xm - 2*(1-xm)
    der[0] = -400*x[0]*(x[1]-x[0]**2) - 2*(1-x[0])
    der[-1] = 200*(x[-1]-x[-2]**2)
    return der

# Python function which computes hessian
def rosen_hess(x):
    x = np.asarray(x)
    H = np.diag(-400*x[:-1], 1) - np.diag(400*x[:-1], -1)
    diagonal = np.zeros_like(x)
    diagonal[0] = 1200*x[0]**2 - 400*x[1] + 2
    diagonal[-1] = 200
    diagonal[1:-1] = 202 + 1200*x[1:-1]**2 - 400*x[2:]
    H = H + np.diag(diagonal)
    return H

x0 = np.array([1.3, 0.7, 0.8, 1.9, 1.2])
res = minimize(rosen, x0, method='Newton-CG', jac=rosen_der,
               hess=rosen_hess, options={'xtol': 1e-8, 'disp': True})
```

Example output

```
Optimization terminated successfully.
         Current function value: 0.000000
         Iterations: 24
         Function evaluations: 33
         Gradient evaluations: 56
         Hessian evaluations: 24
    fun: 3.5306674342205174e-17
    jac: array([ 2.68742124e-08,  9.26710155e-08,  3.70109749e-07,
                 1.48479130e-06, -8.52636894e-07])
message: 'Optimization terminated successfully.'
   nfev: 33
   nhev: 24
    nit: 24
   njev: 56
 status: 0
success: True
      x: array([1., 1., 1., 0.99999999, 0.99999999])
```

Version note on `njev`

`njev` counts gradient evaluations recorded by the solver. Counts can differ between implementations and versions, particularly when an inner conjugate-gradient iteration also evaluates the gradient. Treat these counts as diagnostic information; compare the final objective, gradient norm, and termination status using the same library version and tolerances.

5.6.1 Hessian-vector products - `hessp`

Avoiding explicit Hessian storage

- **For larger minimization problems, storing the entire Hessian matrix can be time-consuming and memory-intensive.**
- **We require the product of the Hessian matrix with an arbitrary vector.**
- **Instead of supplying the full Hessian, we can provide a function that computes this product.**

Python code

```
def rosen_hess_p(x, p):
    x = np.asarray(x)
    Hp = np.zeros_like(x)
    Hp[0] = (1200*x[0]**2 - 400*x[1] + 2)*p[0] - 400*x[0]*p[1]
    Hp[1:-1] = (-400*x[:-2]*p[:-2]
                + (202 + 1200*x[1:-1]**2 - 400*x[2:])*p[1:-1]
                - 400*x[1:-1]*p[2:])
    Hp[-1] = -400*x[-2]*p[-2] + 200*p[-1]
    return Hp

res = minimize(rosen, x0, method='Newton-CG', jac=rosen_der,
               hessp=rosen_hess_p, options={'xtol': 1e-8, 'disp': True})
```

Hessian-vector product formulation

`hessp` reaches the same `nit=24`, `nfev=33`, `fun=3.53e-17` and the same `x`. Memory drops from $O(n^2)$ to $O(n)$ - this is the trick that makes second-order methods usable when n is large, and it is exactly why Newton-**CG** exists: conjugate gradient only ever needs $\boldsymbol{Hp}$, never $\boldsymbol{H}$ itself.

5.7 Quasi-Newton - `minimize(method='BFGS')`

Quasi-Newton principle

Quasi-Newton Methods: Broyden–Fletcher–Goldfarb–Shanno (BFGS) - approximates the inverse Hessian from gradient information alone, so **no `hess` argument is required at all**.

Python code

```
res = minimize(rosen, x0, method='BFGS', jac=rosen_der,
               options={'disp': True})
```

Example output

```
Optimization terminated successfully.
         Current function value: 0.000000
         Iterations: 25
         Function evaluations: 30
         Gradient evaluations: 30
    fun: 4.0130879949972905e-13
hess_inv: array([[0.00758796, 0.01243893, 0.02344025, 0.04614953, 0.09222281],
                 [0.01243893, 0.02481725, 0.04712952, 0.09298607, 0.18569385],
                 [0.02344025, 0.04712952, 0.09456412, 0.18674836, 0.37282072],
                 [0.04614953, 0.09298607, 0.18674836, 0.37383212, 0.74621435],
                 [0.09222281, 0.18569385, 0.37282072, 0.74621435, 1.49444785]])
    jac: array([-5.68982937e-06, -2.73296557e-06, -2.54520599e-06,
                -7.73460770e-06,  5.78142698e-06])
   nfev: 30
    nit: 25
   njev: 30
success: True
      x: array([1.00000004, 1.0000001, 1.00000021, 1.00000044, 1.00000092])
```

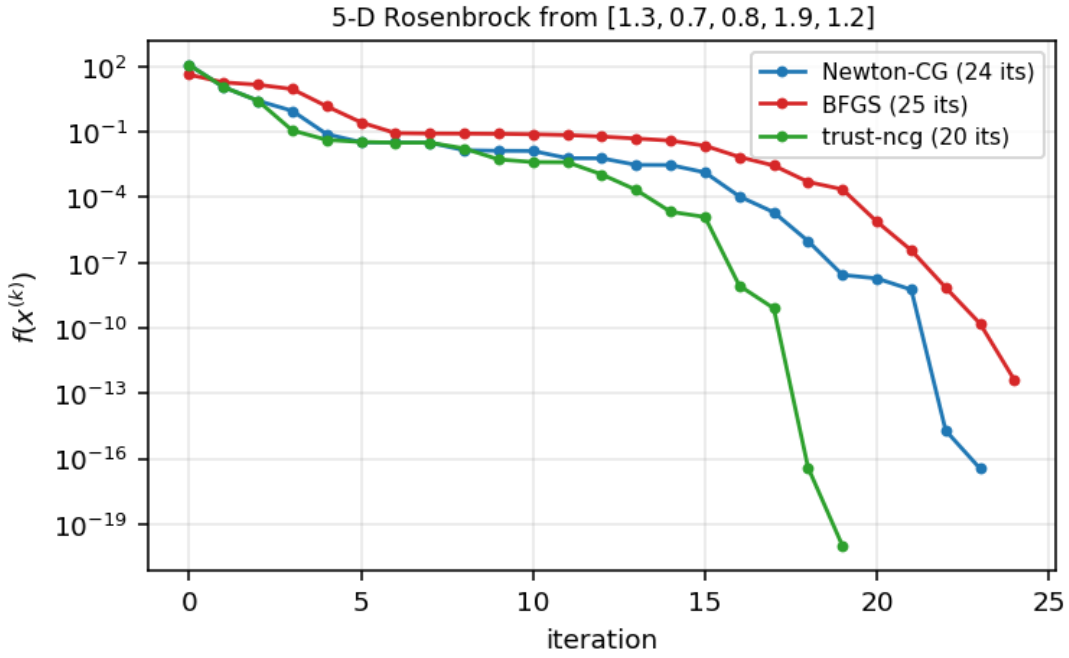
Reproduced exactly, including `fun = 4.0130879949972905e-13`.

Newton-CG and BFGS comparison

	Newton-CG	BFGS
Needs a Hessian?	**Yes** (`hess` or `hessp`)	**No** - only `jac`
Iterations	24	25
Function evaluations	33	30
Final f	3.5×10^{-17}	4.0×10^{-13}
Extra output	-	`hess_inv` - the learned $\boldsymbol{Q}^{(k)} \approx \boldsymbol{H}^{-1}$

BFGS gets within a hair of Newton-CG's iteration count **while never being told the curvature** by updating an inverse-Hessian estimate from successive steps and gradients. The `hess_inv` field stores

the estimated inverse Hessian Q, which can be inspected after termination.



Objective value per iteration for the three solvers on the same 5-D problem. All three fight through the Rosenbrock valley in a few dozen iterations. Curvature information can substantially reduce the iteration count relative to plain gradient descent on a narrow valley.

5.8 Trust Region - `minimize(method='trust-ncg')`

Trust-region subproblem

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \boldsymbol{s}^{(k)}, \qquad \boldsymbol{s}^{(k)} = \underset{\boldsymbol{s}\in\mathbb{R}^d:\ \|\boldsymbol{s}\|\le\Delta_k}{\arg\min} \ m_k(\boldsymbol{s})$$

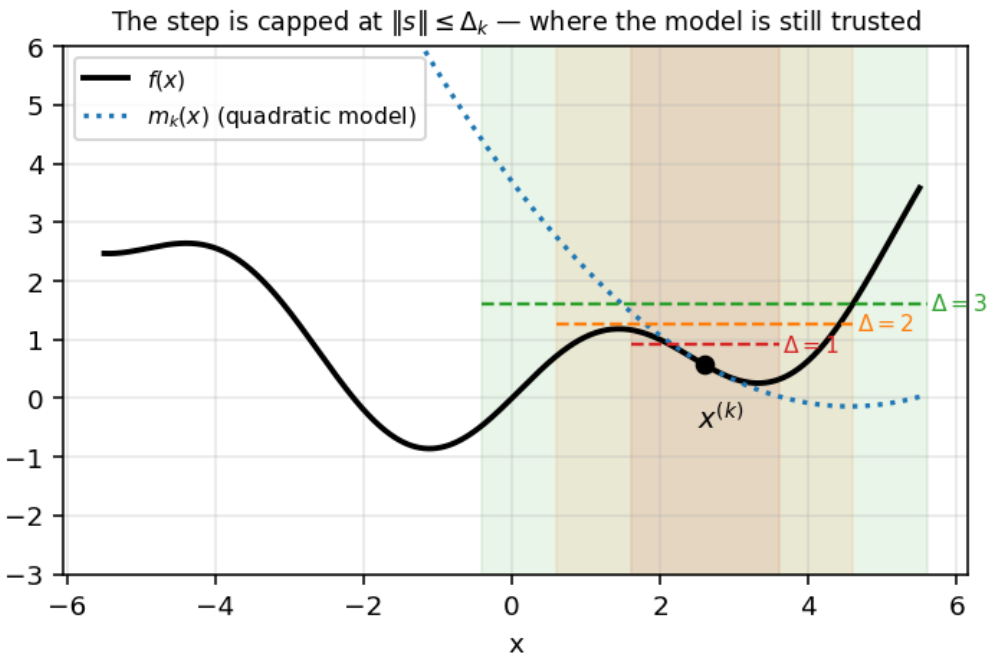
Linear approximation:

$$m_k(\boldsymbol{x}) = f(\boldsymbol{x}^{(k)}) + \left(\boldsymbol{x} - \boldsymbol{x}^{(k)}\right)^T \nabla f(\boldsymbol{x}^{(k)}) \quad \text{or} \quad m_k(\boldsymbol{s}) = f(\boldsymbol{x}^{(k)}) + \boldsymbol{s}^T \nabla f(\boldsymbol{x}^{(k)})$$

Quadratic approximation *(it is also possible to choose a symmetric matrix for $\boldsymbol{H}$)*:

$$m_k(\boldsymbol{x}) = f(\boldsymbol{x}^{(k)}) + (\boldsymbol{x} - \boldsymbol{x}^{(k)})^T \nabla f(\boldsymbol{x}^{(k)}) + \tfrac{1}{2}\left(\boldsymbol{x} - \boldsymbol{x}^{(k)}\right)^T \boldsymbol{H}^{(k)} (\boldsymbol{x} - \boldsymbol{x}^{(k)})$$

$$m_k(\boldsymbol{s}) = f(\boldsymbol{x}^{(k)}) + \boldsymbol{s}^T \nabla f(\boldsymbol{x}^{(k)}) + \tfrac{1}{2}\boldsymbol{s}^T \boldsymbol{H}^{(k)} \boldsymbol{s}$$



the dotted quadratic model $m_k(x)$ hugs $f(x)$ near $x^{(k)}$ and diverges from it further out. The concentric radii are the candidate trust regions - the step is confined to the one the model has earned.

Python code - Trust-Region Newton-Conjugate-Gradient

```
res = minimize(rosen, x0, method='trust-ncg', jac=rosen_der,
               hess=rosen_hess, options={'gtol': 1e-8, 'disp': True})
```

(with the same `rosen`, `rosen_der`, `rosen_hess` and `x0 = [1.3, 0.7, 0.8, 1.9, 1.2]` as above)

Example output

```
nit   = 20      nfev = 21      njev = 20      nhev = 19
fun   = 1.23e-30
x     = array([1., 1., 1., 1., 1.])
```

Trust region wins on this problem

Method	Iterations	Function evals	Final f
Newton-CG	24	33	3.5×10^{-17}
BFGS	25	30	4.0×10^{-13}
trust-ncg	**20**	**21**	$\mathbf{1.2 \times 10^{-30}}$

Fewest iterations, fewest function evaluations, and an answer **13 orders of magnitude** more accurate. A trust region limits the step to a region where the local model is sufficiently reliable: capping the step at Δ_k stops the model being trusted where it is wrong, and poor trial steps can be rejected while the radius is reduced.

5.9 First-Order Methods in PyTorch

Why a deep learning library

Deep learning libraries that provide automatic differentiation capabilities:

- **PyTorch**
- **TensorFlow**

The Newton-CG, BFGS, and trust-region examples require you to **hand-write `rosen_der` and `rosen_hess`** - which introduces opportunities for derivative errors. With automatic differentiation you write only the function; automatic differentiation computes the derivative of the implemented operations, up to floating-point error.

Python code - Adam via `torch.optim`

```python
import torch
import torch.optim as optim
import numpy as np

# Define the function (to be optimized)
def function_to_optimize(x):
    return (x - 2)**2 + 3

x = torch.tensor([0.0], requires_grad=True)
# Adam optimizer
optimizer = optim.Adam([x], lr=0.01)
max_steps = 1500
history = []
# Stopping criterion
tolerance = 1e-6              # Change in function value threshold for stopping
prev_output = float('inf')    # Initialize previous function value
# Optimization loop
for step in range(max_steps):
    # Zero the gradients
    optimizer.zero_grad()
    # Compute the value of the function
    output = function_to_optimize(x)
    # Backpropagation to compute gradients
    output.backward()
    # Update the variable using the optimizer
    optimizer.step()
    history.append((x.item(), output.item()))
    if abs(output.item() - prev_output) < tolerance:
        print(f'Stopping criterion met after {step+1} steps.')
        break
    # Update previous function value
    prev_output = output.item()
print(f'Optimized value of x: {x.item()}')
```

PyTorch optimization cycle

Line	What it does
`requires_grad=True`	Marks x as something to differentiate with respect to
`optimizer.zero_grad()`	Clears old gradients - **PyTorch accumulates them by default**, so omitting this silently corrupts every step
`output.backward()`	Automatic differentiation: computes $\partial f/\partial x$ and stores it in `x.grad`
`optimizer.step()`	Applies Adam's update using the computed gradient

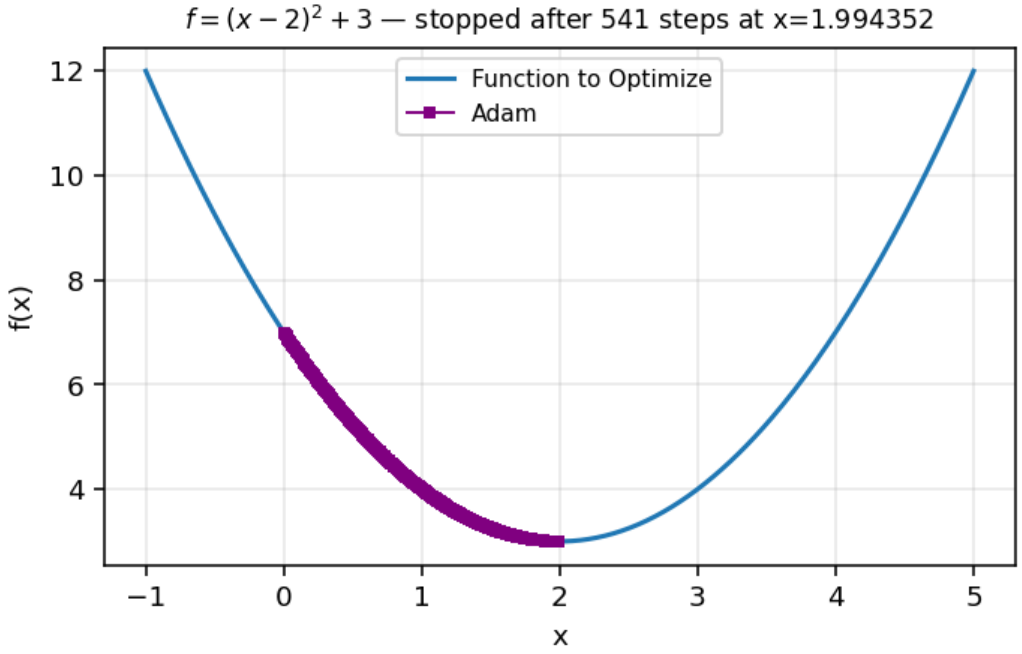
Swapping `optim.Adam` for `optim.SGD`, `optim.RMSprop`, `optim.Adagrad` or `optim.Adadelta` changes the algorithm and **nothing else**. Each optimizer still needs its own suitable hyperparameters.

5.9.1 A convex example

$f(x) = (x-2)^2 + 3$

Starting from $x = 0$, with `lr=0.01` and `tolerance=1e-6`:

```
Stopping criterion met after 541 steps
Optimized value of x: 1.9943516254425049
```



Why 541 steps for such an easy problem, and why not exactly 2?

Two things are visible here:

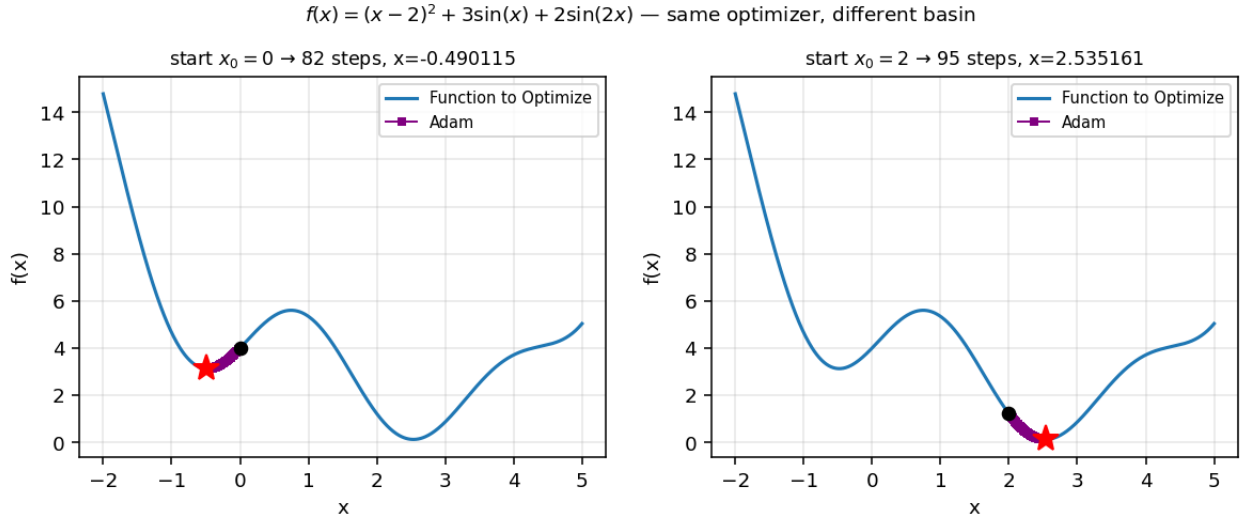
- Adam's default step is roughly `lr` per iteration regardless of gradient magnitude, so travelling a distance of 2 at lr $= 0.01$ **needs hundreds of steps by construction**.
- The run stops at 1.9944, not 2, because the termination test is on the **change in function value** ($< 10^{-6}$), not on the gradient or the position. Near a flat minimum f stops changing before x finishes moving. A loose termination criterion, not the optimizer, is what limits the accuracy here. This illustrates how the stopping criterion can limit positional accuracy.

5.9.2 A multimodal example

$f(x) = (x-2)^2 + 3\sin(x) + 2\sin(2x)$

Same optimizer, same settings, **two different starting points**:

Starting value	Steps	Optimized x	Which minimum
$x_0 = 0$	82	-0.49011531472206116	A **local** minimum ($f \approx 3.1$)
$x_0 = 2$	95	2.535160541534424	The **global** minimum ($f \approx 0.1$)



Dependence on the initial point

The function is **non-convex**, so a local-convergence guarantee applies: a local descent method returns whichever basin you started in, and gives you no information about how far that is from the global optimum. Adam is not at fault - it converged correctly both times. Starting at $x_0 = 0$, the nearest downhill path leads into the shallow left valley, and no gradient-based method can see over the hill at $x \approx 0.8$.

Use **random restarts** to test multiple attraction basins: run the method from several random initial points and keep the best result.

5.10 Quick Reference - Theory to Code

Optimization methods and Python interfaces

Method	Python	Requires
Bracketing algorithm	`scipy.optimize.bracket(f, xa, xb)`	f only
Bisection on f'	`scipy.optimize.bisect(f, a, b)`	f, a sign-changing bracket
Approximate line search (Wolfe)	`scipy.optimize.line_search(f, g, x, d)`	$f, \nabla f$
Secant method	`scipy.optimize.newton(f, x0)`	f only
Newton's method (1-D)	`scipy.optimize.newton(f, x0, fprime=...)`	f, f'
Newton's method (multivariate)	`minimize(..., method='Newton-CG', jac, hess)`	$f, \nabla f, \boldsymbol{H}$
Newton with Hessian-vector products	`minimize(..., method='Newton-CG', jac, hessp)`	$f, \nabla f, \boldsymbol{Hp}$
Quasi-Newton (BFGS)	`minimize(..., method='BFGS', jac)`	$f, \nabla f$
Trust region	`minimize(..., method='trust-ncg', jac, hess)`	$f, \nabla f, \boldsymbol{H}$
Gradient descent, momentum	`torch.optim.SGD(params, lr, momentum)`	f only (autodiff)
Adagrad / RMSProp / Adadelta / Adam	`torch.optim.Adagrad` / `.RMSprop` / `.Adadelta` / `.Adam`	f only (autodiff)

Choosing in practice

1. **Can you write the Hessian cheaply?** → `trust-ncg`, or `Newton-CG` with `hessp` if n is large.
2. **Gradient only?** → `BFGS` (or `L-BFGS-B` for very large n).
3. **Differentiable objective expressed with PyTorch operations?** → use automatic differentiation with a first-order optimizer. A black-box objective without derivative support needs a derivative-free method.
4. **Non-convex?** → whichever of the above, **plus random restarts**.

5.11 Key Takeaways

Summary

`scipy.optimize` implements classical scalar and multivariate methods - `bracket`, `bisect`, `line_search`, `newton`, and `minimize` with `Newton-CG` / `BFGS` / `trust-ncg` - while PyTorch supplies the adaptive first-order family with **automatic differentiation** so no derivative is ever hand-coded. On the 5-D Rosenbrock, **trust-ncg wins** (20 iterations, $f \approx 10^{-30}$); on a non-convex function, **the starting point decides the answer**, whatever the optimizer.

Self-check

1. What seven values does `scipy.optimize.bracket` return, and which of the three abscissae has the lowest function value?
2. Why does 0.61803 appear in `bracket`'s output?
3. What happens if you call `bisect(lambda x: x**2-1, -2, 2)`? Why?
4. How do you use a root-finder like `bisect` to *minimize* a function?
5. In the line-search example, verify 6.13, 1.13 and $[1.6, 1.4]$ by hand.
6. `line_search` returned $\alpha = 1.0$ but the slice is minimized at $\alpha = 1.75$. Is this a bug? Explain.
7. What does `scipy.optimize.newton` do differently when `fprime` is omitted, and what is the cost?
8. Write the Rosenbrock function for general n. Where is its minimum?
9. What is the difference between passing `hess` and passing `hessp`, and when does it matter?
10. BFGS was given no Hessian. How did it still match Newton-CG's iteration count, and what does `hess_inv` contain?
11. Compare Newton-CG, BFGS and trust-ncg on the 5-D Rosenbrock across iterations, function evaluations, and final accuracy.
12. What do `zero_grad()`, `backward()` and `step()` each do, and what breaks if you omit the first?
13. Adam took **541** steps on $(x-2)^2 + 3$ and stopped at 1.9944. Give both reasons.
14. The same Adam run reached -0.490 from $x_0 = 0$ and 2.535 from $x_0 = 2$. Which is the global minimum, and what does this say about local descent methods?
15. Which solver would you choose if you have f and ∇f but no Hessian, and $n = 10^6$?

CHAPTER 6

Stochastic Methods for Unconstrained Optimization

Overview

Deterministic descent selects steps from a fixed set of rules and can remain in one attraction basin. Stochastic methods introduce randomness into the iterate, the gradient estimate, or a population of candidate points to support exploration.

- **Recap:** why exact line search forces $\boldsymbol{g}^{(k+1)} \perp \boldsymbol{d}^{(k)}$, and why that means the algorithm *must* pick a new direction.
- **Noisy descent** - add Gaussian noise to each step so the iterate can drift off a saddle point.
- **Stochastic gradient descent** - replace the true gradient with a cheap, unbiased, noisy estimate computed on a mini-batch.
- **Information theory** - self-information, entropy, conditional entropy, mutual information, and the **KL divergence**, which relates cross-entropy to distribution fitting.
- **The cross-entropy (CE) method** - maintain a *probability distribution over the design space*, sample from it, keep the **elite samples**, refit the distribution, repeat.

Core idea: deterministic local descent exploits local information; randomness provides one mechanism for exploration, and every method in this chapter is a different way of spending it.

Interpreting stochastic results

A random seed fixes the pseudorandom sequence used by an implementation. Different seeds can produce different trajectories, final points, and evaluation counts. A stochastic method should be assessed over repeated independent runs, with the sampling distribution, sample budget, and stopping criterion stated.

6.1 Recap - Gradient Methods and the Orthogonality Property

The update rule

Iteratively update the design point $\boldsymbol{x}$ by taking steps **opposite to the gradient direction**:

$$\boldsymbol{x}^{(k+1)} \leftarrow \boldsymbol{x}^{(k)} + \alpha^{(k)}\boldsymbol{d}^{(k)}$$

Symbol	Meaning
$\alpha^{(k)}$	Step size / learning rate
$\boldsymbol{d}^{(k)} = -\dfrac{\boldsymbol{g}^{(k)}}{\|\boldsymbol{g}^{(k)}\|}$	Normalised **descent direction**
$\boldsymbol{g}^{(k)} = \nabla f(\boldsymbol{x}^{(k)})$	Gradient - the **steepest-ascent** direction

Exact line search chooses the best step size along that direction:

$$\alpha^* = \arg\min_{\alpha} f\left(\boldsymbol{x}^{(k)} + \alpha\boldsymbol{d}^{(k)}\right)$$

The orthogonality result

If $\alpha^{(k)}$ is chosen by **exact** line search, then

$$\boxed{\boldsymbol{g}^{(k+1)} \perp \boldsymbol{d}^{(k)} \quad \Longleftrightarrow \quad \boldsymbol{g}^{(k+1)T}\boldsymbol{d}^{(k)} = 0}$$

and since $\boldsymbol{d}^{(k)} \parallel -\boldsymbol{g}^{(k)}$, consecutive **gradients** are orthogonal too: $\boldsymbol{g}^{(k+1)} \perp \boldsymbol{g}^{(k)}$.

Orthogonality under exact line search

Start from the definition of the optimal step,

$$\alpha^{(k)} = \arg\min_{\alpha} f(\boldsymbol{x}^{(k)} + \alpha\boldsymbol{d}^{(k)}),$$

so at the optimum the derivative with respect to α vanishes:

$$\frac{df(\boldsymbol{x}^{(k)} + \alpha\boldsymbol{d}^{(k)})}{d\alpha^{(k)}} = \boldsymbol{g}(\boldsymbol{x}^{(k)} + \alpha^{(k)}\boldsymbol{d}^{(k)})^T\boldsymbol{d}^{(k)}$$

$$0 = \boldsymbol{g}(\boldsymbol{x}^{(k)} + \alpha^*\boldsymbol{d}^{(k)})^T\boldsymbol{d}^{(k)} \;=\; (\boldsymbol{d}^{(k+1)})^T\boldsymbol{d}^{(k)}$$

Where the chain rule comes in - expanding the total derivative over all N coordinates:

$$\frac{df(\boldsymbol{x}^{(k)} + \alpha\boldsymbol{d}^{(k)})}{d\alpha^{(k)}} = \sum_{i=1}^{N} \frac{\partial f(\boldsymbol{x}^{(k)} + \alpha\boldsymbol{d}^{(k)})}{\partial x_i} \frac{d\left(x_i^{(k)} + \alpha d_i^{(k)}\right)}{d\alpha} = \sum_{i=1}^{N} g_i(\boldsymbol{x}^{(k)} + \alpha\boldsymbol{d}^{(k)})\, d_i^{(k)}$$

$$= \boldsymbol{g}(\boldsymbol{x}^{(k)} + \alpha^*\boldsymbol{d}^{(k)})^T\boldsymbol{d}^{(k)}$$

The inner derivative $d(x_i + \alpha d_i)/d\alpha = d_i$ is what turns the gradient into a **directional derivative**.

Geometric consequence of exact line search

At $\boldsymbol{x}^{(k+1)}$ there is **no further improvement along the current search direction** - you have squeezed that line dry. The remaining descent must be perpendicular to it, so **the algorithm is forced to choose a new direction**. This is exactly the zig-zag you see on elongated contours.

And if the step is *not* optimal?

Suppose $\alpha^{(k)} \neq \arg\min_\alpha f(\boldsymbol{x}^{(k)} + \alpha \boldsymbol{d}^{(k)})$. Then generally

$$\boldsymbol{g}^{(k+1)T} \boldsymbol{d}^{(k)} \neq 0 \qquad \Longrightarrow \qquad \boldsymbol{g}^{(k+1)} \not\perp \boldsymbol{g}^{(k)}$$

The new gradient **still has a component along the previous search direction**, so further improvement may still be available in the *same* direction. Orthogonality is a *symptom* of exact line search, not a property of gradient descent in general.

6.2 Why Stochastic Methods? Exploration vs Exploitation

The limitation of purely deterministic descent

- Deterministic descent methods **exploit gradient information** to choose a promising search direction.
- Exact line search finds the best step along that direction: $\alpha^* = \arg\min_\alpha f(\boldsymbol{x} + \alpha \boldsymbol{d})$.
- This exploits local information. On a non-convex objective, a local method can converge to a local minimum without establishing whether a better minimum exists elsewhere.

What randomness buys you

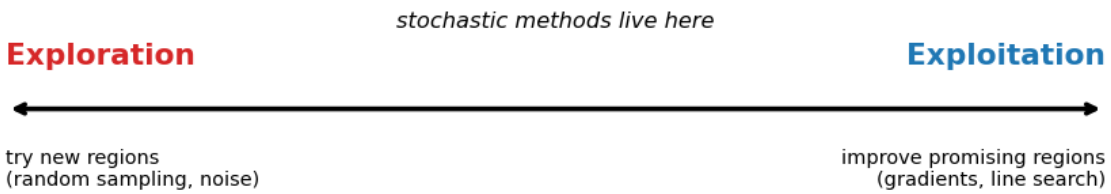
Stochastic methods introduce **randomness** to explore other regions of the design space. Randomness can help the algorithm:

1. **escape local optima**,
2. **explore new search directions**, and
3. **increase the chance of finding better solutions** (a higher likelihood of reaching the global optimum).

But too much randomness is also bad

Too much randomness:

- reduces the use of gradient/history information,
- makes the search **less directed**, and
- **slows convergence**.



Exploration and exploitation

Too Much Exploration	Too Much Exploitation
Too random	Too greedy
Ignores useful information	May get stuck in a local optimum
Slow convergence	Poor global search

Three stochastic search strategies

#	Method	Where the randomness lives
1	**Noisy descent**	Added to the *iterate* - an explicit noise term on each step
2	**Stochastic gradient**	In the *gradient estimate* - a random mini-batch of data
3	**Cross-entropy method**	In the *whole search* - a probability distribution over the design space, no gradient at all

6.3 Noisy Descent

The problem it solves

- At a **saddle point** the gradient is zero, so a pure descent method has nothing to move on.
- Near such flat regions, descent methods select **very small step sizes**, producing extremely slow progress.
- One strategy: **add Gaussian noise at each descent step**.

Noisy descent update

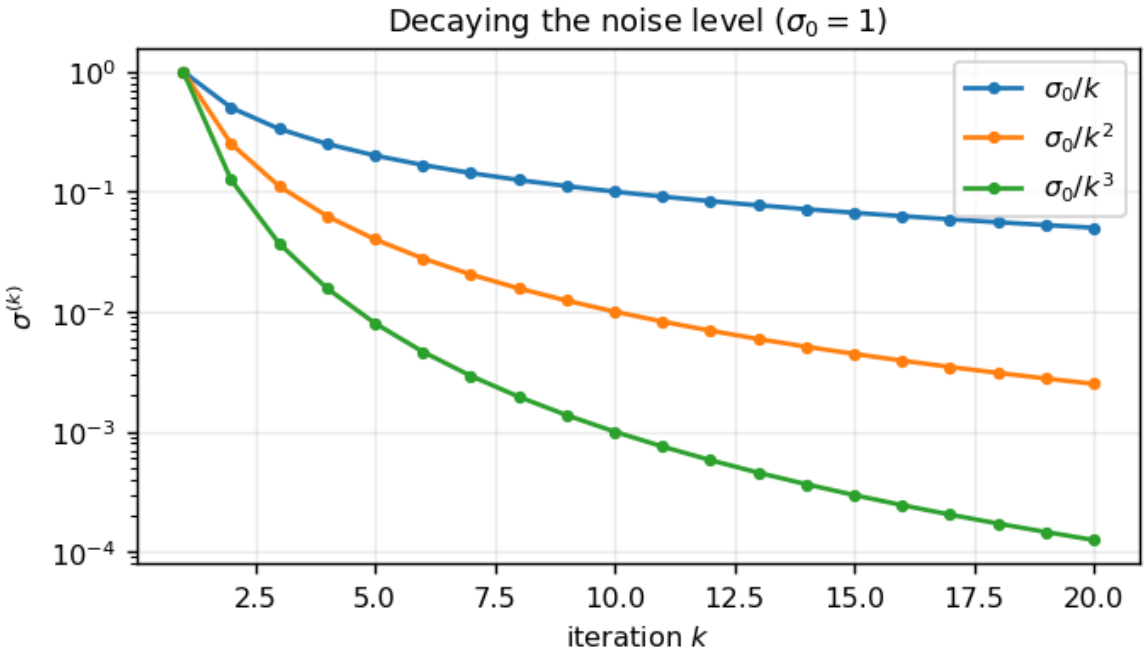
$$\boxed{\boldsymbol{x}^{(k+1)} \leftarrow \boldsymbol{x}^{(k)} - \alpha^{(k)}\boldsymbol{g}^{(k)} + \boldsymbol{\epsilon}^{(k)}}$$

where $\boldsymbol{\epsilon}^{(k)}$ is **zero-mean Gaussian noise** with standard deviation σ:

$$\boldsymbol{\epsilon}^{(k)} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \boldsymbol{I})$$

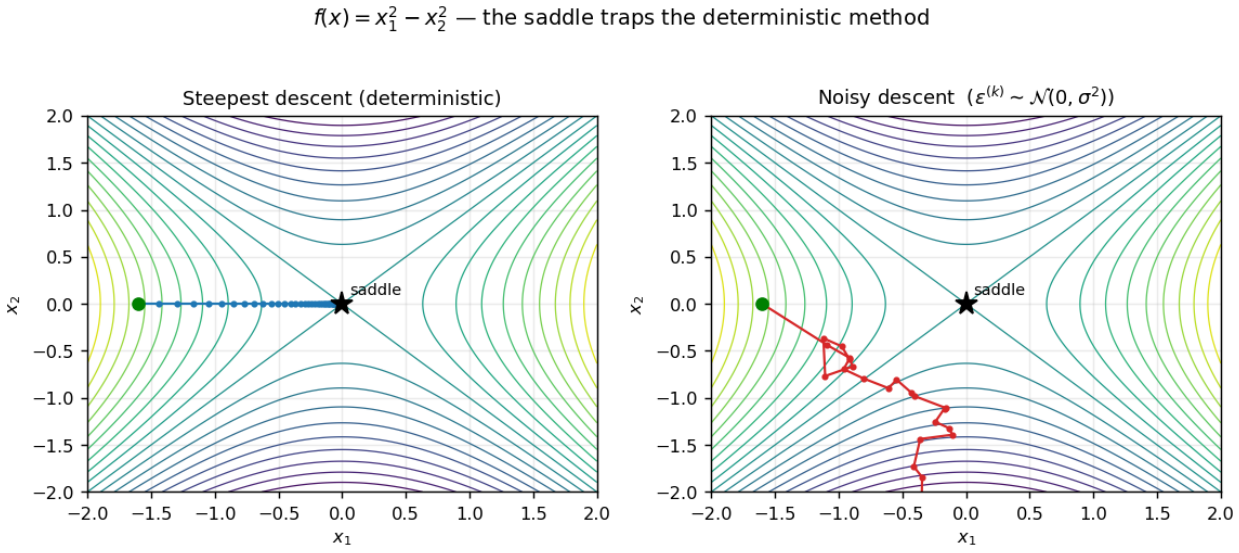
The noise must decay

Noise levels generally **decrease gradually over time** - e.g. σ is reduced by a factor of $1/k^3$, where k is the iteration number.
The reason is the exploration/exploitation trade-off *in time*: early on you want to move around freely; late on you want to settle. If σ stayed constant the iterate would rattle around the optimum forever and never converge.



Three decay schedules on a log axis. $1/k^3$ is aggressive - by $k = 10$ the noise is a thousandth of its initial level, so noisy descent behaves like plain gradient descent after a short exploratory phase.

6.3.1 Worked demonstration: the saddle $f(\boldsymbol{x}) = x_1^2 - x_2^2$



Example - both methods start at $(-1.6,\ 0)$, $\alpha = 0.05$, 45 iterations

Method	Final iterate	Outcome
Steepest descent	$(-0.014,\ 0.000)$	**Converged onto the saddle** and stopped
Noisy descent, $\sigma_0 = 0.18$ decaying	$(0.125,\ -19.57)$	**Escaped** along the $-x_2$ direction and kept descending

The deterministic run starts on the line $x_2 = 0$, where $\partial f/\partial x_2 = -2x_2 = 0$ **exactly**. There is no force

pushing it off that line, so it approaches the saddle along that line and satisfies $\nabla f = \mathbf{0}$ - a perfectly valid stationary point that is not a minimum at all.

Why the deterministic method is *stuck*, not *slow*

This is not a step-size problem. The saddle sits on the descent path and the gradient there is genuinely zero, so **no choice of** α rescues it. The only thing that can move the iterate off the invariant line $x_2 = 0$ is something that does not come from the gradient - which is precisely the $\boldsymbol{\epsilon}^{(k)}$ term.

Saddle-point optimality test

Recall the SONC/SOSC classification: at $(0, 0)$ the Hessian of $x_1^2 - x_2^2$ is $\begin{bmatrix} 2 & 0 \\ 0 & -2 \end{bmatrix}$ - **indefinite**, so the point is a saddle. First-order methods only test $\nabla f = \mathbf{0}$, and a saddle passes that test. Noise is a cheap substitute for the second-order information that would have detected it.

6.4 Stochastic Gradient Descent (SGD)

The idea

Stochastic gradient descent is the standard method for training neural networks. It uses a **noisy gradient approximation**: instead of computing the true gradient over *all* training data, a **randomly selected subset (a batch)** is used at each step.

- The main gain is **computational efficiency** compared with computing the true gradient over all data samples at every iteration.
- The batch gradient can be computed **in parallel**.
- **Convergence requirement:** step sizes must decrease *sufficiently* for convergence, **but not too rapidly**, or the iterate freezes far away from a minimum.

Terminology

This is sometimes called **mini-batch gradient descent**. In *strict* SGD, a **single** randomly selected data sample is used per step. In practice "SGD" almost always means the mini-batch version, and the batch size interpolates between the two extremes.

What makes a gradient estimate valid

SGD uses a gradient estimate $\boldsymbol{g}(\boldsymbol{x})$ such that

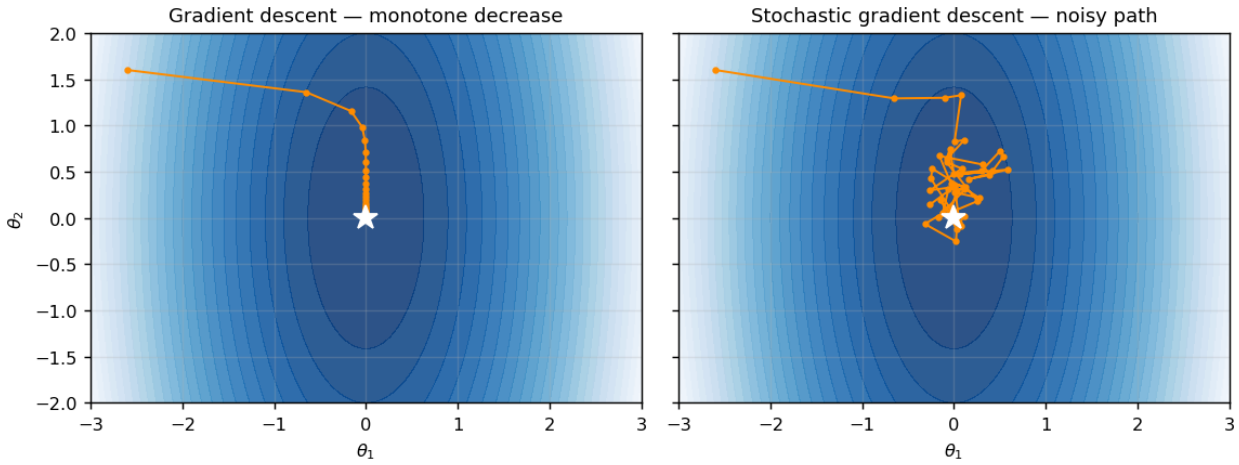
$$\boxed{\mathbb{E}[\boldsymbol{g}(\boldsymbol{x})] = \nabla f(\boldsymbol{x})}$$

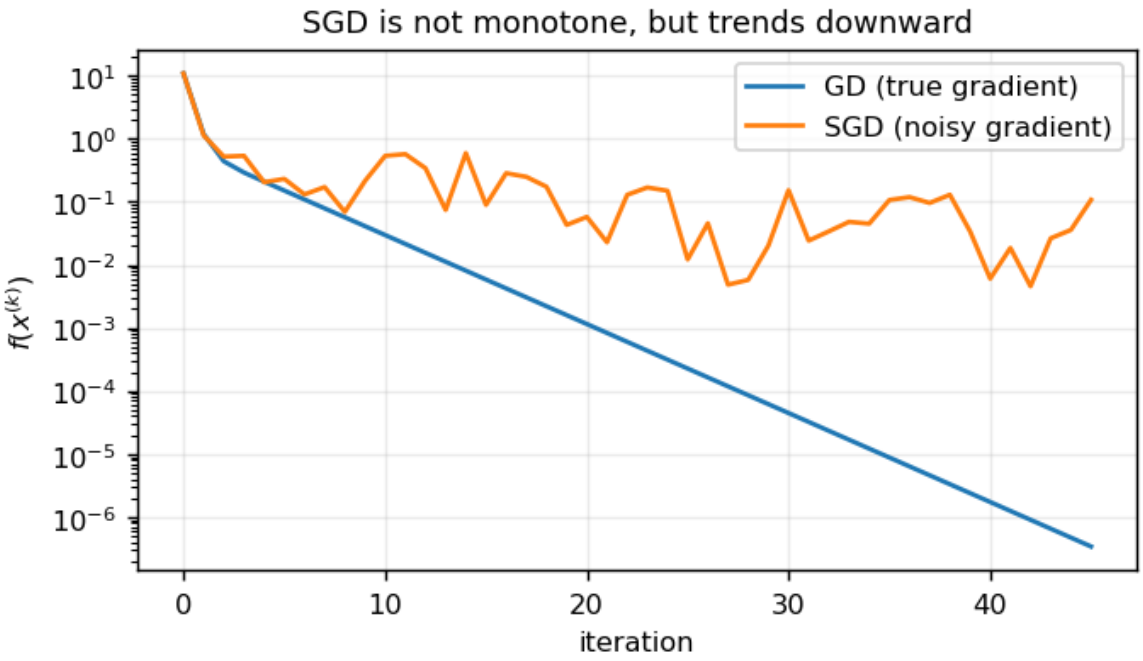
i.e. the estimate is **unbiased** - right *on average*, wrong at every individual step.
The other important factor is the **variance** of $\boldsymbol{g}(\boldsymbol{x})$: smaller variance is better, and increasing the batch size is the standard way to buy it (variance falls roughly as $1/|\text{batch}|$).

Two sampling policies

1. A data sample previously used is **not used again** (sampling without replacement - one *epoch* over shuffled data).
2. A data sample previously used **may be used again** (sampling with replacement - used in many toolboxes).

6.4.1 GD vs SGD on a quadratic



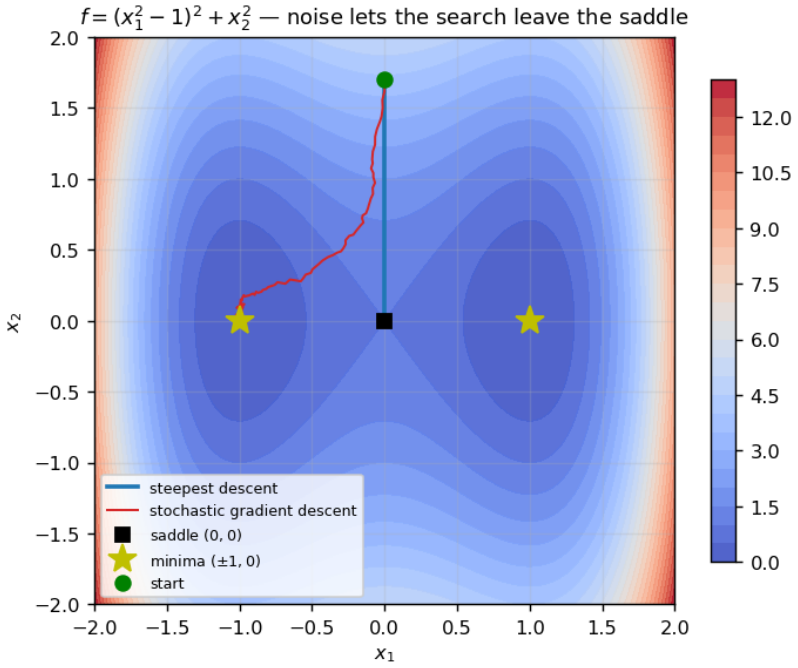


Full-gradient and stochastic-gradient trajectories

1. **No monotone decrease.** Unlike GD, there is **no guarantee that the objective decreases at each step** - the noisy curve goes up and down. But the *trend* is downward, and gradually the objective does decrease.
2. **SGD is more sensitive to the step size than GD.** With a fixed α, the iterate settles into a *noise ball* of radius roughly $\alpha\sigma$ around the optimum rather than converging to a point. Shrinking α

shrinks the ball - this is why decaying step sizes are mandatory.
3. "**Stochastic**" simply refers to *some randomness involved in the process* - in the *gradient*, not in the objective function itself.

6.4.2 The saddle again: $f(\boldsymbol{x}) = (x_1^2 - 1)^2 + x_2^2$



Example - both start at $(0,\ 1.7)$, $\alpha = 0.02$, 120 steps

Method	Final iterate	Which point
Steepest descent	$(0.000,\ 0.013)$	The **saddle** at $(0, 0)$
Stochastic gradient	$(-0.990,\ 0.028)$	The **minimum** at $(-1, 0)$

The function has two global minima at $\boldsymbol{x}^* = (\pm 1, 0)$ with $f = 0$, and a saddle at the origin with $f = 1$.

Escaping a stationary saddle

Because of its initialization, the steepest descent method converges to the saddle point, where the gradient equals zero. However, the noisy/stochastic approach is able to bypass the saddle point. The starting point lies exactly on the $x_1 = 0$ axis, which is invariant under the true gradient flow - so the deterministic method can never leave it, and the stochastic one leaves it almost immediately.

Design choices in stochastic gradient descent

1. What is the **step size**?
2. What is the size of the **mini-batch**?
3. **Constant** step size or **decreasing** step size?
4. Including **momentum**?

Momentum accumulates past gradients, while Adagrad, RMSProp, Adadelta, and Adam adapt the update using gradient history. These mechanisms address direction persistence and step-size selection;

their settings remain problem-dependent.

6.5 Interlude - Information Theory

Self-information, entropy, and KL divergence describe probability distributions and motivate fitting a sampling distribution to an elite set of candidate points.

6.5.1 What is information?

Two working definitions

- "Measure of knowledge or data that provides **meaning or context**"
- "Something that **reduces uncertainty** or increases knowledge"

The traffic-jam sentences

- **Sentence 1:** There is a traffic jam on E01 Southern Expressway.
- **Sentence 2:** There is a traffic jam on E01 Southern Expressway **near Kottawa Exit**.

Sentence 2 seems to have **more information** - it narrows down the possibilities.
Now compare a harder pair:

- **Sentence 1:** There is a traffic jam on E01 Southern Expressway near **Athurugiriya** Exit.
- **Sentence 2:** There is a traffic jam on E01 Southern Expressway near **Kottawa** Exit.

Which has more information? It is *not clear*. Both are equally specific. So - **how do we measure the amount of information?**

The cricket illustration

Match	Situation	Win probability	Outcome	Reaction
Australia vs Sri Lanka (target 240)	SL 111/8 after 26 overs	**5%**	SL won by 1 wicket	"**SL won… surprise!**"
Bermuda vs Sri Lanka (SL 321/6)	BMU 73/8 after 23 overs	**95%**	SL won by 243 runs	"**SL won… no surprise.**"

Same *event* ("SL won"), wildly different amounts of information. The information is in the **surprise**, and the surprise is set by the **probability**.

6.5.2 Self-information

Definition

Occurrence of a **low probability** event provides **more information** than a high-probability event. For event a_i with probability p_i:

$$I(a_i) = \log_b\left(\frac{1}{p_i}\right) = -\log_b(p_i)$$

Here b is the base; **base 2 is usual in information theory**, giving units of **bits**.

Logarithm of a reciprocal

The definition is $I(p) = \log_b(1/p) = -\log_b p$, for $0 < p \leq 1$ and $b > 1$. It is not $1/\log_b p$. For a certain event, $p = 1$, the information is zero; as p approaches zero, the information grows without bound.

Two anchor values

p_i	$I(a_i) = -\log_2 p_i$	Reading
1	$-\log_2(1) = 0$ bit	A certain event carries **no** information
0.5	$-\log_2(0.5) = 1$ bit	A fair coin flip carries exactly **1 bit**
0.05	≈ 4.32 bits	The Sri Lanka upset win
0.95	≈ 0.074 bits	The expected win - almost no information

Properties of self-information

1. $I(a_i) \geq 0$ for $0 \leq p(a_i) \leq 1$
2. If $p_i = 1$, then $I(a_i) = 0$
3. If $I(a_i) > I(a_j)$, then $p(a_i) < p(a_j)$ *(more information $\Leftrightarrow$ less likely)*

6.5.3 Entropy

First-order entropy of a source

Let random vector $\boldsymbol{x} = [x_1, \ldots, x_N]$ be the outcome of a random experiment. The **entropy** is the *expected* self-information:

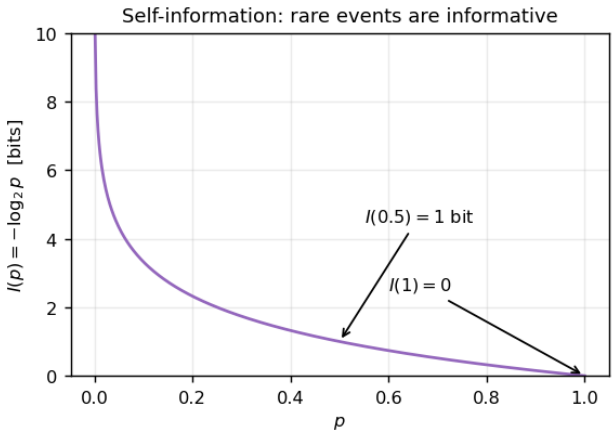
$$H = \sum_{i=1}^{N} p(x_i) \log_2\left(\frac{1}{p(x_i)}\right) = \sum_{i=1}^{N} -p(x_i) \log_2(p(x_i)) \text{ bits}$$

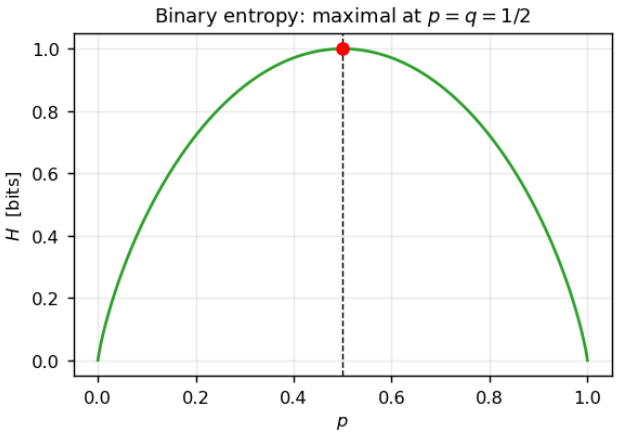
Binary Memoryless Source (BMS)

A BMS emits a stream such as `0 1 1 0 1 0 0 0` .... Let $p(x_i = 1) = p$ and $p(x_i = 0) = q = 1 - p$:

$$H = \sum_{i=1}^{N} -p(x_i) \log_2(p(x_i)) = -p \log_2(p) - q \log_2(q) = -p \log_2(p) - (1-p) \log_2(1-p)$$

The information is greatest when $p = q = 1/2$ - i.e. when the events are **equally probable**.





Bounds and interpretation

$$0 \leq H \leq \log_2 N$$

- Maximum entropy $H = \log_2 N$ is achieved when events are **equiprobable** ($p_i = 1/N$).
- Minimum entropy $H = 0$ when one outcome is certain.
- **Shannon entropy** "characterizes the average amount of bits needed to transmit a message $\boldsymbol{x}$ over a (binary) communication channel."

Introduced by **Claude Shannon** in *"A Mathematical Theory of Communication"* (1948), the paper that founded information theory.

6.5.4 Conditional entropy, joint entropy, mutual information

Conditional entropy

$$H(\boldsymbol{y}|\boldsymbol{x}) = -\sum p(\boldsymbol{x}, \boldsymbol{y}) \log\left(\frac{p(\boldsymbol{x}, \boldsymbol{y})}{p(\boldsymbol{x})}\right)$$

where $p(\boldsymbol{x}, \boldsymbol{y})$ is the **joint probability distribution**. Equivalently

$$H(\boldsymbol{y}|\boldsymbol{x}) = H(\boldsymbol{x}, \boldsymbol{y}) - H(\boldsymbol{x})$$

Meaning: the amount of uncertainty or randomness **remaining in $\boldsymbol{y}$ given that $\boldsymbol{x}$ is known**.

Chain rule

$$H(\boldsymbol{x}, \boldsymbol{y}) = H(\boldsymbol{x}) + H(\boldsymbol{y}|\boldsymbol{x}) = H(\boldsymbol{y}) + H(\boldsymbol{x}|\boldsymbol{y})$$

Here $H(\boldsymbol{x}, \boldsymbol{y})$ is the **cross entropy** (joint entropy) of $\boldsymbol{x}$ and $\boldsymbol{y}$.

Mutual information

$$M(\boldsymbol{x}, \boldsymbol{y}) = H(\boldsymbol{x}) + H(\boldsymbol{y}) - H(\boldsymbol{x}, \boldsymbol{y})$$

$$M(\boldsymbol{x}, \boldsymbol{y}) = H(\boldsymbol{x}) - H(\boldsymbol{x}|\boldsymbol{y}) = H(\boldsymbol{y}) - H(\boldsymbol{y}|\boldsymbol{x})$$

Meaning: the "**reduction of the uncertainty of one random variable due to the knowledge of the other**". If $\boldsymbol{x}$ and $\boldsymbol{y}$ are independent, $M = 0$.

6.5.5 Kullback–Leibler divergence and cross-entropy

Definition

The **Kullback–Leibler divergence** is a *statistical distance* measuring the difference between two probability distributions:

$$\boxed{D_{\mathsf{KL}} = \mathbb{E}_g\left(\log \frac{g(\boldsymbol{x})}{h(\boldsymbol{x})}\right) = \sum_i p_g(x_i) \log \frac{p_g(x_i)}{p_h(x_i)}}$$

and it can be written in terms of entropies as

$$D_{\mathsf{KL}}(g(\boldsymbol{x}) \| h(\boldsymbol{x})) = H(g, h) - H(g)$$

i.e. **cross-entropy minus entropy**.

Interpretation of KL divergence

- D_{KL} quantifies **how** $g(\boldsymbol{x})$ **differs from a reference distribution** $h(\boldsymbol{x})$.
- It is the **expected extra surprise** incurred when using $h(\boldsymbol{x})$ as a model for $g(\boldsymbol{x})$ - the **information lost** when approximating g with h.
- $D_{\mathsf{KL}} \geq 0$, with equality **iff** $g = h$.

KL divergence is not symmetric

$$D_{\mathsf{KL}}(g(\boldsymbol{x}) \| h(\boldsymbol{x})) \neq D_{\mathsf{KL}}(h(\boldsymbol{x}) \| g(\boldsymbol{x}))$$

It is therefore not a metric - there is no triangle inequality either. “Distance” here is loose language.

Worked example - the biased coin

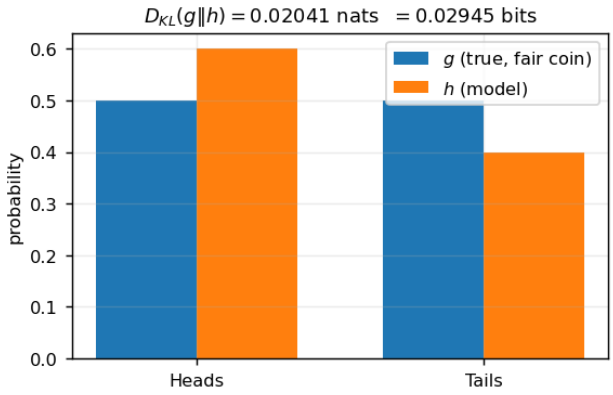
The **true** distribution g is a fair coin: $P(H) = P(T) = 0.5$. The **estimated** distribution h has $P(H) = 0.6$, $P(T) = 0.4$. Then

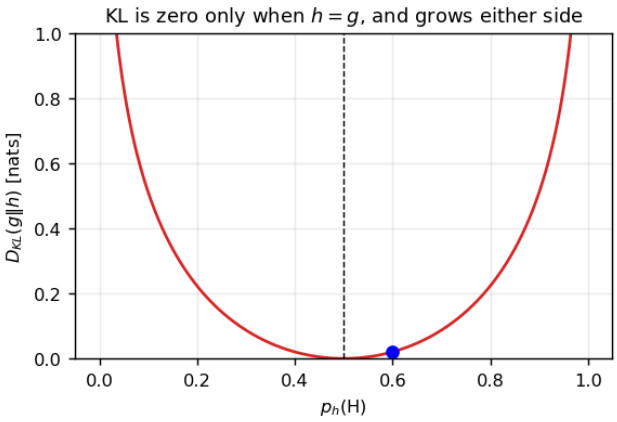
$$\begin{aligned} D_{\mathsf{KL}} &= p_g(H)\log(p_g(H)) - p_g(H)\log(p_h(H)) \\ &\quad + p_g(T)\log(p_g(T)) - p_g(T)\log(p_h(T)) \\ &= 0.5\log(0.5) - 0.5\log(0.6) + 0.5\log(0.5) - 0.5\log(0.4) \end{aligned}$$

Calculated values:

Quantity	Value (nats, natural log)	Value (bits, $\log_2$)
$D_{\mathsf{KL}}(g\|h)$	0.0204110	0.0294468
$D_{\mathsf{KL}}(h\|g)$	0.0201355	0.0290494

The two differ - small here, but **provably not equal**, which is the point of the asymmetry warning above.





6.6 The Cross-Entropy Method

Main idea

The **Cross-Entropy (CE) method** solves optimization problems by **sampling candidate solutions** and gradually modifying a probability distribution so that it **concentrates around good solutions**.

- The CE method maintains a **probability distribution over the design space**, known as the **proposal distribution**.
- The proposal distribution is used to suggest new samples for the subsequent iteration.
- Samples are drawn from it, and the distribution is **updated to fit the best samples**.
- The proposal distribution is selected from a **family of distributions**, often **multivariate normal**.
- Depending on the chosen family, fitting can be done **analytically**.

The method proceeds in **two phases**:

1. **Draw a sample** from the proposal distribution.
2. **Minimize the cross-entropy** between the proposal distribution and a target distribution to improve subsequent samples.

The loop

$$\text{Sample} \rightarrow \text{Evaluate} \rightarrow \text{Update} \rightarrow \text{Repeat}$$

1. Choose an initial parameter vector $\boldsymbol{v}^{(0)}$.
2. Generate random samples $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N \sim f\left(\boldsymbol{x}; \boldsymbol{v}^{(t-1)}\right)$.
3. Evaluate the quality of each sample with $H(\boldsymbol{x}_i)$.
4. Update the distribution parameters so future samples are more likely to be good.

6.6.1 The update rule

Updating the probability distribution

At iteration t, the new parameters are obtained by solving

$$\boldsymbol{v}^{(t)} = \arg\max_{\boldsymbol{v}} \frac{1}{N} \sum_{i=1}^{N} H(\boldsymbol{x}_i) \frac{f(\boldsymbol{x}_i; \boldsymbol{u})}{f(\boldsymbol{x}_i; \boldsymbol{v}^{(t-1)})} \log f(\boldsymbol{x}_i; \boldsymbol{v})$$

Term	Role
$\frac{1}{N}\sum_{i=1}^{N}$	**average over samples** - a Monte-Carlo estimate of an expectation
$H(\boldsymbol{x}_i)$	**quality** of sample $\boldsymbol{x}_i$ - gives greater importance to high-quality samples
$\frac{f(\boldsymbol{x}_i;\boldsymbol{u})}{f(\boldsymbol{x}_i;\boldsymbol{v}^{(t-1)})}$	**importance-sampling correction** - corrects for the fact that samples were generated from the *previous* distribution, not the target
$\log f(\boldsymbol{x}_i;\boldsymbol{v})$	**log-likelihood** - measures how well the new distribution explains the samples
$\arg\max_{\boldsymbol{v}}$	choose the new parameters $\boldsymbol{v}^{(t)}$ that maximize the objective

Here $f(\boldsymbol{x};\boldsymbol{u})$ is the **target distribution**, $f(\boldsymbol{x};\boldsymbol{v})$ the **proposal distribution** defined by parameter vector $\boldsymbol{v}$, and $\boldsymbol{u}$ an auxiliary/reference parameter used in the importance-sampling distribution.

Intuition

The update **shifts the probability distribution toward regions containing high-quality solutions**. Good samples have greater influence on the new sampling distribution.
The above optimization problem **can be formulated as minimizing the KL divergence** between $f(\boldsymbol{x};\boldsymbol{u})$ and $f(\boldsymbol{x};\boldsymbol{v})$ - which is where the method gets its name.

6.6.2 Elite samples and maximum likelihood

The indicator-function special case

A particularly useful choice is

$$H(\boldsymbol{x}) = \mathbb{I}_{\{\boldsymbol{x}\in A\}}, \qquad \mathbb{I}_{\{\boldsymbol{x}\in A\}} = \begin{cases} 1, & \boldsymbol{x}\in A, \\ 0, & \boldsymbol{x}\notin A, \end{cases}$$

where A is the set of **elite (good) solutions**.

Why this simplification matters

With H an indicator and $f(\boldsymbol{x}_i;\boldsymbol{u}) = f(\boldsymbol{x}_i;\boldsymbol{v}^{(t-1)})$, the importance ratio becomes 1 and the arg max collapses to **ordinary maximum likelihood estimation on the elite samples only**:

$$\text{Random samples} \rightarrow \text{Elite samples} \rightarrow \text{MLE update} \rightarrow \text{Concentrated distribution}$$

1. Generate many candidate solutions.
2. Select the best samples (**elite samples**).
3. Fit the probability distribution to these samples using **maximum likelihood estimation (MLE)**.
4. Repeat until convergence.

This is why the practical algorithm looks nothing like the intimidating arg max formula - it is just "keep the best 10, refit, repeat".

Cross-entropy search procedure

1. Choose an initial parameter vector $\boldsymbol{v}^{(0)}$ and set $t = 1$.
2. Generate random samples $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N \sim f(\cdot;\boldsymbol{v}^{(t-1)})$.
3. Solve for $\boldsymbol{v}^{(t)}$ using the update rule above.
4. If convergence is reached, stop. Otherwise increase t by 1 and return to Step 2.

Analytical solutions for Step 3 are available when:

- f belongs to the **natural exponential family**;
- f is **discrete with finite support**;
- $H(\boldsymbol{x}) = \mathbb{I}_{\{\boldsymbol{x}\in A\}}$ **and** $f(\boldsymbol{x}_i;\boldsymbol{u}) = f(\boldsymbol{x}_i;\boldsymbol{v}^{(t-1)})$.

6.6.3 A concrete parametrisation

Maximizing $S(x) = (1-x)^2$ with a Gaussian family

- Suppose the objective is to **maximize** $S(x) = (1-x)^2$.
- To use CE, determine $\mathbb{P}_{\boldsymbol{\theta}}(S(x) \geq \gamma)$ for a **threshold** γ and a parametric family $\{f(\cdot;\boldsymbol{\theta})\}$ - e.g. the 1-D Gaussian.
- The Gaussian is parameterized by its mean μ_t and variance σ_t^2, so $\boldsymbol{\theta} = (\mu, \sigma^2)$.
- The objective becomes: find $\boldsymbol{\theta}$ minimizing $D_{\mathsf{KL}}(I_{\{S(x)\geq\gamma\}} \| f_{\boldsymbol{\theta}})$ for a given γ.
- $I(\cdot)$ is the indicator function: $I_{\{S(x)\geq\gamma\}} = 1$ if $S(x) \geq \gamma$.
- Parameters are updated via **maximum likelihood estimation**.

γ **is the elite threshold in disguise** - "the best N_e of N samples" is the same as "all samples with $S(x) \geq \gamma$" for the appropriate γ.

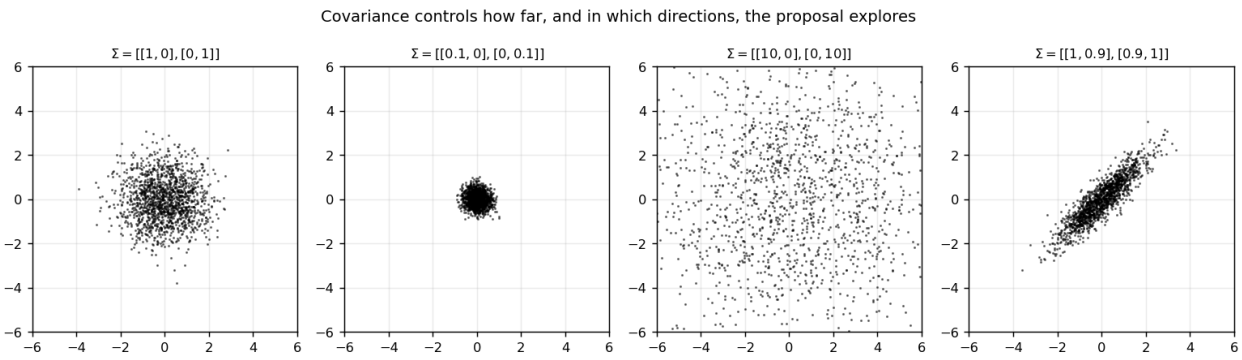
6.6.4 The multivariate normal proposal

Multivariate normal distribution

$$f(\boldsymbol{x}|\boldsymbol{\mu},\boldsymbol{\Sigma}) = \frac{1}{\sqrt{(2\pi)^k|\boldsymbol{\Sigma}|}} \exp\left(-\tfrac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right)$$

Symbol	Meaning
$\boldsymbol{x} = [x_1,\ldots,x_i,\ldots,x_k] \in \mathbb{R}^k$	k-dimensional random vector
$\boldsymbol{\mu} \in \mathbb{R}^k$	mean vector
$\boldsymbol{\Sigma} \in \mathbb{R}^{k\times k}$	covariance matrix (**positive definite**)
$\lvert\boldsymbol{\Sigma}\rvert$	determinant of the covariance matrix

A multivariate normal is fully characterized by $(\boldsymbol{\mu}, \boldsymbol{\Sigma})$. $\boldsymbol{x}$ is called a **standard normal random vector** if each component is independent and follows a standard normal distribution with zero mean and unit variance.



Covariance and sampling geometry

- **Diagonal entries** set the spread along each axis. $\boldsymbol{\Sigma} = 0.1\boldsymbol{I}$ gives a tight cloud (little exploration); $\boldsymbol{\Sigma} = 10\boldsymbol{I}$ gives a wide one (lots of exploration).
- **Off-diagonal entries** set the correlation. **When the off-diagonal elements are high, it implies that changes in one variable are strongly associated with changes in another variable** - the cloud stretches along a diagonal ridge. This is exactly what lets CE learn the *shape* of a valley, not just its location.

MLE update for the multivariate normal

$$\boxed{\boldsymbol{\mu}^{(k+1)} = \frac{1}{N_e}\sum_{i=1}^{N_e}\boldsymbol{x}_i}$$

$$\boxed{\boldsymbol{\Sigma}^{(k+1)} = \frac{1}{N_e}\sum_{i=1}^{N_e}(\boldsymbol{x}_i - \boldsymbol{\mu}^{(k+1)})(\boldsymbol{x}_i - \boldsymbol{\mu}^{(k+1)})^T}$$

Here N_e is the **number of elite samples** - those whose objective value is **greater than** γ for a maximization problem (**or** $\leq \gamma$ **for a minimization problem**).
That is: **sample mean and sample covariance of the elite set**. Nothing more.

6.6.5 Pseudocode

Cross-entropy search algorithm

```
Input : mu = mu_0, Sigma = Sigma_0, maxits = 100, N = 100, Ne = 10
Output: mu  (mean of final sampling distribution)

t ← 0;
while t < maxits and sigma_t^2 > eps_i do
    x     ← SampleMultivariateGaussian(mu, Sigma, N);
    S     ← f(x);
    Sort x by S in ascending order;      # descending for maximization
    mu    ← mean(x(1:Ne));
    Sigma ← cov(x(1:Ne));
    t     ← t + 1;
end
return mu;
```

One-dimensional Gaussian cross-entropy implementation

```
import numpy as np

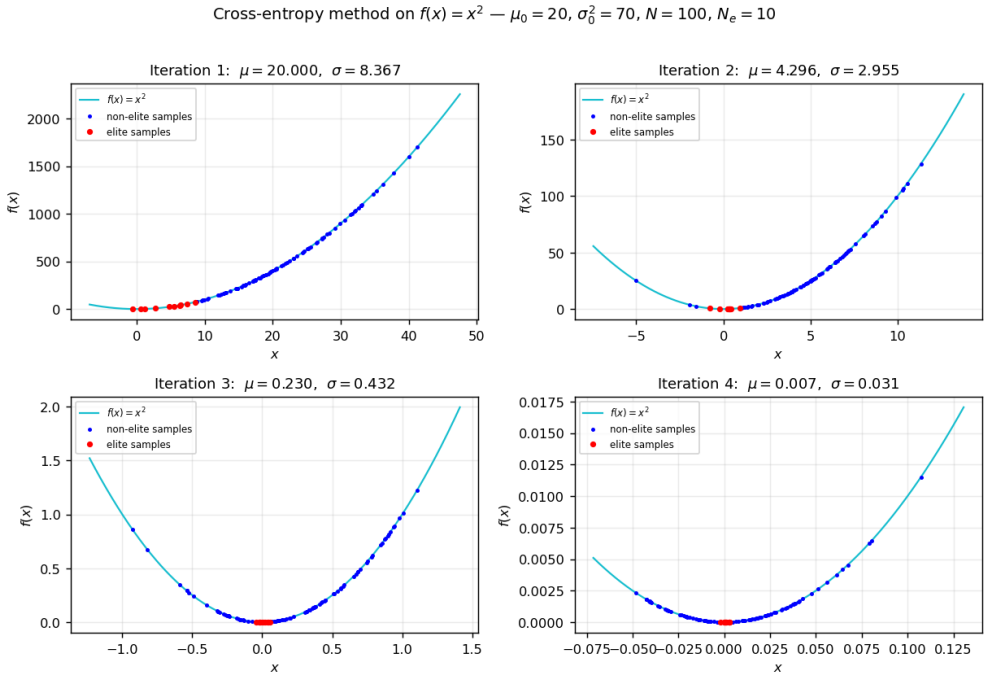
def cross_entropy_1d(f, mu, var, N=100, Ne=10, iters=5, seed=8):
    rng = np.random.default_rng(seed)
    sd = np.sqrt(var)
    for t in range(iters):
        S = rng.normal(mu, sd, N)          # 1. sample
        F = f(S)                           # 2. evaluate
        elite = S[np.argsort(F)[:Ne]]      # 3. keep the Ne best (minimization)
        mu, sd = elite.mean(), elite.std() # 4. MLE refit
    return mu, sd
```

```
mu, sd = cross_entropy_1d(lambda x: x**2, mu=20, var=70)
```

No gradient, no Hessian, no line search - only function evaluations. This is what makes CE applicable to black-box, discontinuous, and combinatorial problems.

6.7 Cross-Entropy in Action: $f(x) = x^2$

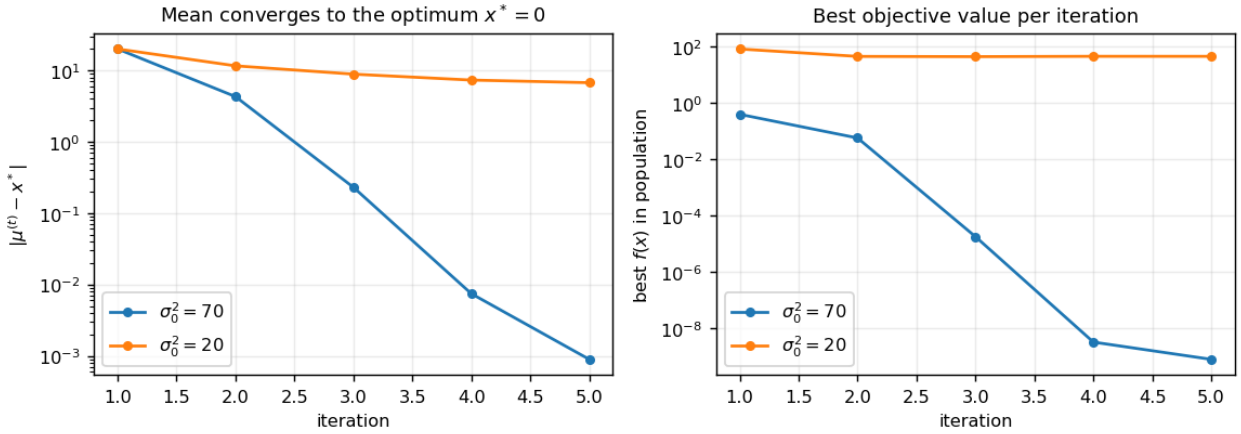
6.7.1 Good exploration: $\mu_0 = 20$, $\sigma_0^2 = 70$, $N = 100$, $N_e = 10$



Example - parameters at the start of each iteration (seed 8)

Iter.	μ	σ	best $f(x)$ in the population
1	20	8.3666	3.90×10^{-1}
2	4.2960	2.9547	5.66×10^{-2}
3	0.2298	0.4319	1.80×10^{-5}
4	0.00742	0.03088	3.31×10^{-9}
5	0.000891	0.001436	8.10×10^{-10}

Final answer: $\mu = -2.7 \times 10^{-6}$, against the true optimum $x^* = 0$ - five iterations, 500 function evaluations, no derivatives.



Evolution of the sampling distribution

1. **Iteration 1** - the population is still spread out; the elite samples (red) sit on the low-f shoulder of the parabola.
2. **Iteration 2** - elite samples move toward $x = 0$, dragging μ from 20 to 4.3.
3. **Iteration 3** - samples concentrate near the optimum; note the x-axis has rescaled from $[-8, 50]$ to $[-1.3, 1.5]$.
4. **Iteration 4** - very tight concentration around $x = 0$; axis range is now ± 0.1.
5. **Iteration 5** - convergence: $f(x) \to 0$.

Convergence is **superlinear-looking on the log plot** because σ shrinks multiplicatively each iteration - but this is a *sampling* effect, not the quadratic convergence of Newton's method.

6.7.2 Poor exploration: the same problem with $\sigma_0^2 = 20$

Example - $\mu_0 = 20$, $\sigma_0^2 = 20$, same seed

Iter.	μ	σ	best $f(x)$
1	20	4.4721	80.56
2	11.6059	1.5794	43.97
3	8.8783	0.8615	43.14
4	7.3288	0.3742	44.30
5	6.7382	0.03477	44.21

Final answer: $\mu = 6.68$ - nowhere near $x^* = 0$. The method has *converged*, confidently, to the wrong place.

Premature variance collapse

The variance of the elite set shrinks **faster than the mean can travel**. By iteration 5, $\sigma = 0.035$: every sample lands within ± 0.1 of 6.74, so no sample can ever be drawn near the optimum again and the search is permanently trapped - not by a local minimum, but by its own confidence.
Compare the two runs at iteration 2: $\sigma = 2.95$ (converges) versus $\sigma = 1.58$ (fails). That single number decides the outcome.

Influence of the initial sampling variance

- A **smaller** σ^2 leads to **less exploration** of the search space - generated samples are more concentrated around the current solution.
- A **larger** σ^2 encourages **more exploration**, potentially allowing the method to escape local optima and find better solutions.
- The choice of σ^2 affects the **robustness** of the CE method: too small makes it sensitive to initial conditions and noise; too large leads to **excessive exploration**.
- However, **after the first iteration the mean and variance are updated based on elite samples** - so σ_0^2 only sets the starting point, and the collapse above is generated by the algorithm itself.
- To address this, an **adaptive strategy to update the covariance matrix dynamically** during optimization can be used.

Standard fixes in practice

Fix	What it does
Variance injection / smoothing	Add a decaying constant to $\boldsymbol{\Sigma}$ each iteration: $\boldsymbol{\Sigma} \leftarrow \boldsymbol{\Sigma} + \beta_t \boldsymbol{I}$
Smoothed updates	$\boldsymbol{v}^{(t)} \leftarrow \alpha \boldsymbol{v}^{\text{new}} + (1-\alpha)\boldsymbol{v}^{(t-1)}$, slowing the collapse
Larger N / smaller elite fraction	More samples per iteration keeps the elite covariance better estimated
Variance floor	Clamp $\sigma \geq \sigma_{\min}$

6.8 Cross-Entropy on Multimodal Problems

6.8.1 The Branin function

Cross-entropy search on the Branin function

The standard Branin function is

$$f(x_1, x_2) = \left(x_2 - \frac{5.1x_1^2}{4\pi^2} + \frac{5x_1}{\pi} - 6\right)^2 + 10\left(1 - \frac{1}{8\pi}\right)\cos x_1 + 10.$$

On $[-5, 10] \times [0, 15]$, it has three global minimizers, each with objective approximately 0.397887. A Gaussian cross-entropy search samples 40 points per iteration, retains the 10 lowest objective values, and refits the mean and covariance. Candidate points are clipped to the domain; $0.05I$ is added to the covariance to retain some exploration.

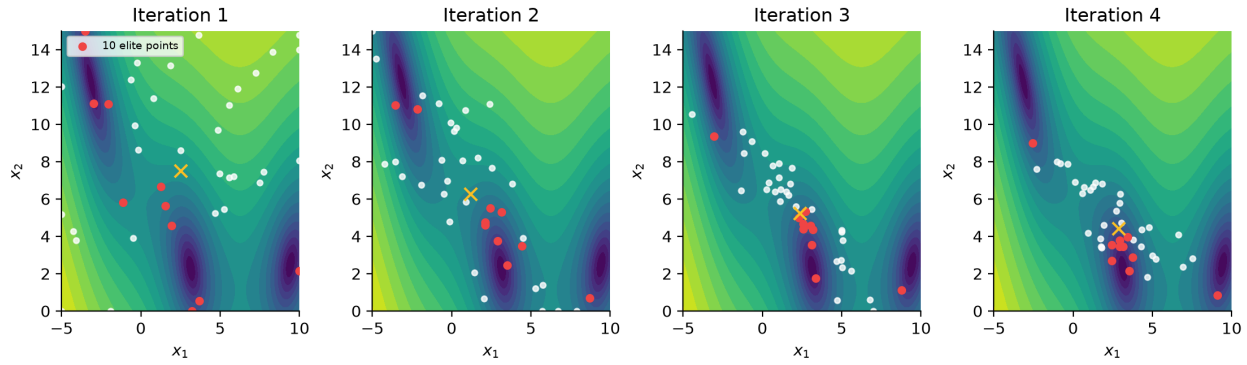


Figure 6.1: Four Gaussian cross-entropy iterations on the Branin function, using seed 12. White points are candidates, red points are elites, and a yellow cross marks the current sampling mean. Darker contours indicate smaller objective values.

6.8.2 The Rastrigin function

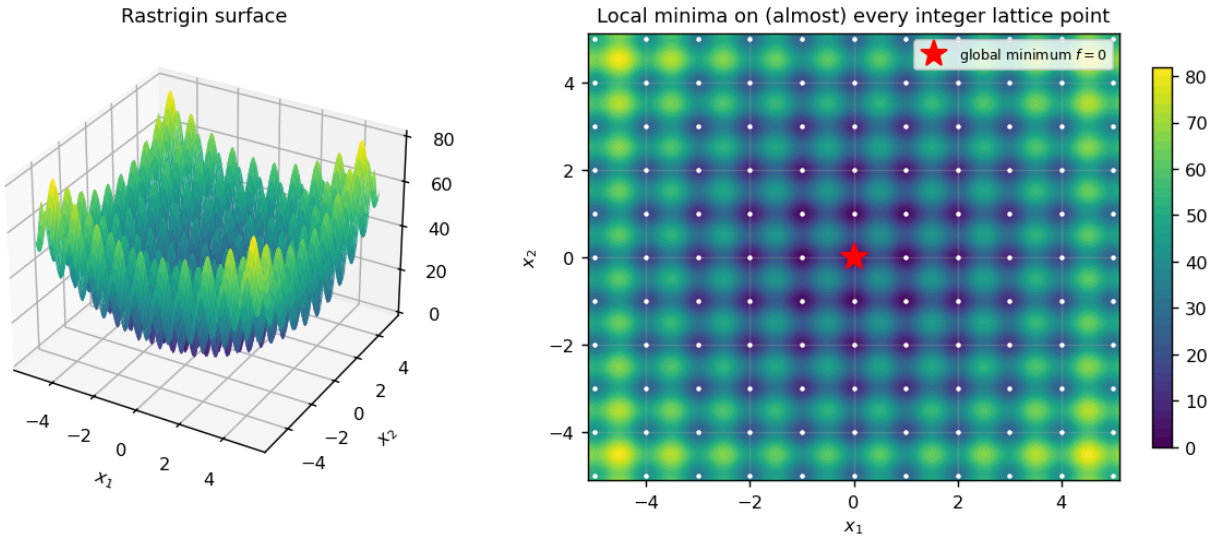
Definition

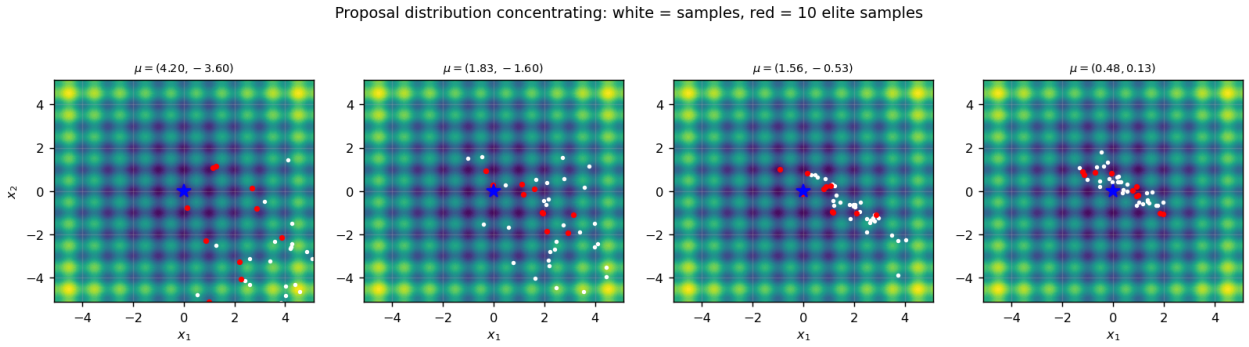
$$f(\boldsymbol{x}) = An + \sum_{i=1}^{n} \left[x_i^2 - A\cos(2\pi x_i)\right], \qquad A = 10$$

Global minimum: $\boldsymbol{x}^* = [0, 0]$ with $f(\boldsymbol{x}^*) = 0$.

Why it is the standard stress test

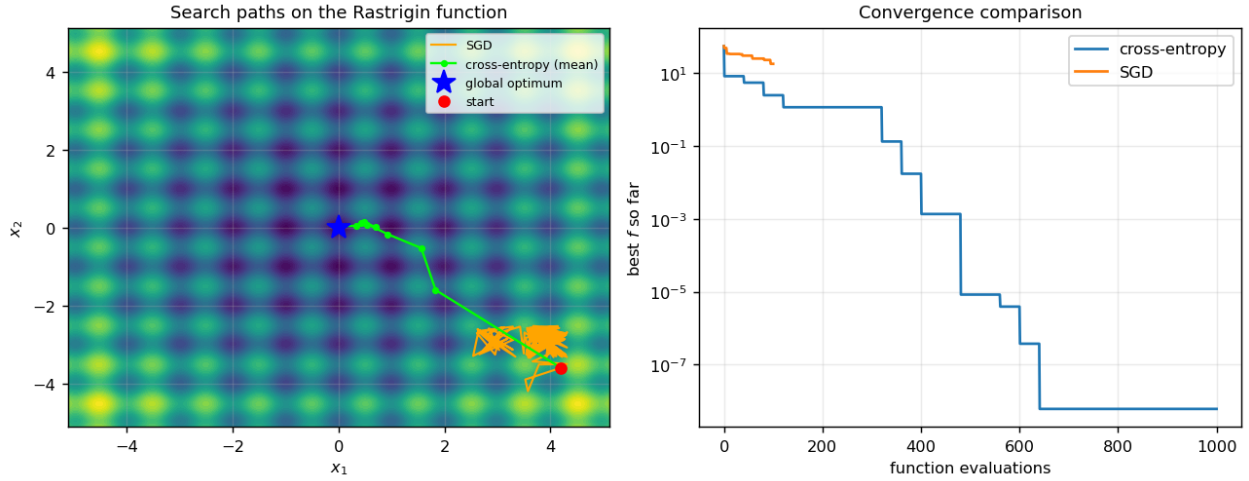
- **Highly multimodal** optimization function.
- **Contains many local minima** - roughly one per integer lattice point, so $11^2 = 121$ of them in $[-5.12, 5.12]^2$ alone, and 11^n in n dimensions.
- **Difficult for gradient-based optimization methods**: the quadratic bowl $\sum x_i^2$ gives the right global trend, but the $-A\cos(2\pi x_i)$ ripple dominates the *local* gradient everywhere.





Population search on Rastrigin: white candidate points and red elite points illustrate how repeated refitting concentrates the sampling distribution in a low-value basin. Concentration alone does not certify global optimality.

6.8.3 Cross-entropy vs SGD on Rastrigin



Example - both start at $x^{(0)} = (4.2,\ -3.6)$

Method	Budget	Final point	Best f found
SGD ($\alpha = 0.01$, gradient noise $\sigma = 1$)	100 iterations	$(3.10,\ -3.07)$	18.15
Cross-entropy ($N = 40$, $N_e = 10$)	25 iterations = 1000 evaluations	$(-6.8 \times 10^{-6},\ 9.0 \times 10^{-6})$	6.08×10^{-9}

SGD rattles around inside the local basin it started in and never leaves it. CE walks its mean across the landscape to the **global** optimum.

Trade-offs in population search

Exploration benefit: candidate samples can span several attraction basins, allowing the elite set to favor regions that a single local trajectory may not reach. Success depends on maintaining sufficient sampling variance; a successful run does not establish a global-convergence guarantee.
Loses: CE needs N function evaluations per iteration where gradient descent needs one gradient. On a smooth convex problem in high dimensions, a gradient-based method such as BFGS can use substantially fewer objective evaluations. Population methods spend objective evaluations on exploration;

gradient methods exploit derivative information for efficient local improvement.

6.9 Comparison of All Three Stochastic Methods

Randomness and computational cost

	Noisy descent	**Stochastic gradient (SGD)**	**Cross-entropy method**
Randomness lives in	The iterate: $+\boldsymbol{\epsilon}^{(k)}$	The gradient estimate (mini-batch)	The proposal distribution over the whole space
Needs ∇f	Yes	Yes (on a batch)	**No**
Cost per iteration	1 gradient	1 batch gradient (cheap)	N function evaluations
State carried	$\boldsymbol{x}^{(k)}$	$\boldsymbol{x}^{(k)}$	$(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ - a *distribution*
Escapes saddles	Yes	Yes	Not applicable (never uses ∇f)
Escapes distant local minima	Only if σ is large	Rarely	**Yes**, if $\boldsymbol{\Sigma}$ stays wide enough
Main tuning risk	σ decay too fast/slow	Step size, batch size	**Premature variance collapse**
Typical use	Escaping flat regions	Neural network training	Black-box, multimodal, low-n problems

Choosing in practice

1. **Smooth, convex, gradient available?** $\rightarrow$ deterministic gradient and curvature methods (BFGS, trust-ncg). Do not pay for randomness you do not need.
2. **Gradient available but the objective is a huge sum over data?** $\rightarrow$ **SGD** and its adaptive relatives.
3. **Gradient available but the path stalls at saddles / plateaus?** $\rightarrow$ **noisy descent**, or SGD's own noise.
4. **No gradient, or badly multimodal, and n is modest?** $\rightarrow$ **cross-entropy method** (or another population method).
5. **Any non-convex problem** $\rightarrow$ whichever of the above, **plus random restarts**.

6.10 Key Takeaways

Summary

Deterministic descent only **exploits** - exact line search even proves it, forcing $\boldsymbol{g}^{(k+1)} \perp \boldsymbol{d}^{(k)}$ so a new direction must be found each step, and leaving the method helpless at a saddle where $\nabla f = \mathbf{0}$. The three stochastic remedies inject randomness at three different levels: **noisy descent** perturbs the iterate ($\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \alpha \boldsymbol{g}^{(k)} + \boldsymbol{\epsilon}^{(k)}$, σ decaying like $1/k^3$), **SGD** perturbs the gradient (unbiased, $\mathbb{E}[\boldsymbol{g}] = \nabla f$, low variance preferred), and the **cross-entropy method** replaces the point with a whole distribution, refitting $(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ by MLE on the **elite samples** - which is exactly minimizing the **KL divergence** $D_{\mathsf{KL}} = H(g,h) - H(g)$ between proposal and target. CE solved $f = x^2$ to 10^{-6} in five iterations with $\sigma_0^2 = 70$ and **failed at** $\mu = 6.7$ **with** $\sigma_0^2 = 20$ - the whole method lives or dies on **not letting the variance collapse**.

Self-check

1. Prove that exact line search implies $\boldsymbol{g}^{(k+1)T}\boldsymbol{d}^{(k)} = 0$, showing the chain-rule step explicitly.
2. Why does the orthogonality result *fail* when $\alpha^{(k)}$ is not optimal, and what does that failure tell you about the remaining descent along $\boldsymbol{d}^{(k)}$?
3. Write the noisy-descent update. What distribution does $\boldsymbol{\epsilon}^{(k)}$ come from, and why must its standard deviation decay?
4. On $f = x_1^2 - x_2^2$ starting at $(-1.6, 0)$, prove that steepest descent can *never* leave the line $x_2 = 0$.
5. Classify the point $(0,0)$ of $f = (x_1^2 - 1)^2 + x_2^2$ using the Hessian. Why does a first-order method not notice?
6. What condition makes a stochastic gradient estimate valid, and why is low variance desirable?
7. Give two reasons SGD is preferred over full-batch gradient descent for neural network training.
8. Does SGD decrease the objective at every step? Explain what happens with a *constant* step size near the optimum.
9. Compute $I(p)$ for $p = 0.05$ and $p = 0.95$ in bits. Relate the answers to the two cricket matches.
10. Show that the binary entropy $H = -p\log_2 p - (1-p)\log_2(1-p)$ is maximized at $p = 1/2$, and state its value there.
11. State the bounds on H for an N-outcome experiment and say when each is attained.
12. Write $H(\boldsymbol{y}|\boldsymbol{x})$ in terms of joint and marginal entropies, and define mutual information two different ways.
13. Compute $D_{\mathsf{KL}}(g\|h)$ and $D_{\mathsf{KL}}(h\|g)$ for $g = (0.5, 0.5)$, $h = (0.6, 0.4)$. What does the difference demonstrate?
14. Identify the four factors in the CE update rule $\boldsymbol{v}^{(t)} = \arg\max_{\boldsymbol{v}} \frac{1}{N}\sum H(\boldsymbol{x}_i)\frac{f(\boldsymbol{x}_i;\boldsymbol{u})}{f(\boldsymbol{x}_i;\boldsymbol{v}^{(t-1)})}\log f(\boldsymbol{x}_i;\boldsymbol{v})$ and say what each does.
15. Under what three conditions does Step 3 of the CE algorithm have an analytical solution?
16. Show that with $H = \mathbb{I}_{\{\boldsymbol{x}\in A\}}$ and $\boldsymbol{u} = \boldsymbol{v}^{(t-1)}$, the CE update reduces to MLE on the elite samples.
17. Write the MLE updates for $\boldsymbol{\mu}^{(k+1)}$ and $\boldsymbol{\Sigma}^{(k+1)}$. How is N_e selected for a minimization problem?
18. Sketch the sample cloud for $\boldsymbol{\Sigma} = \begin{bmatrix} 1 & 0.9 \\ 0.9 & 1 \end{bmatrix}$ versus $\begin{bmatrix} 0.1 & 0 \\ 0 & 0.1 \end{bmatrix}$. What does each tell the optimizer?
19. The $\sigma_0^2 = 20$ run converged to $\mu = 6.7$ on $f = x^2$. Diagnose the failure precisely and propose two fixes.
20. Why is the Rastrigin function hard for gradient methods but easy for CE? What is CE paying for that advantage?

Appendix A

Revision Reference

A.1 Formulation and Optimality

The modelling template

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad g_i(x) \le 0, \quad h_j(x) = 0.$$

State the decision variables, their domains, the objective, and all constraints. A feasible point satisfies the constraints; an optimal point has the best objective value among all feasible points. A problem can be infeasible, unbounded below, or have a finite infimum that is not attained.

Test at an interior stationary point	Conclusion for a twice continuously differentiable function
$\nabla f(x^*) = 0$	Necessary for a local minimum; also satisfied by maxima and saddles.
$\nabla^2 f(x^*) \succeq 0$	Necessary for a local minimum; a singular Hessian may be inconclusive.
$\nabla^2 f(x^*) \succ 0$	Sufficient for a strict local minimum.
$\nabla^2 f(x^*) \prec 0$	Sufficient for a strict local maximum.
Indefinite Hessian	Saddle point.
Convex differentiable f	Every stationary point is a global minimizer on an unconstrained convex domain.

Scope of the derivative tests

These are unconstrained, interior-point tests. A boundary optimum need not have zero gradient. A zero Hessian does not mean a flat function: compare x^4, $-x^4$, and x^3 at the origin.

A.2 Directions and Step Sizes

The shared iteration

$$x_{k+1} = x_k + \alpha_k d_k, \qquad \phi_k(\alpha) = f(x_k + \alpha d_k).$$

A smooth descent direction satisfies $g_k^T d_k < 0$, where $g_k = \nabla f(x_k)$. A sufficiently small positive step then decreases the objective.

Wolfe conditions: $0 < c_1 < c_2 < 1$

$$
\begin{aligned}
f(x_k + \alpha d_k) &\le f(x_k) + c_1 \alpha g_k^T d_k && \text{(Armijo)},\\
\nabla f(x_k + \alpha d_k)^T d_k &\ge c_2 g_k^T d_k && \text{(curvature)},\\
\left|\nabla f(x_k + \alpha d_k)^T d_k\right| &\le c_2 |g_k^T d_k| && \text{(strong curvature)}.
\end{aligned}
$$

Backtracking with Armijo alone ensures sufficient decrease; it does not automatically enforce the curvature condition.

Scalar search

Bracketing finds an interval around a local minimum. Golden-section and Fibonacci search reduce a unimodal interval using function values. Bisection finds a root of a continuous function with a sign-changing bracket; for optimization, apply it to f' and check the resulting stationary point.

A.3 Deterministic Update Rules

Steepest descent and Newton

$$d_k = -g_k \quad \text{(steepest descent)}, \qquad H_k d_k = -g_k \quad \text{(Newton)}, \quad H_k = \nabla^2 f(x_k).$$

Solve the Newton linear system rather than explicitly computing an inverse. A positive-definite Hessian makes the Newton direction a descent direction; an indefinite Hessian does not provide that guarantee.

Conjugate gradient on an SPD quadratic

For $f(x) = \frac{1}{2}x^T Ax - b^T x + c$, with $A \succ 0$:

$$g_k = Ax_k - b, \qquad \alpha_k = -\frac{g_k^T d_k}{d_k^T A d_k}, \qquad d_{k+1} = -g_{k+1} + \beta_k d_k.$$

$$\beta_k^{\text{FR}} = \frac{g_{k+1}^T g_{k+1}}{g_k^T g_k}, \qquad \beta_k^{\text{PR}} = \frac{g_{k+1}^T (g_{k+1} - g_k)}{g_k^T g_k}.$$

Finite termination in at most n steps assumes exact arithmetic and the quadratic setting. Nonlinear conjugate-gradient methods need line-search and restart safeguards.

Inverse-Hessian quasi-Newton updates

Write $s_k = x_{k+1} - x_k$, $y_k = g_{k+1} - g_k$, and let Q_k approximate H_k^{-1}:

$$d_k = -Q_k g_k, \qquad Q_{k+1} y_k = s_k.$$

$$Q_{k+1}^{\text{DFP}} = Q_k + \frac{s_k s_k^T}{s_k^T y_k} - \frac{Q_k y_k y_k^T Q_k}{y_k^T Q_k y_k},$$
$$Q_{k+1}^{\text{BFGS}} = (I - \rho_k s_k y_k^T) Q_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T, \quad \rho_k = (y_k^T s_k)^{-1}.$$

Positive curvature $y_k^T s_k > 0$ helps preserve positive definiteness. L-BFGS stores a limited history of (s_k, y_k) pairs.

Trust-region agreement

$$\min_{\|s\| \le \Delta_k} m_k(s), \qquad r_k = \frac{f(x_k) - f(x_k + s_k)}{m_k(0) - m_k(s_k)}.$$

Poor agreement calls for a smaller region; good agreement supports accepting the step and possibly enlarging the region. Evaluate the ratio only when the predicted reduction is positive.

A.4 Adaptive and Stochastic Methods

Momentum and Adam

One consistent momentum convention is

$$v_k = \beta v_{k-1} + g_k, \qquad x_{k+1} = x_k - \alpha v_k.$$

Adam maintains first and second moments, with elementwise squares and division:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t, & v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^{\odot 2}, \\ \widehat{m}_t &= m_t/(1-\beta_1^t), & \widehat{v}_t &= v_t/(1-\beta_2^t), \\ x_{t+1} &= x_t - \alpha\, \widehat{m}_t/(\sqrt{\widehat{v}_t} + \varepsilon). \end{aligned}$$

Three sources of randomness

$$\begin{aligned} x_{k+1} &= x_k - \alpha_k g_k + \eta_k && \text{(noisy descent)}, \\ x_{k+1} &= x_k - \alpha_k \widehat{g}_k, \quad \mathbb{E}[\widehat{g}_k \mid x_k] = g_k && \text{(unbiased SGD)}. \end{aligned}$$

Cross-entropy search instead samples candidate points from a distribution. For a Gaussian family and elite set E of size N_e:

$$\mu_{k+1} = \frac{1}{N_e} \sum_{x \in E} x, \qquad \Sigma_{k+1} = \frac{1}{N_e} \sum_{x \in E} (x - \mu_{k+1})(x - \mu_{k+1})^T.$$

Use the lowest objective values as elites for minimization. Premature covariance collapse can prevent further exploration.

Information-theory identities

$$\begin{aligned} I(x) &= -\log_2 p(x), & H(X) &= -\sum_x p(x) \log_2 p(x), \\ H(X,Y) &= H(X) + H(Y \mid X), & I(X;Y) &= H(X) - H(X \mid Y), \\ D_{\mathrm{KL}}(p\|q) &= \sum_x p(x) \log \frac{p(x)}{q(x)}. \end{aligned}$$

KL divergence is generally asymmetric. Cross-entropy is $H(p,q) = H(p) + D_{\mathrm{KL}}(p\|q)$ when the same logarithm base is used throughout.

Chapter Summary

1. Formulate the problem before choosing a solver.
2. Check assumptions: smoothness, convexity, unimodality, positive definiteness, and feasibility.
3. Separate the direction rule, the step-size rule, and the stopping rule.
4. Inspect gradient norms, objective changes, step sizes, and solver status together.
5. A small step, a stalled objective, or one successful run does not certify global optimality.